

Universidad ORT Uruguay

Facultad de Ingeniería

Un enfoque para el análisis causal de defectos de software conducido por ontologías

Entregado como requisito para la obtención del título de
Maestría en Ingeniería (por investigación)

Autor:

Ing. Santiago Arreche (128132)

Tutor:

Ing. Santiago Matalonga

- 2011 -

Abstract

Defect causal analysis (DCA) is a means of identifying causes of defects used by software organizations in order to taking actions to prevent them from occurring in the future. An ontology is a formal explicit specification of shared conceptualization of a domain of interest. This work presents an approach at applying ontologies for causal analysis activities.

A systematic review (protocol and results) was conducted in order to identify which tools are available to support causal analysis activities. None of the tools identified in the review proposes a learning approach to provide management of the knowledge acquired during the DCA activities.

Assuming that causal relationships are ontological, we propose an approach the application of ontologies to DCA in order to help software organizations with the adoption of DCA activities, while also enhancing improvements opportunities managing cause and effect knowledge.

We understand that the application of ontologies to the software defect causal analysis helps software organizations in the adoption of DCA activities. Our proposal merges the ability to model causal relationships in a knowledge base, with a common vocabulary for sharing causal information. And, in combination with other information systems (software), this mix should improve causal analysis capabilities.

As a result of the research process, an ontology is presented. This ontology complies with the purpose of offering a potential solution to the proposed approach. Entities within the ontology were obtained from the analysis of characteristics of each of the DCA tools reviewed. Furthermore, a set of competency questions are presented which also fulfill the purpose of guiding the design of the ontology.

Ontology development process was carried out following Ontology 101, an iterative ontology development process by Stanford University. The ontology was formalized using OWL.

The representation capabilities of the designed ontology and her applicability to the DCA domain were validated in two stages. The first stage took place during the execution of the development methodology, based on data obtained from a laboratory experiment. In the second stage, with 66 defects provided by an IT company in Uruguay.

As a contribution to the community the ontology developed was published in the Stanford public repository for ontologies.

Resumen

Se presenta un enfoque para el análisis causal de defectos de software conducido por ontologías. El análisis causal de defectos es el medio por el cual las organizaciones productoras de software analizan sus defectos en busca de identificar las causas y así promover acciones para que no vuelvan a ocurrir en el futuro. Las ontologías son una forma de especificar formalmente el conocimiento en un dominio de interés.

Se realiza una revisión sistemática (protocolo y resultados) con el objetivo de identificar cuáles son las herramientas disponibles para dar soporte a las actividades de análisis causal. Ninguna de las herramientas identificadas con la revisión propone un enfoque de aprendizaje que facilite la gestión del conocimiento adquirido durante las actividades de DCA que realice una organización de software.

Aceptando que las relaciones causales son de naturaleza ontológica, la propuesta es brindar un marco para la aplicación de ontologías al DCA que facilite a las organizaciones de software la adopción de actividades de DCA, potenciando las oportunidades de mejoras de sus procesos de producción.

Entendemos que la aplicación de ontologías al análisis causal de defectos de software, ayuda a las organizaciones productoras de software en la adopción de actividades de DCA. A partir del modelado de relaciones causales en una base de conocimiento que a través de un vocabulario común, y en combinación con otros sistemas de información (software) permite mejorar las capacidades de análisis.

Como resultado del proceso se presenta la ontología que cumple con la finalidad de brindar una potencial solución al enfoque propuesto. Para identificar el dominio que debe modelar la ontología se combinan las entidades (deducidas tras analizar las características de representación de las herramientas disponibles para DCA) con un conjunto preguntas de competencia (que además cumplen la función de guiar el diseño de la ontología).

La ontología fue desarrollada con Protégé. El desarrollo consta de dos iteraciones utilizando el proceso Ontology 101 de Stanford y la formalización escrita con OWL.

La capacidad de representación de la ontología diseñada y su aplicabilidad al dominio del DCA, es evaluada en dos etapas. La primera etapa se realizó durante la ejecución de la metodología de desarrollo, a partir de datos obtenidos de una experiencia de laboratorio. En la segunda etapa, a partir de 66 defectos cedidos por una empresa IT uruguaya.

Como aporte a la comunidad se publica en el repositorio público de ontologías de Stanford la ontología desarrollada.

Índice

1. Introducción	5
1.1 El problema	5
1.2 Objetivo	6
1.3 Resumen del trabajo de tesis	7
1.4 Contribución de la tesis	8
2. Estado del arte	9
2.1 Marco conceptual	9
2.2 Metodología de investigación	13
2.2.1 Aspectos de una Revisión Sistemática	13
2.2.2 El proceso de revisión	14
2.2.3 Resultados de la revisión	16
2.3 Herramientas y Técnicas aplicadas al análisis causal	18
2.3.1 Tradicionales	18
2.3.2 No tradicionales	19
2.3.3 Solución de problemas	20
2.3.4 Discusión: Técnicas en ingeniería de software	21
2.4 Relaciones probabilísticas y relaciones causales	22
2.5 Aspectos de Ontologías	25
2.5.1 Definiciones de ontología	25
2.5.2 Clasificación de ontologías	26
2.5.3 Metodología de diseño de ontologías	28
2.5.4 Lenguajes para representar ontologías	31
2.5.5 Herramientas para el desarrollo de ontologías	32
3. Planteamiento: un enfoque para el análisis causal de defectos de software conducido por ontologías	33
3.1 Enfoque ontológico y el análisis causal de defectos	33
4. Resolución	35
4.1 Las herramientas para análisis causal tienen la misma capacidad de expresión	35
4.1.1 Presentación del problema	36
4.1.2 Ejecución del ejercicio	37
4.1.3 Interpretación de los resultados del ejercicio	40
4.2 Diseño de ontologías	41
4.2.1 Iteración 1: Ontología de alto nivel (top-level)	41
4.2.2 Iteración 2: Ontología de dominio (domain-level)	46
4.2.3 Formalización de la ontología	52
5. Validación de la ontología	53
5.1 Descripción del juego de datos	53
5.2 Modelado de los datos en la ontología	54
5.3 Preguntas de competencia	58
5.3.1 Método de resolución	59
5.3.2 Ejemplo de resolución	59
5.3.3 Resumen	65
6. Conclusiones y futuras líneas de investigación	66
6.1 Futuras líneas de investigación	68
7. Referencias	71
Anexo I: Formalización de la ontología (OWL)	75

1. Introducción

1.1 El problema

Gestionar defectos es una tarea compleja para muchas organizaciones productoras de software (Mizukami, 2007; Bezerra, Albuquerque *et al.*, 2008). No todas las organizaciones realizan un análisis de los defectos e inclusive, cuando este análisis se realiza, usualmente no presenta los mecanismos adecuados para aprender de los problemas principales y las causas raíces. Como resultado, los problemas no son resueltos completamente o son empeorados debido al re trabajo, redundando finalmente en el incremento de costos asociados a la calidad (Bezerra, Albuquerque *et al.*, 2008).

Todo emprendimiento de mejora continua requiere del análisis y la identificación de las debilidades, problemas y oportunidades de mejora de los procesos organizacionales (Bezerra, Albuquerque *et al.*, 2008).

En particular, el análisis causal es utilizado por las organizaciones para identificar las causas de los defectos a fin de tomar las acciones más adecuadas para prevenir su futura ocurrencia (Kalinowski, Travassos *et al.*, 2008b). Actividades relacionadas con análisis causal son consideradas en diferentes modelos de mejora de procesos tales como CMMI(SEI, 2006), ISO/IEC 12207 (ISO, 2004), y Six Sigma (Pyzdek, 2003).

Este trabajo de tesis se centra en las *organizaciones de software*, entendiendo como *organizaciones de software* a todas aquellas organizaciones productoras de software.

El análisis causal de defectos (en más DCA por las siglas en inglés para *Defect Causal Analysis*) es la aplicación de actividades de análisis causal a un tipo de problema en particular, a saber: defectos introducidos en activos de proceso de producción de una organización de software durante el ciclo de vida del software (Card, 1993).

Las actividades de DCA facilitan la identificación de oportunidades de mejora para los activos del proceso de la organización y la implementación de acciones para prevenir la ocurrencia del defecto en el futuro (Kalinowski, Travassos *et al.*, 2008b). Entendiéndose como activo de proceso a todos aquellos artefactos desarrollados a partir del proceso de producción de una organización productora de software (SEI, 2006).

Las dificultades que se presentan con la gestión de los defectos hacen que las organizaciones de software tengan problemas en la adopción de modelos de mejora que guíen la implementación de actividades de DCA en sus proyectos de software.

1.2 Objetivo

El análisis causal es una de las actividades fundamentales para las iniciativas de mejora de procesos (Card, 2006). Sin embargo, no está siendo aplicado de forma eficaz por las organizaciones de software (Kalinowski y Travassos, 2008; Kalinowski, Mendes *et al.*, 2010).

Pretendemos entonces identificar oportunidades de mejora que faciliten a las organizaciones de software la adopción de DCA en sus procesos.

El objetivo del presente trabajo es proveer a las organizaciones de software un mecanismo de soporte al proceso de DCA, que a su vez fomente la captura y reutilización del conocimiento causal adquirido durante la ejecución del proceso.

Para promover la captura y reutilización del conocimiento causal se propone el diseño y desarrollo de una ontología.

La aplicación de ontologías al dominio del DCA brinda un marco para la representación de conocimiento, este hecho facilitaría a las organizaciones de software el proceso de compartir y reutilizar el conocimiento causal.

En cuanto al proceso de DCA, la combinación de conocimiento administrado por la ontología propuesta y las capacidades de inferencia sobre el modelo facilitan que las hipótesis de causalidad se identifiquen a partir del análisis de información histórica adquirida durante el proceso de producción de una organización de software.

Es entonces que con este trabajo, se plantea brindar a las organizaciones de software un marco para la aplicación de un enfoque para DCA conducido por las ontologías (enfoque ontológico).

1.3 Resumen del trabajo de tesis

Para comprender cuáles son las oportunidades de mejora en cuanto al proceso de DCA que se pueden ofrecer dentro del ámbito de este trabajo se debe conocer cuál es el significado de los términos relacionados con este concepto.

Motivado por la convicción de la existencia de claras oportunidades de mejora en cómo las organizaciones de software están realizando las actividades de DCA, se presenta una revisión sistemática (en adelante SR por las siglas en inglés para Systematic Review) respecto de las herramientas de soporte al análisis causal.

La [sección 2](#) presenta el estado del arte describiendo el marco conceptual en el ámbito de este trabajo, el proceso de investigación y los principales hallazgos.

A partir de los principales hallazgos de la SR es posible identificar diferentes tipos de herramientas, entre las cuales se destacan los diagramas de Ishikawa como la técnica más ampliamente utilizada en ingeniería de software (IS) para actividades de DCA.

El análisis del conjunto de técnicas identificadas como resultado de la revisión motiva una evolución del problema de investigación, llevando a considerar que tal vez el aspecto de mejora sea el concepto de causalidad involucrado en dichas herramientas.

Dadas las características de las relaciones causales, éstas responderían a un enfoque ontológico por sobre uno epistemológico. Sin embargo, ninguna de las herramientas identificadas durante la SR sugiere la utilización de ontologías como una posible alternativa a las herramientas actuales.

Este planteamiento motiva una nueva evolución en el problema de investigación. La propuesta ya no es investigar las herramientas de soporte al DCA, sino que es un enfoque para actividades DCA en organizaciones de software conducido por las ontologías (enfoque ontológico) que facilite a las organizaciones la adopción de actividades de DCA en sus procesos de producción.

El desarrollo de la propuesta para la aplicación del enfoque ontológico al DCA y su resolución, tipificada por el diseño de la ontología, se presentan al detalle durante la [sección 3](#) y [sección 4](#) respectivamente.

Al respecto de la ontología diseñada como parte de la solución, la [sección 5](#) describe el escenario de validación al cual la ontología fue sometida al finalizar la ejecución del proceso de la metodología de diseño. La validación utiliza como entrada un conjunto de datos cedidos por una organización de software para defectos registrados en 7 proyectos realizados en 2010 para clientes extranjeros.

Finalmente la [sección 6](#) presenta el resumen de la investigación y las principales líneas para trabajos futuros.

1.4 Contribución de la tesis

Las contribuciones de esta tesis están compuestas por los siguientes aspectos:

- ❖ *Una revisión sistemática del estado del arte de las herramientas de DCA*

Se presenta una revisión sistemática cuyo propósito es la conducción de una revisión del estado del arte en DCA, en términos de las herramientas que las organizaciones de software utilizan para soportar las actividades de DCA en sus procesos.

- ❖ *Un enfoque ontológico para DCA*

A partir de la revisión se ha constatado diferencias entre relaciones causales y las relaciones probabilísticas. Entendemos que estas diferencias no han sido evidentes en el contexto de la ingeniería de software. Estas diferencias mostrarán que las relaciones causales son de naturaleza ontológica. Por lo cual se propone un enfoque ontológico para DCA.

- ❖ *Una oportunidad para mejorar la gestión del conocimiento en DCA*

Se propone la aplicación de ontologías como alternativa a las herramientas de soporte al DCA identificadas durante la revisión. La aplicación de ontologías además motiva la mejora en la gestión y administración del conocimiento que las organizaciones de software generen durante el proceso.

- ❖ *Una ontología para la gestión del conocimiento causal*

Se presenta el desarrollo de una ontología capaz de representar el conjunto de conceptualizaciones compartidas en el dominio de DCA, de forma de hacer de estas conceptualizaciones una representación formal y explícita.

- ❖ *Un escenario de validación para la ontología*

Se presenta el proceso de validación para la ontología diseñada. El proceso de validación consiste en evaluar la capacidad de representación e inferencia de la ontología diseñada a partir de información de defectos y las causas identificadas para ellos durante el proceso de DCA de una organización de software en particular. El proceso de validación se realiza en dos etapas: en la primera etapa se utilizan como entrada datos de laboratorio, mientras que para la segunda fase se utilizan datos cedidos por una organización de software.

- ❖ *Una ontología para compartir con la comunidad*

Se ha publicado la ontología resultante de esta investigación en el sitio institucional de Stanford dedicado a proyectos que utilizan Protégé para desarrollar y validar ontologías. Esto facilitará la evolución de la ontología con la colaboración de la comunidad conforme avanza el conocimiento respecto de DCA. <http://protege.cim3.net/cgi-bin/wiki.pl?ProjectsThatUseProtegeLtoQ> (fecha último acceso 24/05/2011)

2. Estado del arte

Esta sección presenta el estado del arte del DCA en términos de herramientas para su implementación. Se realiza una SR con el objetivo de identificar cómo las organizaciones de software están realizando las actividades relacionadas con DCA en sus procesos.

Para conocer cuál es el marco conceptual en el cual se ubica esta investigación, en la [sección 2.1](#) se revisa la definición de los diferentes términos involucrados con DCA.

El proceso de revisión así como los principales hallazgos identificados se presentan en la sección [2.2](#) y [2.3](#) respectivamente. A partir de los hallazgos una discusión de su aplicación en ingeniería de software es presentada con la [sección 2.3.4](#).

De la discusión planteada en la [sección 2.4](#) al respecto de los conceptos de *asociación* y *causalidad*, surge la oportunidad de diseñar una ontología de soporte al DCA ya que las relaciones causales responden a un enfoque ontológico.

El concepto de ontología en el contexto de este trabajo de tesis y los diferentes esquemas a partir de los cuales las ontologías pueden ser clasificadas, se detallan en la sección [2.5.1](#) y [2.5.2](#) respectivamente.

En cuanto al proceso de ingeniería ontológica, la sección [2.5.3](#) presenta una introducción a las diferentes metodologías para el desarrollo de ontologías disponibles, profundizando en particular en la metodología de desarrollo de Stanford seleccionada para ser aplicada el contexto de este trabajo. Los lenguajes y las herramientas disponibles para el desarrollo de ontologías se presentan en las secciones [2.5.4](#) y [2.5.5](#) respectivamente.

2.1 Marco conceptual

El análisis causal es utilizado por las organizaciones para identificar las causas de los defectos a fin de tomar las acciones más adecuadas de forma de prevenir que ellos no vuelvan a ocurrir (Kalinowski, Travassos *et al.*, 2008b).

El análisis causal se enfoca en comprender las relaciones causa-efecto. Un sistema causal es un conjunto de causas (eventos y condiciones) que interactúan, produciendo efectos (consecuencias) reconocibles (Chillarege, Bhandri *et al.*, 1992; Card, 2006). El análisis causal es la investigación sistemática de sistemas causales para identificar las acciones que los influyen, usualmente con el objetivo de minimizar consecuencias indeseables (Card, 2006).

El DCA representa la aplicación de actividades de análisis causal a un tipo de problema en particular: defectos introducidos en activos del proceso de producción de una organización productora de software durante el ciclo de vida del software (Card, 1993).

El DCA busca comprender las relaciones entre los defectos introducidos durante el ciclo de vida del software y sus causas, de forma de minimizar futuras consecuencias indeseables. Se puede apreciar que el DCA opera a partir de tres conceptos elementales: *Causa*, *Defecto* y la *Relación* entre estos.

¿Qué es un Defecto?

De la bibliografía se pueden extraer diferentes definiciones para el término de defecto, pero las definiciones predominantes se dan fundamentalmente desde dos perspectivas: la de los clientes y la de la ingeniería de software.

El estándar IEEE 610.12-1990 (IEEE, 1990), por ejemplo, aporta una visión desde la perspectiva de la ingeniería de software que propone la existencia de diferentes acepciones para el término de acuerdo con la circunstancia en la cual ha sido detectado:

- Cuando es detectado en una revisión por pares este es relacionado con una *falta* o *defecto* (del término en inglés *fault*) en el artefacto que está siendo revisado.
- Cuando es encontrado durante actividades relacionadas con etapas de pruebas, es relacionado con una *falla* (del término en inglés *failure*) en el producto de software que está siendo probado.
- Por otra parte un *error* (del término en inglés *error*) puede verse como una acción humana que produce un resultado incorrecto, esto implica por ejemplo una acción realizada por un programador u operador.
- La vinculación entre los conceptos anteriores comienza con la existencia de una falla en un producto de software, esta representa un síntoma de la existencia de un defecto en los artefactos de software. Un error en piezas de software puede ser el causante de provocar uno o más defectos.

Metodologías de mejora de procesos como ISO/IEC 12207 (ISO, 2004) y Six Sigma (Pyzdek, 2003) aportan una visión desde la perspectiva del cliente a partir de la cual asocian el término defecto con incumplimiento de requisitos o necesidades. En particular Six Sigma define el término defecto como cualquier evento que provoque un incumplimiento en cuanto a las necesidades del cliente (Eckes, 2000).

Mientras que ISO/IEC 12207 lo define como incumplimiento de un *requisito* (necesidad o expectativa establecida, generalmente implícita u obligatoria) asociado a un uso previsto o específico (ISO, 2004).

El Project Management Institute (PMI), a través de la guía para el cuerpo del conocimiento de la gestión de proyectos (A Guide to the Project Management Body of Knowledge - PMBOK), define a un defecto como una imperfección o deficiencia en un componente de un proyecto, que hace que dicho componente no cumpla con sus requisitos o especificaciones y deba ser reparado o reemplazado (PMI, 2008).

En el contexto de este trabajo de tesis, se adopta la acepción del término más cercana a la perspectiva de la ingeniería, particularmente el término *Defecto* es utilizado para representar faltas (*fault*) revelados mediante actividades de verificación de software.

Por otro lado, existen diferentes clasificaciones que se pueden utilizar para tipificar defectos desde diferentes perspectivas (Jalote y Agrawal, 2005; Kalinowski y Travassos, 2008; Kalinowski, Travassos *et al.*, 2008b).

Resumir completamente la larga variedad de investigaciones al respecto de las diferencias entre los defectos y su naturaleza no es factible. Pero se han elegido tres ejemplos para comprender cómo funcionan los sistemas de clasificación a partir de diferentes dimensiones del defecto.

El IEEE Std. 1044-1993 (IEEE, 1993) define diferentes dimensiones para los defectos, considerando por ejemplo: las actividades del proceso y las fases en la cual ha sido detectado, la causa sugerida, tipo de defecto, la fuente, entre otros atributos.

De acuerdo con (Wagner, 2008) la tipificación más utilizada es la clasificación ortogonal de defectos de IBM (ODC) (Chillarege, Bhandri *et al.*, 1992). En esta clasificación los defectos son clasificados a partir de lo que se conoce de ellos para ello utiliza las siguientes dimensiones: tipo de defecto, fuente, impacto, disparador, fase y severidad.

Una clasificación similar a ODC es la clasificación Modos, Origen y Tipo de Defectos de HP (Grady, 1992). Las dimensiones propuestas son aquellas que aparecen en el nombre, el origen es la fuente del defecto, el tipo es una categorización para los defectos y el modo determina el efecto.

De acuerdo con el argumento presentado en (Wagner, 2008; Kalinowski, Mendes *et al.*, 2010) las clasificaciones generales son difíciles de utilizar en la práctica y necesitan ser adaptadas al dominio y ambiente específicos de una organización. De esta forma la tipificación de defectos que se utilice será creada para dar soporte a los intereses específicos de análisis en actividades de DCA de la organización.

¿Qué es una relación?

Analizando la bibliografía y en particular los artículos (Holland, 1986; Eckes, 2000; Juristo y Moreno, 2001; Card, 2004, 2006), es posible concluir que no existe una definición consensuada para el término *relación* y que a su vez sea propia del contexto de las relaciones causa-efecto. La acepción de la palabra *relación* que más se ajusta a este contexto es aquella que proviene del término en inglés *relation*.

En este sentido una relación es una conexión lógica o natural, que resulta de la interacción y/o dependencia entre un conjunto de eventos y condiciones (Farlex, 2010).

Los diferentes niveles de relación que se pueden encontrar están dados por el grado de entendimiento que se ha adquirido respecto de la interacción entre eventos y condiciones en el sistema causal (Juristo y Moreno, 2001).

De acuerdo con (Juristo y Moreno, 2001) existen diferentes tipos de relaciones según el conocimiento que se tenga del fenómeno:

- **Relaciones descriptivas:** Cuando la relación entre las variables es desconocida pero es posible describir ciertos patrones de comportamiento a partir de la observación.
- **Relaciones probabilísticas:** Cuando la relación puede ser explicada mediante una medida o una función, permite observar la correlación entre los fenómenos pero no permite distinguir una relación causal.

- **Relación causal:** Cuando se conoce completamente el fenómeno de influencia, de forma que se puede afirmar que la variación del efecto se da siempre a través de los componentes que lo causan y sólo éstos.

A partir de lo anterior la *Causalidad* se presenta en dos niveles: *determinista* y *probabilística* (Holland, 1986; Juristo y Moreno, 2001).

La causalidad determinista implica que cada vez que se invoque una causa en particular se obtenga el efecto esperado (Holland, 1986).

Mientras que la causalidad probabilística implica una probabilidad de que exista la relación entre la causa y el efecto entre los fenómenos de estudio (Juristo y Moreno, 2001).

¿Qué es una Causa?

Desde la perspectiva de los sistemas causales, el término *Causa* responde a aquellos eventos y condiciones que contribuyen con la ocurrencia de un problema. Un problema es una situación particular que genera consecuencias indeseables (Card, 2004).

Mientras que una *Causa* desde una perspectiva experimental de la ingeniería de software, responde a una variable que cuando es invocada bajo determinada configuración siempre se va a manifestar el efecto que esta provoca (Juristo y Moreno, 2001).

Existen diferentes esquemas de clasificación para las causas. De hecho la revisión que se incluye en (Kalinowski y Travassos, 2008) presenta un listado donde reúne las clasificaciones que se pueden encontrar, como por ejemplo la clasificación propuesta por Ishikawa para manufactura (Ishikawa, 1976), la propuesta por el estándar IEEE 1044 para clasificación de anomalías (IEEE, 1993), la propuesta de Robitalle (Kalinowski, Travassos *et al.*, 2008a) que agrega la categoría de *Entorno* a las categorías del esquema original de Ishikawa, y la categoría de Jacobs (Kalinowski, Travassos *et al.*, 2008a) que al esquema de Ishikawa agrega la categoría de *Organización*.

La búsqueda de causas a problemas es un comportamiento normal del ser humano, sin embargo en ciertas ocasiones entender la causalidad como un hecho intuitivo provoca que el análisis sea realizado tan solo porque sí (Card, 2006).

Por ejemplo, un error común es buscar causalidad entre eventos que ocurren de forma simultánea (Card, 2004). Considerar la situación en la cual se analiza la relación entre el tiempo invertido en revisión y los defectos encontrados. Estos son dos aspectos que frecuentemente nos mostraran una alta correlación, además es intuitivo que es necesario invertir tiempo para descubrir defectos. Sin embargo invertirlo a tiempo no asegura que se descubran los defectos. Por lo cual la relación entre estos dos aspectos es necesaria pero no es causal (Card, 2006).

El problema que está detrás de la confusión que generan estas situaciones, es qué no siempre se comprende exactamente qué se busca cuando se busca por una *causa*. Lo cual redundan en que las instancias de análisis dentro del proceso de DCA no produzcan el beneficio esperado (Card, 1993, 2004, 2006).

De acuerdo con (Card, 2006), serían tres las condiciones esenciales para poder intuir la existencia de causalidad entre dos fenómenos de estudio:

1. Debe existir al menos una relación de asociación o de correlación entre la causa y el efecto establecidos en la hipótesis.
2. La causa debe preceder temporalmente al efecto.
3. Debe ser posible identificar el mecanismo que vincula la supuesta causa-efecto.

Estas tres condiciones anteriores, permiten conducir la línea de pensamiento disminuyendo el riesgo de arribar a conclusiones erróneas.

Las organizaciones de software deben ser capaces de combinar estos conceptos de forma de poder analizar sus defectos, las causas de estos y así encontrar las acciones correctivas pertinentes.

En las siguientes sub-secciones se profundiza el análisis acerca de cuáles son las herramientas y técnicas a disposición de las organizaciones de software para este propósito.

2.2 Metodología de investigación

En esta sección se presenta el proceso de revisión y los principales hallazgos para la SR que se ejecuta con el objetivo de identificar cómo las organizaciones de software están realizando las actividades relacionadas con DCA en sus procesos.

2.2.1 Aspectos de una Revisión Sistemática

Con la finalidad de llevar a cabo una SR se aplica una metodología de investigación basada en las guías para realizar SR en ingeniería de software de Kitchenham (Kitchenham, 2007). Una SR permite aplicar un enfoque sistemático a la investigación y provee de valor científico a los resultados.

En esta sección, se expone una breve descripción al respecto de cada uno de los conceptos involucrados en el proceso de SR.

Son varios los autores que han propuesto metodologías para realizar SR en ingeniería de software (Dyba, Kampenes *et al.*, 2006; Jørgensen y Shepperd, 2007; Vicente, Delamaro *et al.*, 2008), sin embargo ninguno de estos artículos presenta una propuesta con el nivel de detalle y la rigurosidad con la cual es presentada la metodología en (Kitchenham, 2007).

Por tal motivo y considerando el antecedente de adaptación de las guías para realizar una SR presentado en (Mian, Conte *et al.*, 2005), es que se decide adaptar las guías para realizar una SR propuestas por Kitchenham (Kitchenham, 2007) para realizar la investigación.

De acuerdo con Kitchenham, una SR es una forma de identificar, evaluar e interpretar todo el material de investigación relevante a una pregunta de investigación, tópico o fenómeno de interés. Las revisiones sistemáticas tienen por objetivo presentar una evaluación razonable de un tema de investigación mediante el uso de una metodología fiable, rigurosa y auditable (Kitchenham, 2007).

El enfoque de realización resume las etapas en una SR en tres fases principales: Planificación de la revisión, conducción de la revisión y reporte de revisión (Kitchenham, 2007).

En la fase de planificación deben definirse los objetivos para la revisión y la forma en que la revisión será realizada, esto incluye formular preguntas de investigación y planificar cómo la identificación de fuentes y el proceso de selección de estudios se realizará (Kitchenham, 2007).

En la fase de conducción se ejecuta la búsqueda sobre las fuentes identificadas y los estudios obtenidos deben ser evaluados de acuerdo al conjunto de criterios definidos. Finalmente, la información relevante respecto de las preguntas de investigación planificadas debe ser extraída desde los estudios seleccionados (Kitchenham, 2007).

La fase de reporte es la fase final e implica redactar los resultados de la revisión y la planificación de los mecanismos de difusión de forma de poder comunicar los resultados entre potenciales interesados (Kitchenham, 2007).

2.2.2 El proceso de revisión

La metodología utilizada para la SR es una adaptación de las guías para realizar una SR en ingeniería de software (Kitchenham, 2007) presentadas en la sección anterior. Aunque en (Kitchenham, 2007) no se menciona explícitamente como adaptar su metodología, en (Mian, Conte *et al.*, 2005), describen una adaptación para estas guías.

Tomando (Mian, Conte *et al.*, 2005) como precedente, se adaptó el proceso de SR. La adaptación resultante conserva la integridad de las guías originales, mientras facilita sortear las restricciones del presente proyecto de investigación.

El propósito es la conducción de una revisión de estado del arte en DCA, en términos de las herramientas que las organizaciones software utilizan para soportar las actividades de DCA en sus procesos. Con tal propósito la Tabla 1 presenta la pregunta de investigación que se ha formulado:

Tabla 1 - Formalización de preguntas de investigación

Objetivo	Identificar las herramientas utilizadas para realizar análisis causal.
Problema	El análisis causal pese a ser una de las actividades más importantes de los procesos de mejora no está siendo aplicada de forma eficaz por las organizaciones software.
Pregunta	¿Cuáles son las principales herramientas identificadas por el cuerpo de conocimiento para proveer soporte operativo a las actividades relacionadas con DCA?
Keyword y sinónimos	Se presentan en la Tabla 2
Intervención	Se observaran las herramientas para análisis causal identificadas buscando por oportunidades de mejora para las organizaciones software
Efecto	Antecedentes en DCA para la administración del conocimiento
Medidas	Características de las herramientas identificadas
Población	Herramientas utilizadas para análisis causal por organizaciones software
Aplicación	Mejora en la eficacia y eficiencia de los procesos de análisis causal en organizaciones software con emprendimientos de mejora de procesos

La Tabla 2 resume la lista de palabras clave, derivadas a partir de la pregunta de investigación.

Tabla 2 - Palabras clave para la revisión

Defect Classification	Orthogonal Classification	Causal analysis Techniques
Root-Cause Analysis(RCA)	Defect Prevention	Cognitive Map
Causal Analysis	Software Quality Improvement	Defect Detection
Defect Causal Analysis(DCA)	Total Quality Management	Problem- Solving Techniques
Inference Cognitive Map	Cause Effect	Mental Map
Mental Model	Causal Analysis Methods	Software Engineering
Ontology	Ontologies	

Las bibliotecas digitales seleccionadas se presentan en la Tabla 3.

Tabla 3 - Criterio de selección de fuentes

Criterio de selección	Se utilizan bases de datos electrónicas indexadas
Lenguaje de fuentes	En idioma original, sin traducir. (de preferencia Inglés)
Método de búsqueda	Búsqueda manual utilizando las keyword presentadas en Tabla 2.
Cadena de búsqueda	Se combinan las keyword presentadas en Tabla 2.
Lista inicial de fuentes	IEEE Xplore, ACM Digital library, Google Scholar, Proquest, JStor, SpringerLink, CiteSeer library, Scopus.

Los criterios para exclusión utilizados seleccionar los artículos incluidos en la revisión son presentados en la Tabla 4:

Tabla 4 - Selección de estudios

Criterio de inclusión	El artículo presenta conceptos relacionados con DCA.
	El artículo presenta algún tipo de metodología, proceso, técnica o herramienta relacionada con la implementación de DCA.
	El artículo presenta nuevas teorías, aplicaciones novedosas, u otros aspectos relacionados con herramientas para apoyar la implementación de DCA.
Criterio de exclusión	Título duplicados en el conjunto de artículos seleccionados
	Título o resumen (abstract) no está relacionado con DCA
	El artículo en cuestión no ha sido publicado
Tipos de estudio	Estudios cuantitativos y cualitativos
Procedimiento de selección	Se seleccionan los estudios que cumplen los criterios indicados y han sido recuperados desde las fuentes presentadas en Tabla 3.
Selección inicial	Comprendido por 107 artículos recuperados tras las búsqueda realizada según los criterios, keywords y bases de datos seleccionados
Validación de calidad	Para conformar la primera selección se toman aquellos estudios que no cumplen con los criterios de exclusión. Posteriormente son estudiados en su texto completo y sometidos a los criterios de inclusión definiendo la selección final.
Revisión de la selección	De la selección inicial se revisan nuevamente los estudios seleccionados a fin de verificar si no se han descartado estudios que puedan ser utilizados durante la selección final.

2.2.3 Resultados de la revisión

Como resultado del proceso de selección, son seleccionados 50 artículos para así realizar un análisis más detallado. Para cada uno de estos 50 artículos se obtiene el texto completo y es valorado nuevamente bajo los criterios de inclusión/exclusión presentados antes. Como consecuencia 30 artículos son seleccionados para ser utilizados como entrada para la etapa de extracción y síntesis de datos.

La Figura 1 enseña la distribución de los estudios al final de cada etapa de la fase de conducción.

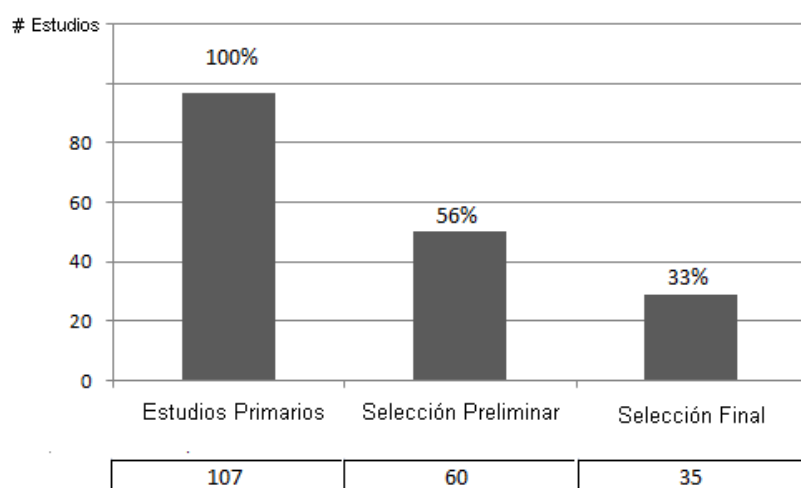


Figura 1: Selección final de artículos

La Figura 2 muestra la distribución de los estudios incluidos en la selección final según librería digital de origen.



Figura 2: Estudios primarios según fuente

La información extraída de cada artículo, permite analizarlos desde diferentes perspectivas. Una de las perspectivas es la evolución de los artículos según año de publicación, que se presenta en la Figura 3.

Esta perspectiva muestra como el análisis causal ha tenido un flujo sostenido de publicaciones en los últimos años, lo cual da indicios de que se trata de un tema que aun genera interés entre los investigadores.

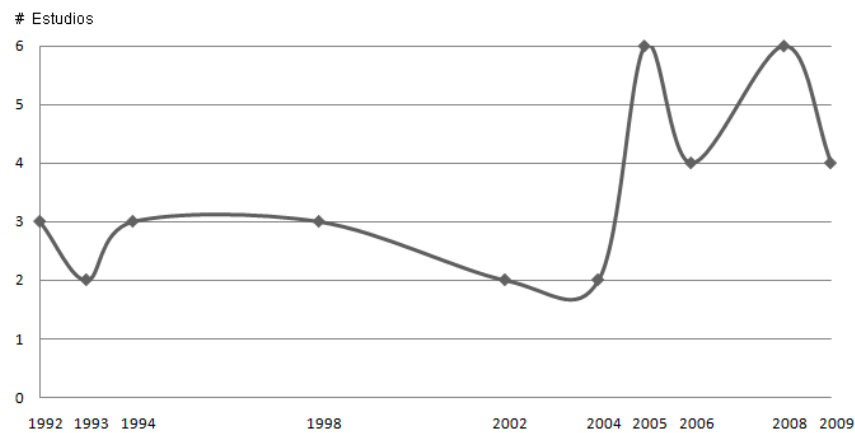


Figura 3: Evolución de artículos según año de publicación

Los artículos revisados sugieren que hay distintos tipos de herramientas y técnicas utilizados para realizar análisis causal.

El 37 % de los artículos analizados muestran que los diagramas causa-efecto (Ishikawa, 1976) es la técnica más ampliamente usada como soporte a DCA en el software. Por otra parte, se encuentran un conjunto más complejo de técnicas con diferencias interesantes entre sus capacidades.

El 60 % de los artículos (18 en total) analizados describen otras técnicas (un total de 13) para realizar análisis causal y sus *antecedentes* (cualquier aplicación o propuesta de aplicación) en el contexto de la ingeniería de software.

Del total de las técnicas identificadas: 3 han sido utilizadas para realizar análisis causal en otras áreas de conocimiento pero aun no han sido aplicadas en el contexto de ingeniería de software, 5 han sido propuestas de aplicación y finalmente las 5 restantes cuentan con antecedentes de aplicación en ingeniería de software.

2.3 Herramientas y Técnicas aplicadas al análisis causal

A continuación se resumen los principales hallazgos identificados a partir de los resultados de la revisión. Para comprender las interrelaciones entre el conjunto de herramientas se diseña una taxonomía, que a su vez colabore con la identificación de posibles huecos (*gaps*) en el conocimiento.

Para obtener la clasificación que se adapte a este propósito, se aplica un proceso de tres pasos propuesto por (Vegas, Juristo *et al.*, 2009):

- 1) Identificar los criterios subyacentes de clasificación;
- 2) Buscar y analizar las fuentes de información;
- 3) Instanciar los criterios de clasificación con las técnicas seleccionadas.

El resultado de este proceso es la clasificación de las técnicas seleccionadas en las siguientes categorías:

1. Aquellas técnicas que tienen amplia aceptación entre las organizaciones de software como soporte a las actividades de DCA en sus procesos de producción de software, son clasificadas como *Tradicionales*.
2. Las técnicas que fueron propuestas en alguno de los estudios como una alternativa a las técnicas tradicionales, son clasificadas como *No-Tradicionales*.
3. Finalmente, se clasifican como *Solución de Problemas* (del término en inglés Problem-Solving (Oslapas, 1993)) aquellas técnicas que han sido utilizadas para análisis causal en otras áreas de conocimiento, pero aun no han sido aplicadas en ingeniería de software.

2.3.1 Tradicionales

En el contexto de las técnicas tradicionales, los artículos analizados muestran que los diagramas causa-efecto (Ishikawa, 1976) son la técnica más ampliamente utilizada para la implementación de actividades de DCA en la industria del software.

Estos diagramas ayudan a las organizaciones de software en la búsqueda de las causas para clases específicas de defectos y es propuesta por (Ishikawa, 1976; Chillarege, Bhandri *et al.*, 1992; Card, 1993, 1998; VanMoll, Jacobs *et al.*, 2002; Jalote y Agrawal, 2005; Buglione, 2006; Card, 2006; Kalinowski, Travassos *et al.*, 2008b; Matalonga y Feliu, 2008; Wagner, 2008).

2.3.2 No tradicionales

Mapas Mentales (*Mind Mapping*) es una técnica desarrollada por Tony Buzan desarrollada para rastrear y registrar pensamientos. Debido a que el cerebro procesa información a alta velocidad es imposible documentar e incluso recordar todos los pensamientos e ideas que van surgiendo. Entonces ya que el ser humano habla en palabras y piensa en imágenes, esta técnica incorpora ambas en el proceso mientras intenta documentar todos los pensamientos que sea posible (Buzan y Buzan, 1994; Buzan, 2002). En (Buglione, 2008) se presenta una propuesta para la aplicación de Mapas Mentales a la ingeniería de software.

Pensamiento sistémico (*Systemic thinking/system dynamics*) fue desarrollada por Jay Forrester et. al. del MIT. Esta técnica ilustra las interrelaciones dentro de un sistema, basado en diferentes objetos que forman parte del sistema. Así pueden ser identificadas varias relaciones causa-efecto con el efecto de cada ciclo dentro del sistema (Senge, 2004). En el pensamiento sistémico cualquier elemento (variable) en una situación, se pueden trazar flechas (eslabones) que representan la influencia de un elemento sobre otro. A su vez estos revelan ciclos que se repiten una y otra vez, mostrando como varían las situaciones. Por lo tanto en el pensamiento sistémico, a diferencia de otras herramientas de diseño lineal se admiten ciclos. En el contexto de la ingeniería de software, esta técnica fue utilizada fue utilizada para explicar cómo los procesos de software funcionan (van Oorschot, K. et al., 2009).

Mapas Cognitivos (*Cognitive Mapping*) es una técnica fundada sobre la teoría de constructos personales de George Kelly. Esta técnica está basada en conocimiento causal. Su objetivo es la representación de conceptos y relaciones causales. La representación de esta técnica está dada por grafos (Ackermann, Eden et al., 1992; Chaib-draa y Desharnais, 1998). Los mapas cognitivos son en una forma un modelo gráfico y cualitativo cuyo foco está en las relaciones causales abstractas entre conceptos. Son utilizados como soporte a los procesos de adquisición de conocimiento y toma de decisiones (Wellman, 1994).

Modelo de Ecuaciones Estructurales (*SEM* por las siglas en inglés para Structural Equation Model) es una técnica fundamentada sobre la teoría estadística. En esta técnica información de causa-efecto cualitativa es combinada con datos estadísticos para proveer de valoración cuantitativa de las relaciones causa-efecto entre las variables de interés (Pearl, 2000, 2009). En el contexto de ingeniería de software (Shah y Goldstein, 2005; Itabashi y Ochimizu, 2008), SEM propone un enfoque cuantitativo para predecir debilidades específicas de las organizaciones respecto del proceso de software basado en las relaciones causa-efecto entre áreas del proceso que pueden tener efecto sobre el resultado y las medidas resultantes en el proceso específico de una organización de software.

Modelos Gráficos (*Graphic Model*) es una técnica fundada sobre la unión entre la teoría de probabilidad y la teoría de grafos. Así las relaciones causa-efecto dentro de un modelo causal son significativas estadísticamente cuando la técnica es utilizada bajo ciertas condiciones específicas (Lauritzen, 1996; Sander y Pearl, 2006). En el contexto de la ingeniería de software (Itabashi y Ochimizu, 2008), los modelos gráficos son utilizados (como parte de SEM) para la predicción de debilidades específicas de los procesos de software de una organización basado en el estudio de las relaciones causa-efecto.

Modelo Causal Estructural (*SCM* por las siglas en inglés para Structural Causal Model) es una técnica creada por Judea Pearl y desarrollada para ser aplicada en ciencias cognitivas y

ciencias sociales. La técnica combina muchas de las ventajas de los modelos causales tales como: SEM y modelos gráficos (Pearl, 2000, 2009). En el contexto del software, SCM ha sido aplicado para el análisis de causa-efecto en proyectos de desarrollo de inteligencia artificial (AI) (Pearl, 2009).

Análisis de Causa Raíz (*Causal Path Analysis*) es una técnica desarrollada y descrita por Simon, Blalock et. al. (Bento y Bento, 2004) se trata de una herramienta simple y muy poderosa basada en mapas cognitivos para explorar relaciones causa-efecto entre variables, utilizada en el contexto de ingeniería de software como parte del análisis de relaciones causa-efecto en proyectos de AI (Bento y Bento, 2004).

Análisis de Gráfico Radial (*Radial Analysis Chart*) esta técnica es en el contexto de ingeniería de software, una propuesta para extender los diagramas radiales (Radial Charts) para el análisis de defectos de forma de ser utilizadas para DCA, en donde cada banda radial representa un elemento al cual puede atribuirse un defecto (Henderson, 2008).

Modelo Basado en Reglas (*Rule-based Models*) se trata de una forma común de modelo computacional. Básicamente es un conjunto de cláusulas denominadas reglas las cuales codifican conocimiento acerca de los fenómenos que están siendo modelados (Reformat y Igbide, 2005; Siau y Tan, 2005). En el contexto de ingeniería de software, esta técnica es aplicada como parte de un sistema de predicción de defectos multi-modelo (Reformat y Igbide, 2005).

Teoría Relacional para Mapas Causales (*Relational Theory for Causal Maps*) básicamente es un grafo dirigido que representa afirmaciones individuales sobre creencias respecto a su entorno (Chaib-draa y Desharnais, 1998). En el contexto de ingeniería de software, este tipo de enfoques es utilizado habitualmente en proyectos de AI y es aplicable únicamente en el caso de grafo sin ciclos (Chaib-draa y Desharnais, 1998).

2.3.3 Solución de problemas

Analogía Simbólica (*Symbolic Analogy*) es una técnica creada por Joel Whitcamp y utiliza analogía, metáforas y similares con el objetivo de desarrollar un punto de vista fresco y único del dominio. Siendo que las imágenes rigen la memoria, esta técnica comienza con imágenes genéricas las cuales son abstraídas simbólicamente y eventualmente pueden conducir hacia una nueva solución (Oslapas, 1993).

Conexiones Forzadas (*Forced Connections*) esta técnica comienza listando atributos del sujeto que está siendo estudiado. Luego se listan tantas alternativas debajo del atributo como sea posible. Finalmente, se escoge un ítem de cada lista y se combinan en una nueva forma. Crear implica no solamente obtener algo nuevo desde ningún lugar, sino que se relaciona con la organización u combinación de aquellas cosas que ya están allí en nuevas e imaginativas formas (Oslapas, 1993).

Manipulación de Verbos (*Manipulative verbs*) se trata de una técnica creada por Alex Osborne, esta técnica es un método que utiliza una serie de verbos con el objetivo de visualizar un sujeto de estudio de una forma nueva u original. Este método utiliza verbos porque sugieren que el sujeto está siendo manipulado (Oslapas, 1993).

2.3.4 Discusión: Técnicas en ingeniería de software

Con base en los resultados de la SR, se identifican un conjunto de técnicas de soporte a la realización de actividades de análisis causal en el ámbito del software. Estas son listadas en la Tabla 5.

Tabla 5 – Hallazgos, su aplicación y sus correspondientes referencias

<i>Aplicación</i>	<i>Técnica</i>	<i>Referencias</i>
Mejora de procesos	Diagramas causa-efecto	(Ishikawa, 1976; Chillarege, Bhandri <i>et al.</i> , 1992; Card, 1993, 1998; VanMoll, Jacobs <i>et al.</i> , 2002; Jalote y Agrawal, 2005; Buglione, 2006; Card, 2006; Kalinowski, Travassos <i>et al.</i> , 2008b; Matalonga y Feliu, 2008; Wagner, 2008)
	Mapas Mentales	(Buzan y Buzan, 1994; Buzan, 2002; Buglione, 2008)
	Pensamiento Sistémico	(Senge, 2004; van Oorschot, K. <i>et al.</i> , 2009)
	Análisis de Causa Raíz	(Bento y Bento, 2004)
	Análisis de Gráfico Radiales	(Henderson, 2008)
Inteligencia Artificial	Modelos Gráficos	(Lauritzen, 1996; Sander y Pearl, 2006; Itabashi y Ochimizu, 2008)
	Modelo Causal Estructural (SCM)	(Pearl, 2000, 2009)
	Mapas Cognitivos	(Ackermann, Eden <i>et al.</i> , 1992; Wellman, 1994; Chaib-draa y Desharnais, 1998)
	Modelo de Ecuaciones Estructurales (SEM)	(Pearl, 2000; Shah y Goldstein, 2005; Itabashi y Ochimizu, 2008; Pearl, 2009)
	Modelos Basados en Reglas	(Reformat y Igbide, 2005)
	Teoría Relacional de Mapas Causales	(Chaib-draa y Desharnais, 1998)

Todas las herramientas listadas en la Tabla 5 han sido utilizadas para mejorar los procesos de toma de decisiones de una organización y así hacer de ellos procesos más eficaces. El matiz está en que se han utilizando en dos dimensiones de análisis diferentes: aquéllas utilizadas en el contexto de la mejora de procesos de software y aquéllas utilizadas en el contexto de proyectos de inteligencia artificial.

En el conjunto de herramientas que fueron utilizadas para la mejora de la toma de decisiones en el contexto de la mejora de procesos de software, se ubican: Diagramas causa-efecto de Ishikawa, Mapas Mentales, Pensamiento sistémico, Análisis de Causa Raíz, Análisis de Gráfico Radiales.

Mientras que en el conjunto de herramientas que han sido utilizadas para la mejor de la toma de decisiones en el contexto del desarrollo de proyectos de AI, se ubican: Mapas Cognitivos, SEM, Modelos Gráficos, SCM, Modelos Basados en Reglas, Teoría Relacional de Mapas Causales.

La clasificación permite apreciar diferencias entre las técnicas, de forma tal que es posible identificar al grupo de técnicas utilizadas en AI como el de aquellas técnicas que son capaces de inducir relaciones causales. Mientras que el grupo de técnicas aplicadas a la mejora de los procesos de software, está formado por técnicas que son capaces de inducir relaciones probabilísticas.

Por lo tanto existen diferencias conceptuales respecto del concepto de causalidad involucrado por las herramientas en el marco de DCA, y entendemos que no han sido evidentes en el contexto de ingeniería de software.

Entendemos que las diferencias no han sido evidentes dado el concepto de causalidad involucrado en las herramientas para DCA aplicadas en IS. Las herramientas no proponen capacidades que permitan evaluar al defecto en su contexto y también conocer como éste varia su comportamiento bajo condiciones cambiantes.

La siguiente sección presenta una discusión al respecto de la importancia de estas diferencias en lo que respecta a DCA, a partir de la cual se muestra por qué los conceptos involucrados en relaciones causales y relaciones probabilísticas son diferentes.

2.4 Relaciones probabilísticas y relaciones causales

A partir del análisis realizado sobre los hallazgos identificados durante la SR se identifican dos grupos de técnicas: aquéllas que operan sobre relaciones de tipo causal y por otra parte aquéllas que operan sobre relaciones probabilísticas.

Como ha sido mencionado en la [sección 2.1](#), el concepto subyacente es la causalidad.

La causalidad probabilística (*Asociación*) (Pearl, 2000) es el área de conocimiento que busca explicar las relaciones causales en términos de relaciones probabilísticas. Una relación probabilística es una relación entre variables descubierta mediante la aplicación de análisis estadístico tradicional. Estas relaciones han sido ampliamente utilizadas en el contexto de ingeniería de software (Dyba, Kampenes *et al.*, 2006). El objetivo del análisis de relaciones probabilísticas es inferir parámetros de distribución a partir de muestras tomadas de una población. Con la ayuda de tales parámetros, se pueden inferir asociaciones entre variables, estimar la probabilidad de eventos pasados y futuros, así como actualizar la probabilidad de los acontecimientos a la luz de nuevas pruebas o nuevas mediciones (Pearl, 2000).

La causalidad determinista (*Causalidad*) tipificada por relaciones causales deben además de inferir probabilidades en condiciones *estáticas* (analizar variables en su contexto), hacerlo además en condiciones *dinámicas* (estimular las variables para analizar la variación de la relación) bajo condiciones cambiantes (Pearl, 2000).

La Tabla 6 muestra las diferencias entre ambas corrientes de causalidad (Pearl, 2000).

Tabla 6 - Diferencias entre Causalidad y Asociación

	Asociación	Causalidad
Objetivo	Evaluar parámetros sobre una distribución a partir de las muestras tomadas sobre esa distribución. Con la ayuda de estos parámetros se podría inferir asociaciones entre variables, estimando esperanzas o probabilidades sobre eventos pasados o futuros, así como también actualizar probabilidades a partir de nuevas observaciones (Pearl, 2009)	Debe ir todavía un paso más allá que la Asociación. Además de inferir esperanzas o probabilidades en condiciones estáticas, debe inferir también en condiciones dinámicas bajo condiciones cambiantes (Pearl, 2009)
Conceptos	Correlación, regresión, dependencia, independencia condicional, probabilidad, marginalidad (Pearl, 2009).	Aleatoriedad, influencia, efecto, confusión, variables instrumentales, intervención, falsa correlación (Pearl, 2009).
Notación	Notación estadística tradicional (Pearl, 2009)	Complementar la notación estadística con vocabulario propio de causalidad de forma de diferenciar claramente hipótesis de causalidad de las de asociación (Pearl, 2009)
Inferencia causal	No existe forma a partir de las funciones de distribución que pueda indicar firmemente que la distribución cambiará si cambian las condiciones, porque las leyes de probabilidad no dictan cómo una propiedad de una distribución debe cambiar cuando otra variable es afectada (Pearl, 2009)	<i>correlación no implica causalidad</i> , no es posible identificar relaciones de causalidad simplemente con la correlación para una población, detrás de cada conclusión causal debería de poder hallarse alguna hipótesis causal que no es comprobable a partir de estudios de observación (Pearl, 2009)
Relación causal	Una relación que puede ser establecida en términos de distribución conjunta de variables observadas (Pearl, 2009)	Una relación que no puede ser definida únicamente a partir de la distribución (Pearl, 2009)

Esta distinción implica que los conceptos probabilísticos y los causales difieren; las relaciones probabilísticas caracterizan condiciones estáticas, mientras que las relaciones causales se ocupan de condiciones cambiantes (Pearl, 2009). Estas últimas, describen objetivamente las restricciones físicas en nuestro mundo, mientras las relaciones probabilísticas son epistémicas, reflejando lo que sabemos o creemos sobre nuestro mundo (Pearl, 2000).

De acuerdo con (Pearl, 2000) comprender cómo actúa la causalidad permite ver cómo las relaciones causales son más estables que las relaciones probabilísticas. Esa diferencia de estabilidad es esperada porque las relaciones causales son *ontológicas (observación realizada en (Pearl, 2000))* ya que describen objetivamente restricciones físicas al dominio de interés facilitando la observación del fenómeno en condiciones cambiantes.

Todas las técnicas identificadas en el entorno de análisis causal representan conocimiento relacionado con el mismo dominio de interés. Su objetivo es determinar si existe una influencia causal entre los conceptos involucrados en un proceso. Pero no se encontraron técnicas que describan restricciones físicas al dominio de interés.

En el caso de DCA en organizaciones de software, el contexto donde se presenta el defecto cambia de acuerdo con diferentes aspectos inherentes a los procesos de producción. Por ejemplo: la organización de software, el proyecto en el que se origina el defecto, la capacidad de los recursos involucrados en el proyecto.

Contar con capacidades de representación de restricciones al DCA en organizaciones de software permite analizar el defecto en su contexto, mejorando el conocimiento sobre el defecto y las causas que lo originan. Por ejemplo, cuando una organización de software tras analizar sus defectos determina que la causa que lidera los defectos es la distracción durante la

construcción del software. Cuando no se cuenta con información del contexto en el cual se han originado los defectos analizados, el conocimiento obtenido no es extrapolable a otras organizaciones de software ni a otros proyectos de la propia organización.

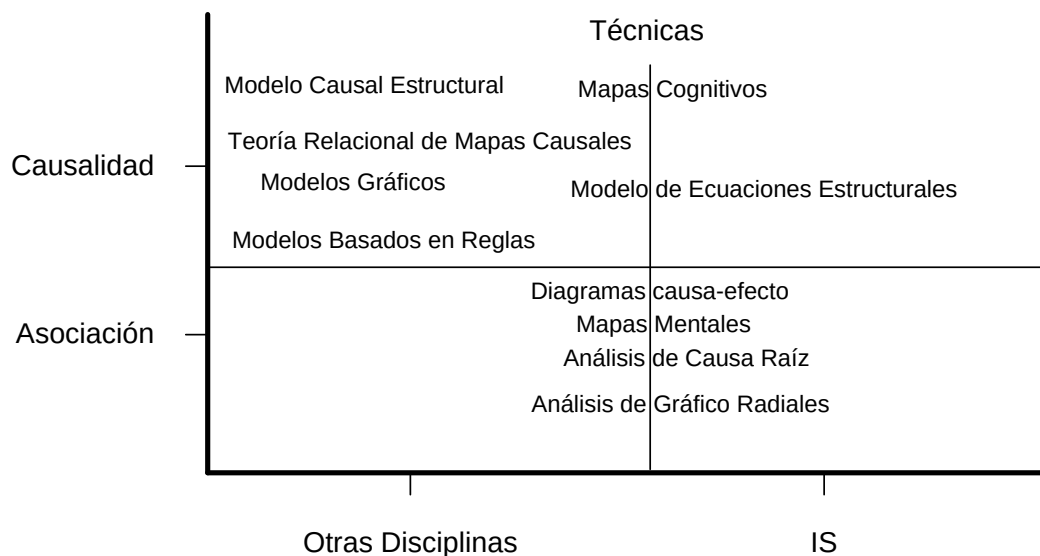


Figura 4 - Técnicas según tipo de relación causal y su aplicación en IS

Por lo tanto entendemos que el concepto de causalidad incluido en las herramientas utilizadas tradicionalmente en DCA es insuficiente para las organizaciones de software, debido a que las herramientas no permiten modelar el defecto en su contexto bajo condiciones cambiantes. Contar con la capacidad de modelar al defecto en condiciones cambiantes permitiría mejorar el conocimiento sobre el defecto y el contexto en el cual se presenta.

A partir de la SR realizada la Figura 4, muestra que tan sólo son dos los antecedentes de aplicación de técnicas de causalidad a IS. Allí se encuentran la aplicación de SEM (Itabashi y Ochimizu, 2008) y el enfoque para DCA aplicando un SCM utilizando para ello redes bayesianas (grafo dirigido acíclicos, asociado a un modelo de probabilidad) (Kalinowski, Mendes *et al.*, 2010).

La propuesta de aplicar SEM aporta un método cuantitativo para analizar la causalidad entre áreas de un proceso. Sin embargo no introduce ningún mecanismo de aprendizaje al proceso. Mientras que el enfoque para DCA aplicando redes bayesianas introduce mecanismos de aprendizaje al proceso, introduce facilidades para el modelado del dominio y utiliza características que mejoran la potencia estadística. Sin embargo no facilita explícitamente el modelado de restricciones al dominio de interés.

Con la SR realizada, no se ha identificado ningún antecedente de aplicación de ontologías al DCA en IS. La aplicación de ontologías aporta un mecanismo de aprendizaje al proceso y la capacidad de modelar restricciones al dominio de DCA. Esto permitirá evaluar al defecto bajo condiciones cambiantes (por ejemplo, modelando características de proyecto y organización en el cual se origina el defecto).

Por lo expuesto, nuestra propuesta se centrará en intentar capturar el conocimiento causal presente en las sesiones de DCA en organizaciones de software mediante la aplicación de Ontologías.

2.5 Aspectos de Ontologías

Para los sistemas informáticos las ontologías son modelos conceptuales de lo que existe en un dominio en particular, codificado en una forma computable por medio de técnicas de representación de conocimiento. En esta sección se presentan diferentes aspectos de la definición de ontologías (Grimm, Hitzler *et al.*, 2007).

2.5.1 Definiciones de ontología

No hay una única definición de lo que es una ontología, y varían según la comunidad y el contexto de aplicación. Sin embargo, la definición dominante respecto de ontologías como elemento computacional es la siguiente:

Una ontología es una especificación formal y explícita de conceptualizaciones compartidas dentro de un dominio específico de interés (Noy y Guinness, 2000; Grimm, Hitzler *et al.*, 2007).

El concepto de *dominio de interés* implica considerar cualquier parte del mundo real o sistema hipotético sobre los cuales se desea representar conocimiento (Grimm, Hitzler *et al.*, 2007). Por otra parte, la definición anterior contempla diferentes características de una ontología que requieren de explicación (Grimm, Hitzler *et al.*, 2007):

- *Formal*: las ontologías se expresan en un lenguaje de representación de conocimiento que provee de una semántica formal. Esto asegura que el conocimiento codificado es computable y se interpreta siguiendo una forma bien definida.
- *Explícita*: las ontologías declaran explícitamente el conocimiento de forma de que sea accesible de forma computable. Aquel conocimiento que no esté explícitamente incluido no será parte de la conceptualización del dominio.
- *Conceptualizaciones*: las ontologías especifican conocimiento de forma conceptual a través de símbolos que representan conceptos y sus relaciones.
- *Compartidas*: formalmente una ontología refleja un acuerdo entre las personas en una comunidad al respecto de las conceptualizaciones en el dominio.
- *Dominio específico*: Las especificaciones en una ontología están limitadas al conocimiento acerca de un dominio de interés en particular.

Los componentes de una ontología son (Noy y Guinness, 2000; Grimm, Hitzler *et al.*, 2007): *conceptos, relaciones e instancias*.

Los conceptos representan categorías ontológicas relevantes para el dominio de interés, mientras que las relaciones conectan los conceptos brindando semántica. Y las instancias identifican objetos concretos e identificables dentro del dominio de interés, particularmente individuos clasificados por los conceptos.

2.5.2 Clasificación de ontologías

De acuerdo con las fuentes analizadas, es posible identificar diferentes clasificaciones para las ontologías:

La clasificación de ontologías en (Sowa, 2001) distingue entre:

- *Ontologías terminológicas*: son aquellas en las cuales las categorías no necesitan ser completamente especificadas por axiomas y definiciones.
- *Ontologías formales*: tienden a ser más pequeñas que las terminológicas pero sus axiomas y definiciones pueden soportar complejas inferencias y cálculos.

Mientras que la clasificación de ontologías en (Heijst, Schreiber *et al.*, 1997) señala que se pueden clasificar las ontologías por la *cantidad* y *tipo* de la estructura de la conceptualización:

- *Ontologías terminológicas*: especifican términos que son usados para representar conocimiento de un dominio.
- *Ontologías de información*: especifican la estructura de los registros de la base de datos.
- *Ontologías de modelado del conocimiento*: especifican conceptualizaciones del conocimiento. Tienen una estructura más rica comparada con las de información.

También de (Heijst, Schreiber *et al.*, 1997) surge la clasificación de ontologías por la *materia* o *tema* de la conceptualización:

- *Ontologías de aplicación*: contienen todas las definiciones que son necesarias para modelar el conocimiento requerido para una aplicación particular. Típicamente son una mezcla de conceptos tomados de las de dominio y las genéricas.
- *Ontologías de dominio*: expresan conceptualizaciones que son específicas para un dominio.
- *Ontologías genéricas*: similar a las de dominio, pero los conceptos que definen son considerados genéricos a varias áreas.
- *Ontologías de representación*: proveen un marco de representación. Las de dominio y genéricas son descriptas usando las primitivas de las ontologías de representación.

Por otra parte la clasificación de ontologías presentada en (Gómez, Juristo *et al.*, 1997):

- *Ontologías dependientes del dominio*: proporcionan un vocabulario sistematizado que tiene por objeto describir los términos empleados en un dominio dado
- *Ontologías dependientes de la tarea*: proporcionan un vocabulario sistematizado con los términos empleados en la resolución de problemas que pueden, o no, ser del mismo dominio, pero que tienen entre sí cierta similitud.

La clasificación en (Grimm, Hitzler *et al.*, 2007) propone una clasificación según el nivel de abstracción de las conceptualizaciones involucradas:

- *Ontologías de alto nivel*: buscan describir conceptos generales y de alto nivel que pueden ser compartidos entre diferentes dominios y aplicaciones.
- *Ontologías de dominio y tareas de dominio*: este tipo de ontologías capturan conocimiento dentro de un dominio específico de interés. Este tipo de ontologías tienen un alcance más concreto y específico que las ontologías de alto nivel. Son ontologías reutilizables en un dominio particular pero no entre diferentes dominios.
- *Ontologías de la tarea*: proporcionan un vocabulario sistematizado de los términos empleados en la resolución de problemas asociados a una serie de tareas que pueden ser, o no, del mismo dominio, pero que tienen entre sí cierta similitud.
- *Ontologías de aplicación*: además de restringir aun más el alcance, las ontologías de aplicación proveen el vocabulario específico requerido para describir una determinada tarea en un contexto particular de aplicación. Normalmente hacen uso de ontologías de dominio y de actividades describiendo el papel que un dominio específico ocupa en una tarea específica.

La Figura 5 muestra la relación entre las categorías de ontologías en (Grimm, Hitzler *et al.*, 2007).

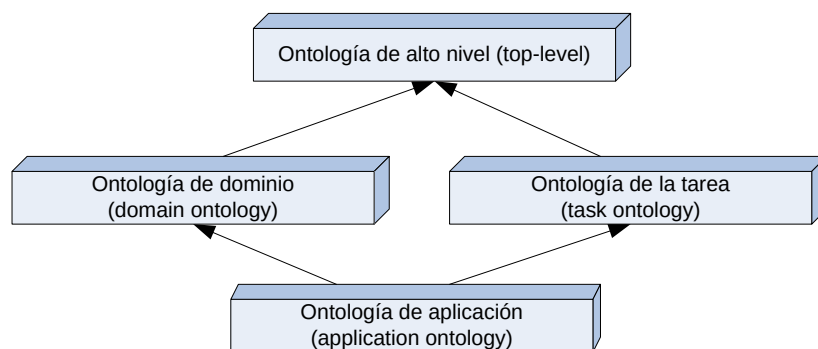


Figura 5 - Tipos de Ontologías, basado en (Grimm, Hitzler *et al.*, 2007)

A los efectos del trabajo de tesis se utiliza la clasificación de ontologías que presentan en (Grimm, Hitzler *et al.*, 2007). Esta clasificación reúne las principales características del resto de las clasificaciones presentadas.

2.5.3 Metodología de diseño de ontologías

Es posible encontrar diferentes recomendaciones y metodologías para el desarrollo de ontologías presentadas como parte de la ingeniería ontológica:

- *Metodología de Uschold y King*: metodología para el diseño de ontologías capaces de modelar procesos empresariales (Uschold y King, 1995; Lopez, 1999).
- *Metodología de Grüninger y Fox*: metodología para el diseño de ontologías capaces de modelar procesos y actividades de un dominio de negocio específico (Lopez, 1999).
- *Enfoque de Amaya Berneras et al.*: trabaja sobre el diseño de ontologías que soporten el reúso de conocimiento en sistemas técnicos muy complejos (KACTUS, 1996; Lopez, 1999).
- *METHONTOLOGY*: framework para el diseño de ontologías que incluye un proceso de desarrollo (Fernández-López, Gómez-Pérez *et al.*, 1999; Lopez, 1999).
- *Metodología basada en SENSUS*: metodología de diseño de ontologías para aplicar en casos es que se requiere procesar lenguaje natural (Swartout, P. *et al.*, 1997; Lopez, 1999).
- *Ontology Development 101*: metodología para el diseño de ontologías que propone iniciar con un esquema básico e irlo evolucionado en sucesivas iteraciones (Noy y Guinness, 2000).

Ninguna de las metodologías revisadas alcanza la madurez ni la aceptación requerida para considerarla como una metodología estándar. Tampoco existe un enfoque unificado sino que cada grupo de diseño ontológico utiliza su propia metodología (Lopez, 1999).

De acuerdo a (Lopez, 1999; Noy y Guinness, 2000), no existe un único camino correcto o una metodología exacta para el desarrollo de ontologías.

Por lo tanto, luego de considerar diferentes opciones se selecciona para el desarrollo de la solución propuesta la metodología para desarrollo de ontologías propuesta por Stanford: *Ontology Development 101* (Noy y Guinness, 2000).

El motivo detrás de la decisión es que se trata de una metodología que propone un proceso iterativo para el desarrollo de ontologías, lo cual permite obtener resultados de forma temprana.

Este enfoque presenta un proceso de siete pasos que se describen a continuación (Noy y Guinness, 2000):

Paso 1. Determinar el dominio y el alcance de la ontología

Los autores sugieren comenzar con el desarrollo de la ontología determinando claramente el dominio y el alcance de la misma.

La recomendación de los autores para determinar el alcance de la ontología es bosquejando la lista de preguntas que la base de conocimiento asociada con la ontología deberá ser capaz de responder.

Estas preguntas reciben el nombre de preguntas de competencia/habilidad (*competency questions*). Las respuestas a esas preguntas pueden cambiar durante el proceso del diseño de la ontología, pero en cualquier momento ayudarán a limitar el alcance del modelo.

Un ejemplo de pregunta de competencia es: *¿cuáles son las causas que lideran un tipo particular de defecto?* donde un tipo de defecto en particular podría ser: *defectos con severidad Alta o Urgente*.

Paso 2. Considerar la reutilización de ontologías ya existentes

Se recomienda luego de haber definido el alcance y el dominio de la ontología, evaluar la posibilidad de reutilización de ontologías existentes, de forma de promover la reutilización de conocimiento.

Para ello existen bases de conocimiento públicas que contienen ontologías que se pueden evaluar para ser utilizadas. El formalismo en el cual una ontología está expresada a menudo no interesa, puesto que muchos sistemas de representación de conocimiento pueden importar y exportar ontologías.

Paso 3. Listar los términos importantes para la ontología

Los autores recomiendan dar un paso previo al diseño de las clases para listar todos los términos clave para el dominio de la ontología, ya que determinan que suele ser de utilidad para mejorar la comunicación de la ontología.

Paso 4. Definir las clases y la herencia entre clases

En esta fase se definen las clases que componen el dominio y las relaciones jerárquicas entre ellas. Los autores destacan tres enfoques principales para el diseño de la jerarquía.

- El enfoque de desarrollo *top-down*, sugiere comenzar por la definición de los conceptos más generales luego proseguir por los consecuentes niveles de especialización.
- Un enfoque *bottom-up*, comienza con la definición de los conceptos más específicas, para luego continuar con el subsecuente agrupamiento de esas clases en conceptos más generales.
- Se prevé un enfoque *combinado*, que se compone de una aleación de las dos alternativas anteriores: primero definimos los conceptos más sobresalientes y luego los generalizamos y especializamos apropiadamente

En relación al diseño de las clases, los autores destacan que todas deben responder a objetos reales o conceptos

Paso 5. Definir las propiedades de cada clase (*Slots*)

Las clases por sí mismas no contienen información suficiente para contestar las preguntas del Paso 1. Una vez que se han definido las clases los autores establecen dedicar esfuerzo a diseñar la estructura interna de los conceptos.

De los conceptos listados en el Paso 3, que no han sido considerados para ser una clase en la ontología probablemente sirvan como una propiedad. Por lo que para cada término en la lista hay que analizar qué clase es descrita por la propiedad que describe.

Hay varios tipos de propiedades de objetos que pueden llegar a ser un *slot* para la ontología:

- Propiedades intrínsecas, estas son específicas del objeto que describe la clase.
- Propiedades extrínsecas, estas son externas al objeto pero le dan contexto.
- Partes que describan al objeto, si es que es un objeto estructurado.
- Relaciones con otros elementos.

Paso 6. Definir las *Facets* (restricciones) de los *Slots*

Los slots definidos para las clases del dominio, pueden tener diferentes *Facets* que describen restricciones para los valores de los slots de una clase en el dominio de discusión.

Las *Facets* describen aspectos de un slot tales como: tipo de valor, valores permitidos, número de valores (*cardinalidad*), entre otros valores que un slot pueda tomar.

Paso 7. Crear Instancias.

El último paso de la metodología establece la creación de instancias para las clases del dominio definido por la ontología diseñada de esta forma comenzar a enriquecer la base de conocimiento que se ha descrito durante el proceso.

El proceso de creación de una instancia individual de una clase requiere: 1) elegir una clase, 2) crear una instancia individual de la clase y 3) completar los valores para los slots.

2.5.4 Lenguajes para representar ontologías

Con el objetivo de que las ontologías estén disponibles para interactuar con los sistemas de información, la ontología debe ser modelada en un lenguaje que pueda ser entendible por un computador. Son varios los lenguajes que han sido desarrollados con tal propósito (Grimm, Hitzler *et al.*, 2007).

En esta sección se presentan aquellos que cuentan con mayor adopción en el contexto de la ingeniería ontológica.

RDF

Resource Description Framework (RDF) (Klyne y Carroll, 2004) es una recomendación de W3C para la representación de metadatos. El modelado de datos en RDF está compuesto de una tripleta que indica que un objeto tiene una propiedad de determinado valor.

Utilizando RDF como esquema se puede especificar un vocabulario que describa el dominio de interés. El formato para la codificación de los documentos RDF es XML.

Sin embargo, este lenguaje tiene la desventaja de ser considerado un lenguaje primitivo útil para el modelado de ontologías simples.

DAML+OIL

Con la falta de poder de expresión que tiene RDF ha surgido la motivación para el desarrollo de DAML+OI (Connolly, Harmelen *et al.*, 2001).

Se trata de un lenguaje basado en RDF, que define un conjunto de clases constructo (un constructo es un objeto conceptual u objeto ideal (Farlex, 2010)) que pueden ser utilizadas para definir clases. Esta propiedad potencia el poder de expresión del lenguaje.

OWL

Ontology WEB Language (OWL) (Patel-Schneider, Hayes *et al.*, 2009) es una adaptación de la W3C de DAML+OIL. Al igual que los lenguajes anteriores es un lenguaje basado en XML. Las diferencias entre OWL y DAML+OIL son mínimas, existen entre ellos una serie de diferencias sintácticas pero semánticamente ambos lenguajes son idénticos.

Se trata del lenguaje más sofisticado, de hecho se puede utilizar en tres niveles: OWL Lite, OWL DL y OW Full. Cada uno de los niveles es un complemento del anterior, a partir de lo cual una ontología modelada con OWL Lite es también válida para OWL DL y OW Full pero no viceversa.

OWL Lite es utilizado cuando la ontología requiere modelar una jerarquía de clases y restricciones simples, basado en lógica descriptiva *SHIF(D)*. Mientras OWL DL es un lenguaje de mayor expresividad que incluye el soporte a clases constructo, basado en lógica descriptiva *SHOIN(D)*. Finalmente OWL Full es la versión del lenguaje que da a los desarrolladores el mayor y más completo poder de expresión de OWL.

2.5.5 Herramientas para el desarrollo de ontologías

Como ha sido presentado durante la sección anterior son varios los lenguajes para modelar ontologías, como consecuencia son varios también las herramientas que se han desarrollado para automatizar la codificación de ontologías a través de estos lenguajes.

Un listado de las diferentes herramientas disponibles es presentado en (Pérez, 2002; Grimm, Hitzler *et al.*, 2007), en esta sección se presentan aquellas herramientas que cuentan con mayor nivel de adopción de acuerdo con una encuesta realizada de forma online a un total de 32 entrevistados durante 2010 (Khondoker y Mueller, 2010).

Protégé

Protégé (Grimm, Hitzler *et al.*, 2007) es una herramienta open-source desarrollada por la Universidad de Stanford con el objetivo de modelar y capturar conocimiento. La herramienta provee un entorno gráfico e interactivo para el diseño de ontologías y el desarrollo de bases de conocimiento. Construido a partir de una arquitectura basada en el desarrollo de plug-in que a diferencia de otras herramientas lo hacen escalable y extensible, Protégé permite a sus usuarios el modelado de clases, slot, facets a través de diferentes herramientas incluidas en el ambiente de desarrollo.

OntoEdit

OntoEdit (Pérez, 2002) es un entorno gráfico de ingeniería ontológica que soporta el desarrollo y mantenimiento de ontologías. Esta herramienta se compone de un conjunto de plug-in que permiten extender su funcionalidad. OntoEdit facilita a sus usuarios el diseño de clases, slot y facets, a partir de diferentes herramientas gráficas incluidas en el ambiente de desarrollo las cuales modelan las diferentes fases del ciclo de ingeniería ontológica.

SWOOP

SWOOP (Kalyanpur, Parsia *et al.*, 2005) es una herramienta web para el modelado de ontologías que trabaja exclusivamente con el lenguaje OWL. La herramienta está basada en el paradigma Model-View-Controller. SWOOP facilita a sus usuarios el modelado de clases, slot y facets mediante un ambiente de desarrollo que emula un navegador (browser) web ya que a través de este paradigma potencia la creencia de que las URI son el concepto central para el entendimiento y el desarrollo de ontologías sobre OWL.

En el contexto de este trabajo de tesis se utilizará Protégé como entorno de desarrollo para la ontología. Para su elección se ha considerado la encuesta realizada durante 2010 (Khondoker y Mueller, 2010) la cual concluye que Protégé (que es utilizado por el 75% de los encuestados) es la herramienta que cuenta con el mayor nivel de adopción (55,5%), el más alto índice de satisfacción en la comunidad (58,8%) y es posible aprender a utilizarla en un corto plazo (según el 64.7% de los encuestados en 1 mes).

Disponible en <http://protege.stanford.edu/>.

3. Planteamiento: un enfoque para el análisis causal de defectos de software conducido por ontologías

Esta sección presenta un marco para la aplicación del enfoque ontológico al DCA. Proponemos la aplicación de ontologías al DCA ya que entendemos que facilitaría a las organizaciones de software la adopción de actividades de DCA en sus procesos, mejorando la gestión del conocimiento involucrado en actividades de DCA.

3.1 Enfoque ontológico y el análisis causal de defectos

La propuesta es brindar un marco para la aplicación de ontologías al DCA que facilite a las organizaciones de software la adopción de actividades de DCA, potenciando las oportunidades de mejoras de sus procesos de producción.

Con tal objetivo, se han identificado una serie de aspectos desde los cuales la aplicación de un enfoque ontológico podría colaborar con las organizaciones de software como facilitador en la adopción de las actividades de DCA en sus procesos de producción.

❖ *Modelado de relaciones causales*

Como ha sido presentado en la [sección 2.1](#), el DCA puede ser visto como un proceso para identificar y analizar relaciones causales entre conceptos, facilitando la identificación de oportunidades de mejora para el proceso organizacional (Kalinowski, Travassos *et al.*, 2008b).

Por otra parte, de lo presentado con la [sección 2.4](#), las relaciones causales por sus características responderían a un enfoque ontológico (Pearl, 2000).

A partir de la combinación de los conceptos anteriores, la aplicación de ontologías como medio para el modelado de relaciones causales, podría servir como instrumento de mejora para potenciar las capacidades de análisis requeridas por las actividades de DCA dado que proponen un esquema formal que brinda capacidades de inferencia sobre el modelo.

❖ *Disponibilidad desde sistemas de información*

Una ontología ([sección 2.5](#)) es una especificación formal y explícita de conceptualizaciones compartidas sobre un dominio de interés (Noy y Guinness, 2000).

Por otra parte, las ontologías son un artefacto computacional que codifica conocimiento respecto del dominio en una forma computable haciéndolo disponible para los sistemas de información (Grimm, Hitzler *et al.*, 2007).

La combinación de los sistemas de información y las conceptualizaciones codificadas en las ontologías pueden generar nuevas oportunidades para una organización, como por ejemplo la obtención de información de mayor calidad, así como también la generación de nuevo conocimiento. Como consecuencia, la combinación de estas entidades actuaría como un instrumento más en la mejora de las capacidades del proceso de la organización.

❖ *Compartir el conocimiento de un dominio con la comunidad*

Las ontologías pueden ser utilizadas como un medio para compartir el conocimiento del dominio con la comunidad (Pearl, 2000). Haciendo explícitos los supuestos que se manejan sobre el dominio y separando además lo que es el conocimiento del dominio del conocimiento operacional (Noy y Guinness, 2000).

El intercambio con la comunidad entorno a DCA permitirá la evolución continua del mapa conceptual (y las suposiciones subyacentes) propuesto por la ontología cuando el conocimiento respecto del dominio de DCA cambie.

❖ *Bases de conocimiento como soporte al mecanismo de aprendizaje*

Mientras que el DCA nos provee de un mecanismo para aprender de los defectos (Card, 1993), las ontologías pueden ser utilizadas para construir bases de conocimiento organizacionales (Noy y Guinness, 2000) que soporten los mecanismos de aprendizaje.

Las ontologías además proveen de un mecanismo de inferencia a partir de las instancias en la base de conocimiento sobre el cual fundamentar la generación de nuevo conocimiento que soporten los mecanismos de aprendizaje en actividades de DCA.

❖ *Vocabulario común como facilitador de la comunicación*

El DCA puede ser visto como un medio para la comunicación de lecciones aprendidas entre equipos de proyecto (Kalinowski, Travassos *et al.*, 2008b). Mientras tanto las ontologías proveen un medio para definir un vocabulario común y compartir el común entendimiento de la estructura de la información en relación con el dominio de interés (Noy y Guinness, 2000).

El lenguaje común queda expresado por las conceptualizaciones compartidas a partir del esquema que define a la ontología, este esquema indica cómo se debe interpretar (o leer) el conocimiento que la ontología administra.

Este lenguaje común podría ser utilizado para estructurar la información que se distribuye entre los diferentes proyectos a partir de los medios de comunicación de lecciones aprendidas

Ninguna de las herramientas de soporte al DCA identificadas durante la SR realizada propone un enfoque de aprendizaje que facilite la gestión del conocimiento adquirido durante las actividades de DCA que realice una organización de software.

En resumen proponemos la aplicación de ontologías al DCA. Entendemos que un enfoque ontológico aplicado a DCA, ayuda a las organizaciones de software en la adopción de actividades de DCA. A partir del modelado de relaciones causales en una base de conocimiento que a través de un vocabulario común, y en combinación con otros sistemas de información (software) permita mejorar las capacidades de análisis. El conocimiento adquirido sobre el dominio de DCA que fuese modelado por la ontología puede ser compartido con la comunidad mejorando el conocimiento sobre el dominio.

4. Resolución

El capítulo anterior presentó una propuesta para la aplicación del enfoque ontológico al DCA. Este capítulo desarrolla la propuesta de tesis para el desarrollo de una ontología que permita capturar el conocimiento causal a partir de una representación formal y explícita de las entidades identificadas en el dominio de DCA.

Con la ontología desarrollada se aporta una potencial solución para el planteamiento de un enfoque ontológico aplicado a DCA.

Si se acepta que las relaciones causales son ontológicas, el dominio que debe representar la ontología se puede aproximar a partir de los conceptos modelados por las herramientas identificadas durante la SR. Para ello se debería aceptar que las herramientas disponibles presentan niveles de abstracción de conceptualizaciones y necesidades de información similares.

Que exista tal similitud entre las herramientas estaría indicando que las herramientas para DCA buscan representar entidades de un mismo dominio. Lo cual sumado a la falta de elementos para modelar restricciones sobre el dominio, que tienen las herramientas potencia la opción de utilizar ontologías como un medio de soporte a las actividades de análisis causal. El análisis de esta observación es prestando en la [sección 4.1](#).

La [sección 4.2](#) profundiza los aspectos relativos a la ejecución del proceso de desarrollo de la ontología, detallando el análisis realizado durante cada una de las fases de la metodología y presentando la ontología al final de cada iteración. Con la primera iteración ([4.2.1](#)) se ha obtenido una ontología de alto nivel, mientras que con la segunda iteración ([4.2.2](#)) se obtiene una ontología de dominio, que presenta un nivel de abstracción más cercano al dominio del DCA.

4.1 Las herramientas para análisis causal tienen la misma capacidad de expresión

De los artículos analizados durante la revisión se observa que las representaciones gráficas utilizadas por las técnicas de DCA presentan similitudes interesantes. Particularmente el artículo (Buglione, 2008) propone explícitamente un método de transformación entre diagramas causa-efecto y mapas mentales.

Con la existencia de tal similitud entre las herramientas será posible modelar en diferentes herramientas el mismo escenario sin necesidad de generar/obtener nueva información. Lo cual implicará que se ha definido un proceso de transformación entre representaciones.

De ser factible encontrar un proceso de transformación, se habrá conseguido una pieza de evidencia sobre la similitud en la capacidad de abstracción de conceptualizaciones entre las representaciones gráficas de diferentes herramientas.

Las siguientes sub-secciones presentan el detalle de la experiencia de verificación realizada a partir de un problema genérico modelado en un diagrama de Ishikawa.

4.1.1 Presentación del problema

Dado el escenario genérico que se presenta en la Figura 6, a partir de un diagrama de Ishikawa. Se propone que es posible representar el mismo escenario propuesto por el ejercicio a través de diferentes herramientas identificadas a partir de la SR realizada, sin que esto signifique obtener mayor información acerca del problema propuesto.

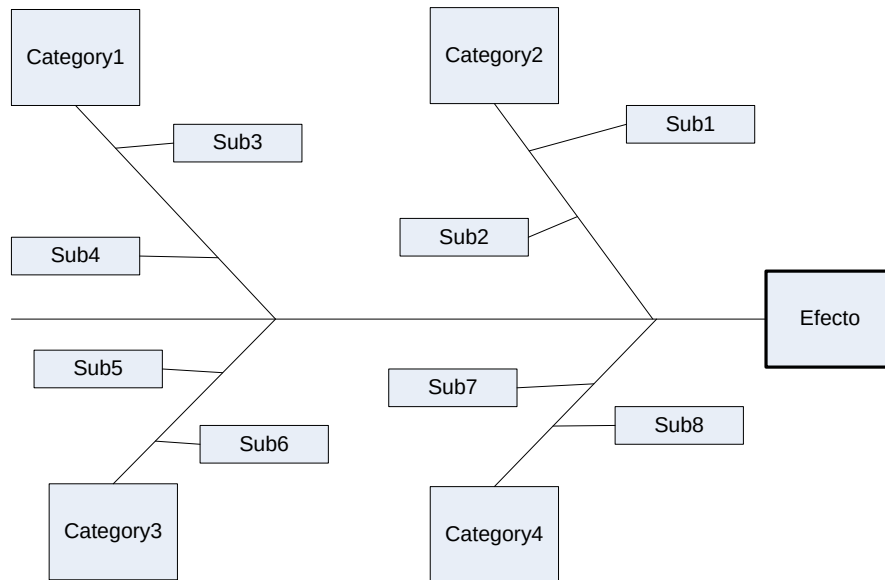


Figura 6 - Problema genérico representado en un diagrama de Ishikawa.

Las herramientas seleccionadas para el ejercicio son las siguientes:

- Mapas mentales
- Modelos gráficos
- Mapas cognitivos
- Modelos de Ecuaciones Estructurales (SEM)
- Modelos causales estructurales (SCM)

4.1.2 Ejecución del ejercicio

Con esta sección se detalla para cada una de las representaciones graficas de las herramientas seleccionadas, un proceso de conversión y su resultado a partir del el cual es posible capturar la información expresada en el diagrama de Ishikawa planteado.

El ejercicio está acotado a aquellos modelos que son lineales y que no admiten ciclos.

Mapa Mental

De acuerdo con (Buzan y Buzan, 1994), los Mapas Mentales pueden ser utilizados como un perfecto sustituto para los diagramas causa-efecto de Ishikawa

El proceso de transformación realizado por (Buglione, 2008) propone la siguiente secuencia de transformación: el efecto a ser analizado se traslada al centro del lienzo ocupando el lugar de concepto central de un mapa mental, mientras las causas principales son ubicadas a los lados del efecto. Finalmente las sub-causas son ubicadas asociadas a cada causa principal correspondiente.

El resultado de la transformación se enseña en la Figura 7.

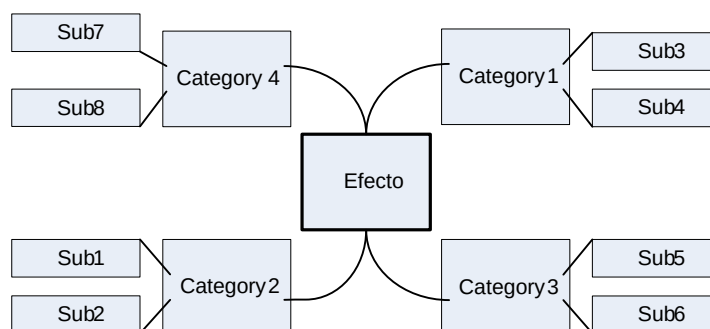


Figura 7 - Representación del problema según mapa mental

Mapa Cognitivo

En un mapa cognitivo, los conceptos son presentados como puntos en el gráfico y las relaciones causales entre ellos se representan con una flecha. Las relaciones causales en este tipo de representaciones pueden tomar valores simples que doten de significado a la relación entre los conceptos (Chaib-draa y Desharnais, 1998).

Considerando entonces el símbolo + como estimulación de causalidad entre conceptos, la secuencia de transformación podría ser: ubicar los conceptos según niveles de izquierda a derecha, considerando que las sub-causas estimulan la causalidad sobre las causas principales y estas estimulan la causalidad sobre el efecto.

El resultado de la transformación se presenta en la Figura 8.

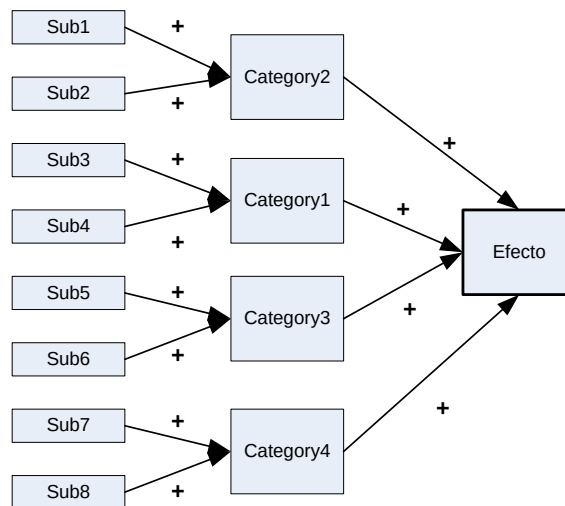


Figura 8 - Representación del problema según mapa cognitivo

Modelos Gráficos

Los modelos gráficos son una herramienta compuesta por la teoría de la probabilidad y la teoría de grafos, que dispone de diferentes perspectivas. Desde la perspectiva de la representación gráfica se trata de grafos en los cuales los nodos representan las variables (conceptos), cuando estos grafos son dirigidos un arco (flecha) entre dos conceptos A y B pueden ser interpretados tal que el concepto A es causa de B (Lauritzen, 1996; Murphy, 2001).

Con ello en mente la secuencia de transformación puede estipularse como un grafo por niveles, con aristas dirigidas siguiendo la secuencia de dependencia desde sub-causas a causas principales y entre causas principales y efectos.

El resultado de la transformación se presenta en la Figura 9.

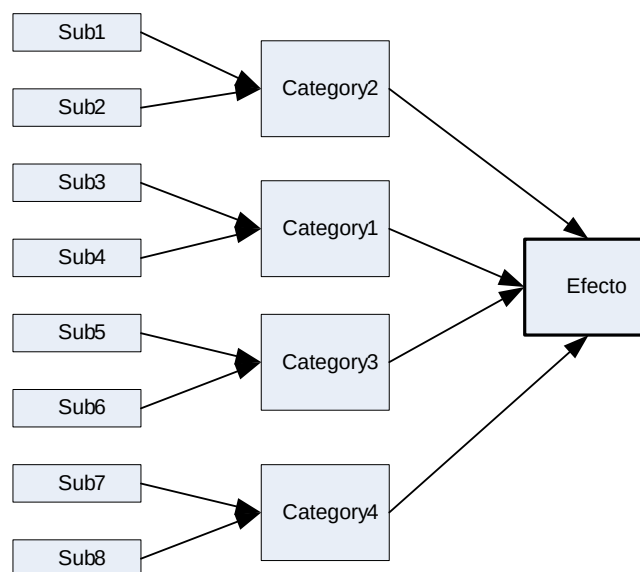


Figura 9 - Problema según modelos gráficos.

Modelos de Ecuaciones Estructurales (SEM)

Se trata de una herramienta que se compone de un modelo de ecuaciones que explican la relación entre los conceptos en un sistema. La perspectiva de representación gráfica propone que existe una flecha entre dos conceptos si hay una expresión en el sistema de ecuaciones que las vincule (Pearl, 2000).

Dado el problema que se busca representar, simplificamos el modelo considerando que las sub-causas responden a variables constantes en el sistema y que existe una función (a los efectos de la demostración no interesa determinar cuál es la función) que representa la existencia de relación causal entre un conjunto de conceptos. Entonces el modelo de ecuaciones y su representación gráfica se muestra en la Figura 10.

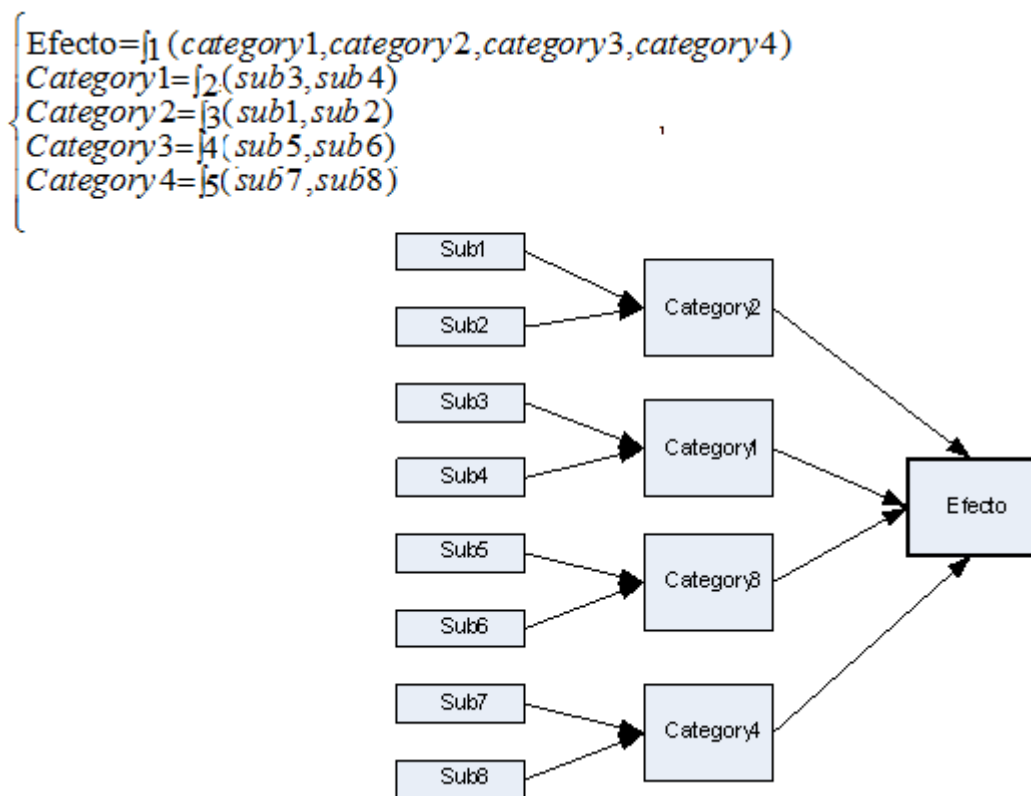


Figura 10 - Representación del problema según SEM.

Modelos causales estructurales (SCM)

Este es un modelo que combina diferentes modelos tomando cualidades de cada uno desde diferentes perspectivas, introduciendo en particular el concepto de intervención en el modelo estructural.

Particularmente desde la perspectiva de la representación gráfica este modelo toma las características de los modelos gráficos y los modelos SEM (Pearl, 2000, 2009).

A partir de lo anterior, la representación del problema según este modelo será homóloga a la presentada en la Figura 10.

4.1.3 Interpretación de los resultados del ejercicio

Según se presentó a partir de los ejemplos anteriores, es posible modelar el diagrama de Ishikawa para el problema original, en otros diagramas simplemente variando las formas y la disposición de los conceptos entre representaciones gráficas.

Para ello es necesario reorganizar la misma información expuesta por el diagrama de Ishikawa que modela el problema original, y simplemente completarlo siguiendo las normas de representación propias de cada herramienta. Lo cual equivale a un procedimiento de transformación.

Al identificar un procedimiento de transformación entre el diagrama de Ishikawa y cada una de las herramientas seleccionadas, hemos identificado un ejemplo que muestra la similitud en la capacidad de abstracción de conceptualizaciones entre las representaciones gráficas de las herramientas de soporte a DCA.

Que exista una similitud en la capacidad de abstracción de las conceptualizaciones involucradas, está indicando que los conceptos que las herramientas modelan son en alguna medida equivalentes.

Por otra parte, el proceso de transformación entre modelos en ninguno de los casos desarrollados ha requerido descubrir información adicional para ser completado. Este hecho está indicando que las herramientas tienen necesidades de información que en alguna medida son equivalentes, lo cual da la pauta de que a través de sus representaciones gráficas las herramientas intentan modelar un mismo dominio de interés.

Como observación adicional, ninguna de las herramientas analizadas dispone de mecanismos que permitan representar restricciones al dominio siendo estas restricciones necesarias al tratarse de relaciones causales (ver [sección 2.4](#)).

Este último punto es un aspecto importante, ya que como se ha presentado en secciones previas la capacidad de poder modelar restricciones al contexto es un aspecto clave para el modelado de relaciones causales.

La ventaja de la aplicación de ontologías para modelar el conocimiento y las conceptualizaciones contenidas en las herramientas para DCA es que permitirá contar con mecanismos suficientes para modelar restricciones subyacentes al dominio de interés, ya que es una de las características de una ontología (ver [sección 2.5](#)). De esta forma reunir en la ontología, el conocimiento del dominio relativo al DCA que exponen las herramientas.

El ejercicio planteado, no es una prueba sino un ejercicio de verificación de la observación acerca de que las herramientas utilizadas para DCA capturan el mismo conocimiento. De esta observación se puede desprender la conjetura que sobre la existencia de un isomorfismo entre representaciones que no consideran ciclos. Aunque interesante, esta observación no será desarrollada como parte del alcance durante el desarrollo de esta tesis.

4.2 Diseño de ontologías

En esta sección se presenta el detalle de la ejecución del proceso de desarrollo para la ontología de acuerdo con la metodología de Stanford (Noy y Guinness, 2000).

El proceso se desarrolla en dos iteraciones: la primera iteración ([sección 4.2.1](#)) describe el diseño de la primera versión de la ontología que surge con la concepción de las relaciones causales como el vínculo entre conceptos genéricos y sus atributos.

Como resultado de la primera iteración de la metodología se obtuvo una versión de alto nivel (top-level) de la ontología, la cual es sometida a una primera instancia de validación durante la última etapa de la metodología de desarrollo. Esta instancia de validación consiste en plantear un problema como objetivo de validación, de forma de generar las instancias necesarias para representar dicho objetivo en la ontología obtenida como resultado de la ejecución del proceso.

Finalmente, se realiza una segunda iteración ([sección 4.2.2](#)) en busca de una ontología que responda a una ontología de dominio. La ontología de dominio permite representar el problema con un enfoque más cercano a la ingeniería de software. Esta ontología es sometida al mismo ejercicio de validación que la versión de alto nivel.

El ejercicio de validación planteado, trata del análisis de un defecto realizado por un alumno del curso de mejora de procesos de software dictado en Universidad ORT Uruguay. El defecto objetivo del análisis proviene de un análisis causal de defectos tomado de un ejercicio de un curso de Personal Software Process (PSP) (Humphrey, 2000).

4.2.1 Iteración 1: Ontología de alto nivel (top-level)

Paso 1. Determinar el dominio y el alcance de la ontología

¿Cuál es el dominio que la ontología cubrirá?

El dominio de la ontología es el análisis de defectos en los procesos de producción de las organizaciones de software. Para ello la ontología deberá ser capaz de capturar el conocimiento causal contenido en las técnicas para DCA utilizadas en organizaciones de software durante el análisis de sus defectos.

En particular, las herramientas que han de ser utilizadas como parte del diseño de la ontología son las que se han mencionado previamente en este documento:

- Diagramas causa-efecto de Ishikawa
- Mapas mentales
- Mapas cognitivos
- Modelos gráficos
- Modelos de Ecuaciones Estructurales (SEM)
- Modelos causales estructurales (SCM)

¿Es válido considerar que sólo los conceptos representados en las herramientas anteriores son suficientes para captar todos los conceptos y las relaciones con respecto a la ontología?

En realidad no, sin embargo no se trata de que sean suficientes sino que se trata de que todas estas herramientas colaboran con el análisis de relaciones causales, por lo tanto son diferentes herramientas para tratar los mismos conceptos sin que ninguna de ellas agregue más información. Lo cual ofrece una base conceptual a partir de la cual desarrollar una ontología para análisis de relaciones causales en DCA que luego pueda ser evolucionada con diferentes instancias de validación.

¿Para qué se utilizará la ontología?

La ontología brindará soporte a las organizaciones productoras de software en cuanto a la clasificación y administración del conocimiento relacionado con el análisis causal de los defectos en los procesos de producción de software.

Además la ontología permitirá realizar inferencias sobre las entidades modeladas en la base de conocimiento de la ontología, mejorando las capacidades de análisis de la organización que la utiliza.

¿Cuáles son las preguntas que debe responder la ontología?

La ontología permitirá generar un modelo causal que permitiría responder preguntas como las siguientes:

- Dado los proyectos pasados en el contexto de una organización, ¿cuáles son las causas que lideran cuales tipos de defectos?
- Dado los pasados proyectos en el contexto de una organización, ¿es posible encontrar que una causa en particular lidere un tipo de defecto en particular?
- Dado los pasados proyectos en el contexto de una organización, ¿cuáles son las causas que lideran un tipo particular de defecto?

Comprender cuál es el concepto de tipo de defecto en el contexto de la ontología facilitará el proceso para responder a estas cuestiones.

Los defectos son tipificados de acuerdo con diferentes perspectivas que se configuran a partir de la combinación de diferentes atributos de los defectos. Esto permitirá responder las preguntas anteriores desde diferentes perspectivas del defecto, de acuerdo con las necesidades de análisis de cada organización.

¿Quiénes utilizan y mantienen la ontología?

El DCA es un proceso intelectual que necesita del conocimiento de las personas involucradas en la ejecución de los procesos de producción de software. Quienes contribuyen a los defectos están mejor calificados para identificar qué ha sido lo que ha ido mal y cómo prevenirlo (Card, 2006).

Por lo tanto, la ontología será utilizada por miembros de organizaciones productoras de software para analizar, administrar y clasificar el conocimiento que se adquiera respecto del análisis casual de defectos capturado normalmente a través de alguna de las herramientas mencionadas anteriormente.

El mantenimiento se realiza en la herramienta Protégé sobre el cual se desarrolla la ontología. La ontología puede ser evolucionada conforme avanza el conocimiento de DCA en la comunidad.

Paso 2. Considerar la reutilización de ontologías ya existentes

Como sugiere la metodología, es valioso revisar las ontologías existentes con la intención de hacer reuso de ellas. Realizada la búsqueda en bancos públicos de ontologías (<http://www.ksl.stanford.edu/software/ontolingua>, www.daml.org/ontologies y www.unspsc.org, entre otros) no se encontró, al momento de escribir esta tesis, una ontología que vincule o presente relacionados los conceptos de causa, defecto, relaciones causales en el ámbito de la ingeniería de software.

Paso 3. Listar los términos importantes para la ontología

De las herramientas identificadas es posible abstraer un conjunto de términos comunes, siendo éstos los siguientes:

Tabla 7 - Términos comunes identificados entre herramientas para DCA

Términos
Concepto central
Conceptos secundarios
Relaciones entre causas y efectos
Métricas
Valoración
Grafos acíclicos
Modelo matemático
Causa raíz
Tipo de causa
Tipo efecto

Los conceptos listados en la Tabla 7 pueden ser vistos como un conjunto de conceptos abstractos. Las relaciones entre los conceptos tendrán un peso (valoración cualitativa o cuantitativa) asociado que permite realizar un juicio de valor sobre la relación.

La Tabla 7 presenta la abstracción de conceptos motivada por la lista de términos anterior.

Tabla 8 – Iteración 1: Abstracción de conceptos a partir de listado de términos

Concepto	Término
Concepto	<i>Concepto central, conceptos secundarios, causa, efecto</i>
Relación	<i>relaciones entre causas y efectos, grafos acíclicos, modelo matemático</i>
Peso	<i>Valoración</i>
Atributos	<i>métricas, causa raíz, tipo de causa, tipo efecto</i>

Paso 4. Definir las clases y la herencia entre clases

Tabla 9 - Iteración 1: Clases y responsabilidades en la ontología

Clase	Responsabilidad
Abstracción	Representa una abstracción conceptual, centraliza información común entre los diferentes conceptos de la ontología
Concepto	Representa los nodos en las representaciones gráficas (causas, efectos y variables)

Relación	Son relaciones entre conceptos
Peso	Es la valoración que tiene una relación (cualitativa o cuantitativa)
Atributos	Son características que se pueden asociar a un concepto

El listado de clases anterior se distribuye en una jerarquía según se ve en la Figura 11

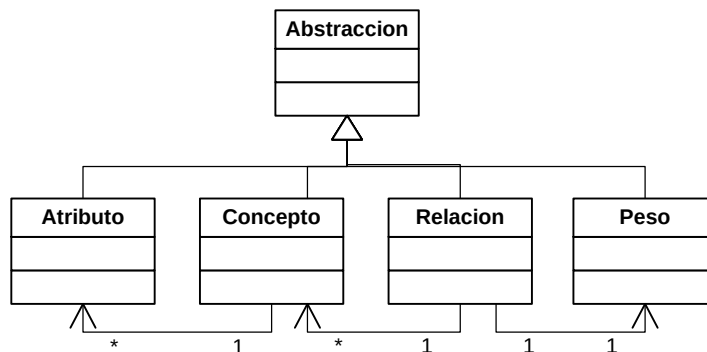


Figura 11 - Jerarquía de clases para la ontología

Paso 5. Definir las *Slots* (propiedades) de cada clase

Tabla 10 - Iteración 1: Propiedades de cada clase en la ontología

Clase	Slot	Descripción
Abstracción	Nombre	Nombre de la abstracción
Concepto	Es_Desde	Relaciones donde aparece en DESDE
	Es_Hacia	Relaciones donde aparece como HASTA
	Atributos	Lista de atributos del concepto
Relación	Desde	Concepto DESDE
	Hacia	Concepto HASTA
	Peso	Peso de la relación
Peso	Valor	Valor (numérico o no)
	Pesa_En	Las relaciones asociadas al peso
Atributo	Valor	Valor específico para el atributo
	Atributo_De	De qué conceptos es atributo

Paso 6. Definir las *Facets* (restricciones) de los *Slots*

Tabla 11 - Iteración 1: Restricciones de cada propiedad en la ontología

Slot	Restricción	
	Tipo	Cardinalidad
Nombre	STRING (cadena de caracteres)	Obligatorio y a lo sumo 1
Es_Desde	Lista de instancias de Relación	Opcional y múltiple (0-*)
Es_Hacia	Lista de instancias de Relación	Opcional y múltiple (0-*)
Atributos	Lista de instancias de Atributo	Opcional y múltiple (0-*)
Desde	Instancia de Concepto	Obligatorio y a lo sumo 1
Hacia	Instancia de Concepto	Obligatorio y a lo sumo 1
Peso	Instancia de Peso	Obligatorio y a lo sumo 1
Valor	ANY (cualquiera)	Obligatorio y a lo sumo 1
Pesa_En	Lista de instancias de Relación	Opcional y múltiple (0-*)
Valor	ANY (cualquiera)	Obligatorio y a lo sumo 1
Atributo_De	Lista de instancias de Concepto	Opcional y múltiple (0-*)

Paso 7. Crear Instancias.

Las instancias son un instrumento para validar la ontología. Esta instancia de validación consiste en plantear un problema como objetivo de validación, de esta forma generar las instancias necesarias para representar dicho objetivo en la ontología obtenida como resultado del proceso de desarrollo.

El problema planteado como objetivo de validación es el análisis causal de un defecto aportado por el docente del curso de mejora de procesos de software dictado en Universidad ORT Uruguay. El objetivo del curso es que los alumnos comprendan y apliquen la metodología Personal Software Process (PSP) (Humphrey, 2000), el ejemplo escogido proviene del análisis causal realizado por un alumno sobre un defecto resultante de uno de los ejercicios del curso el cual consistía en el desarrollo de un prototipo aplicando la metodología PSP. La Figura 12 recoge el análisis del defecto citado.

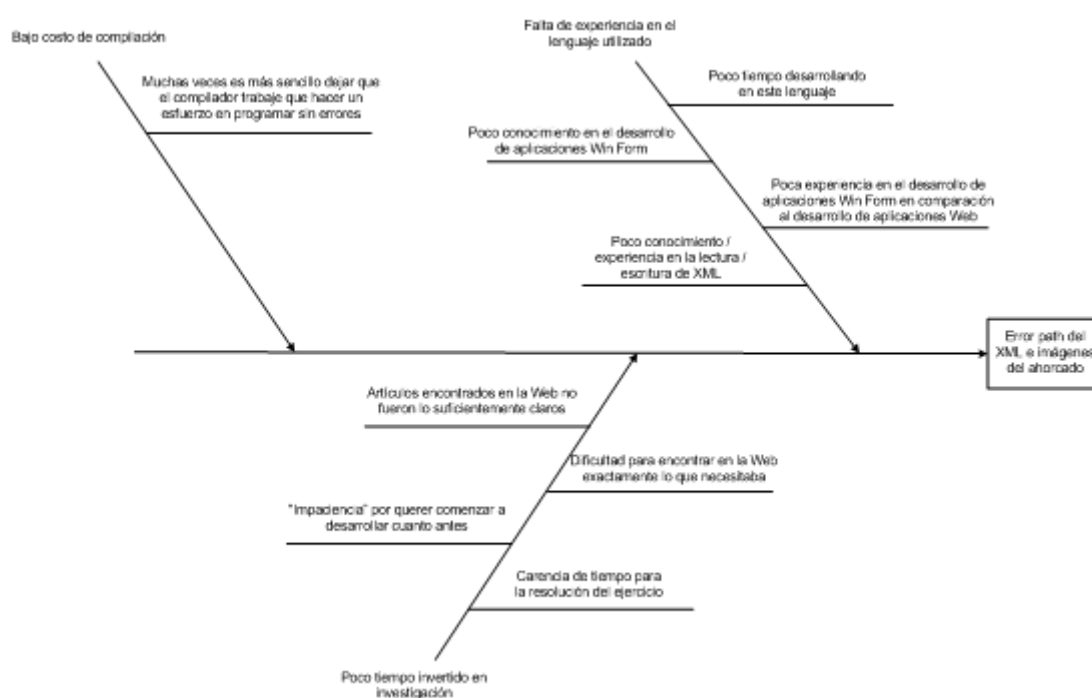


Figura 12 - Problema representado en un diagrama de Ishikawa.

La Tabla 12, presenta para el problema de validación estipulado el listado de instancias generado:

Tabla 12 - Instancias en la ontología para el ejemplo de validación

Nombre	Tipo
CAUSA	Atributo
DEFECTO	Atributo
NEUTRO	Peso
Error path del XML e imágenes del ahorcado	Concepto
Falta de experiencia en el lenguaje utilizado	Concepto
Bajo costo de compilación	Concepto
Poco tiempo invertido en investigación	Concepto
Poco tiempo desarrollando en este lenguaje	Concepto
Poco conocimiento en el desarrollo de aplicaciones WinForm	Concepto
Poca experiencia en el desarrollo de aplicaciones WinForm en comparación al desarrollo de aplicaciones Web	Concepto

Poco conocimiento / experiencia en la lectura / escritura de XML	Concepto
Muchas veces es más sencillo dejar que el compilador trabaje que hacer un esfuerzo en programar sin errores	Concepto
Artículos encontrados en la Web no fueron lo suficientemente claros	Concepto
Dificultad para buscar en la Web exactamente lo que necesitaba	Concepto
“Impaciencia” por querer comenzar a desarrollar cuanto antes	Concepto
Carencia de tiempo para la resolución del ejercicio	Concepto
DEFECTO_CATEGORIA1	Relación
DEFECTO_CATEGORIA2	Relación
DEFECTO_CATEGORIA3	Relación
CATEGORIA1_CAUSA1	Relación
CATEGORIA1_CAUSA2	Relación
CATEGORIA1_CAUSA3	Relación
CATEGORIA1_CAUSA4	Relación
CATEGORIA2_CAUSA5	Relación
CATEGORIA3_CAUSA6	Relación
CATEGORIA3_CAUSA7	Relación
CATEGORIA3_CAUSA8	Relación
CATEGORIA3_CAUSA9	Relación

Como resultado del ejercicio de validación se identifican ciertos aspectos plausibles de críticas a esta versión de la ontología:

- Al representar conceptos abstractos no expone firmemente los conceptos relacionados con el dominio de interés que se buscan representar (Tabla 9).
- Las características genéricas de la ontología diseñada dificultan la representación de restricciones en el dominio de la ingeniería de software.
- Conceptos como Causa, Defecto, o un tipo de relación, tienen la relevancia e influencia suficiente sobre el dominio del problema como para ser modelados a nivel conceptual (clase en la ontología). La primera versión de la ontología modela estos conceptos a través de instancias.
- No es posible modelar características del contexto en el cual se presenta un defecto. Conceptos como el tipo de organización; tipo de proyecto, entre otros atributos del contexto son importantes para el dominio del problema, por lo cual deben cobrar una mayor relevancia en el modelado conceptual de la ontología.

4.2.2 Iteración 2: Ontología de dominio (domain-level)

Como resultado de la primera iteración se obtuvo una ontología de alto nivel (*top-level*) y como resultado del ejercicio de validación se identifican oportunidades de mejora.

Por lo tanto, se realiza una nueva iteración de la metodología de desarrollo en busca de una ontología que responda a una ontología de dominio. Al modificar el nivel de abstracción de las conceptualizaciones involucradas, la nueva versión de la ontología permite representar el dominio del problema con un enfoque más cercano al DCA.

Dado que los objetivos para la ontología no varían en relación a los identificados durante el desarrollo de la primera versión, la nueva iteración comienza en el paso 3 de la metodología, primero de los pasos involucrado directamente con los aspectos de diseño ontológico.

Paso 3. Listar los términos importantes para la ontología

Los términos involucrados en esta versión, surgen tanto de las herramientas (como en la primera) como de la necesidad de obtener un modelo más cercano conceptualmente a la ingeniería de software y en particular el DCA.

Por lo tanto la Tabla 13, muestra los términos a considerar durante esta iteración de la metodología de desarrollo mostrando la entidad correspondiente según la ontología obtenida durante la primera iteración.

Tabla 13 - Iteración 2: Abstracción de conceptos a partir de listado de términos

Término	Iteración I
Defecto	Concepto
Causa	Concepto
Canvas (Contexto)	Concepto
Características del defecto	Atributo (un conjunto)
Categoría	Atributo
Características de causa	Atributo (un conjunto)
Características del contexto	Atributos (un conjunto)
Peso	Atributo
Relación Canvas x Defecto	Relación
Relación Causa x Efecto	Relación
Relación Causa x Causa	Relación

Las propiedades para los defectos, así como los valores permitidos e información al respecto de las causas más comunes se obtienen a partir del estándar IEEE para la clasificación de anomalías en el software (IEEE, 1993) y las sugerencias para análisis de defectos presentadas en (Kalinowski, Mendes *et al.*, 2010).

Paso 4. Definir las clases y la herencia entre clases

Tabla 14 - Iteración 2: Clases y responsabilidades en la ontología

Clase	Responsabilidad
Canvas	Representa el contexto, el modelado del análisis causal para un defecto dado un entorno organizacional conocido
Causa	Modela las causas que una organización considera al momento de analizar las razones por la cual ocurre un defecto (o tipo de defecto) en particular
Defecto	Conceptualización de un defecto (o tipo de defecto) en particular que es objeto de análisis dado un contexto organizacional conocido
Relación	Abstracción conceptual que reúne características comunes a los diferentes tipos de relaciones que se distinguen en torno al dominio de interés.
Canvas_Defecto	Tipo de relación que vincula un lienzo (Canvas) con el defecto en particular
Causa_Causa	Tipo de relación que vincula dos causas en una relación de Causa/Sub-causa
Causa_Defecto	Tipo de relación que vincula una causa (o categoría) a un defecto en particular

El listado de clases anterior se distribuye en una jerarquía según se ve en la Figura 13

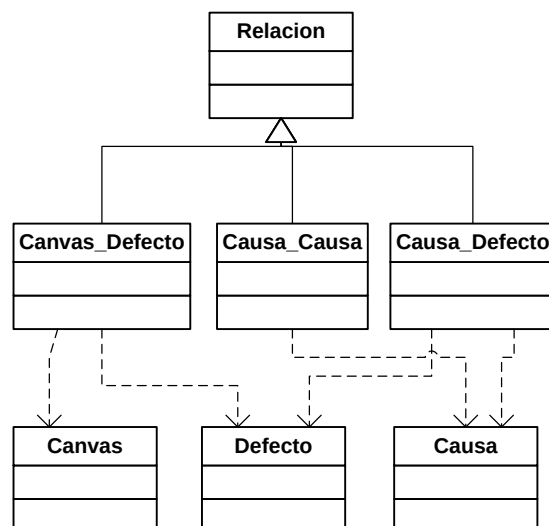


Figura 13 - Jerarquía de clases para la ontología

Paso 5. Definir los Slots (propiedades) de cada clase

Tabla 15 - Iteración 2: Propiedades de cada clase en la ontología

Clase	Slot	Descripción
Canvas	Nombre	Nombre de la instancia de análisis
	Descripción	Descripción de la instancia de análisis
	Tipo de análisis	Describe si el análisis causal se realiza para un defecto en particular, o es el resultado de una categorización previa en base a defectos registrados en una base de datos.
	Rubro de la organización	Rubro/contexto de la organización
	Tipo de organización	Tipo de organización
	Inversa_de_Desde	Implica relaciones en donde una instancia de Canvas ocupa posición DESDE
Causa	Categoría	Categoría asociada a la causa en particular
	Descripción	Descripción de la causa en particular
	Nombre	Nombre asignado a la causa en particular
	Correctivo	Consecuencia de la corrección del defecto generalmente una oportunidad de mejora para el proceso, proyecto u otro aspecto organizacional
	Inversa_de_Desde	Implica relaciones en donde una instancia de Causa en particular ocupa posición Desde
	Inversa_de_Hasta	Implica relaciones en donde una instancia de Causa en particular ocupa posición Hasta
Relación	Descripción	Descripción de la relación
	Peso	Peso asociado a la relación
	Nombre	Nombre dado a la relación
Canvas_Defecto	Descripción	Atributo heredado de Relación
	Desde	Desde dónde surge la relación (Canvas)
	Hasta	Hacia dónde se dirige la relación (Defecto)
	Peso	Atributo heredado de Relación
	Nombre	Atributo heredado de Relación
Causa_Causa	Descripción	Atributo heredado de Relación
	Desde	Desde dónde surge la relación (Causa)
	Hasta	Hacia dónde se dirige la relación (Causa)
	Peso	Atributo heredado de Relación
	Nombre	Atributo heredado de Relación

Causa_Defecto	Descripción	Atributo heredado de Relación
	Desde	Desde dónde surge la relación (Causa)
	Hasta	Hacia dónde se dirige la relación (Defecto)
	Peso	Atributo heredado de Relación
	Nombre	Atributo heredado de Relación
Defecto	Nombre	Nombre del defecto en particular
	Descripción	Descripción del defecto en particular
	Tamaño del módulo	Característica de contexto para el defecto
	Inversa_de_Desde	Implica relacione en donde una instancia de Defecto en particular ocupa posición Desde
	Inversa_de_Hasta	Implica relacione en donde una instancia de Defecto en particular ocupa posición Hasta
	Esfuerzo de detección	Característica de contexto para el defecto
	Actividad	Actividad del proceso por la cual se detecta el defecto
	Fase del proyecto	Fase en el proyecto en la cual se detecta el defecto
	Repetitividad	Que tan repetible es el defecto en el contexto actual
	Fuente	Cuál es el producto que origina el defecto
	Resolución	Acción que debe tomarse sobre el defecto
	Severidad	Severidad estimada para el defecto
	Prioridad	Prioridad estimada para la corrección del defecto
	Valor a Cliente	Valor que tiene la corrección del defecto para el cliente
	Impacto	Impacto del defecto en el producto
	Síntoma	Síntoma que ha permitido desencadenar el descubrimiento del defecto.

Paso 6. Definir las Facets (restricciones) de los Slots

Tabla 16 - Iteración 2: Restricciones de cada propiedad en la ontología

Slot	Restricción	
	Tipo	Cardinalidad
Nombre	STRING	Obligatorio y a lo sumo 1
Descripción	STRING	Obligatorio y a lo sumo 1
Tipo de análisis	STRING	Obligatorio y a lo sumo 1
Rubro de la organización	STRING	Obligatorio y a lo sumo 1
Tipo de organización	STRING	Obligatorio y a lo sumo 1
Categoría	STRING	Obligatorio y a lo sumo 1
Inversa_de_Desde	Instancia de Defecto, Causa o Canvas dependiendo del dominio de la relación	Opcional y múltiple (0-*)
Inversa_de_Hasta	Instancia de Defecto, Causa dependiendo del dominio de la relación	Opcional y múltiple (0-*)
Peso	STRING respondiendo a una escala ordinal que caracterice el valor del peso adecuado según una métrica previamente establecida.	Obligatorio y a lo sumo 1 Valor por defecto: Neutro
Desde	Instancia de Defecto, Causa o Canvas dependiendo del dominio de la relación	Opcional y múltiple (0-*)
Hasta	Instancia de Defecto, Causa dependiendo del dominio de la relación	Opcional y múltiple (0-*)
Tamaño del módulo	STRING, respondiendo a una escala ordinal que caracterice el tamaño del módulo según una métrica previamente establecida	Obligatorio y a lo sumo 1
Esfuerzo de detección	Float (dato numérico)	Opcional y a lo sumo 1
Actividad	STRING, con valores restringidos a un conjunto predeterminado de acuerdo a lo especificado para el atributo en el estándar IEEE (IEEE, 1993)	Obligatorio y a lo sumo 1
Fase del proyecto	STRING, con valores restringidos a un conjunto predeterminado de acuerdo a lo especificado para el atributo en el estándar IEEE (IEEE, 1993)	Obligatorio y a lo sumo 1

Repetitividad	STRING, con valores restringidos a un conjunto predeterminado de acuerdo a lo especificado para el atributo en el estándar IEEE (IEEE, 1993)	Obligatorio y a lo sumo 1
Fuente	STRING, con valores restringidos a un conjunto predeterminado de acuerdo a lo especificado para el atributo en el estándar IEEE (IEEE, 1993)	Obligatorio y a lo sumo 1
Resolución	STRING, con valores restringidos a un conjunto predeterminado de acuerdo a lo especificado para el atributo en el estándar IEEE (IEEE, 1993)	Obligatorio y a lo sumo 1
Correctivo	STRING, con valores restringidos a un conjunto predeterminado de acuerdo a lo especificado para el atributo en el estándar IEEE (IEEE, 1993)	Obligatorio y a lo sumo 1
Severidad	STRING, con valores restringidos a un conjunto predeterminado de acuerdo a lo especificado para el atributo en el estándar IEEE (IEEE, 1993)	Obligatorio y a lo sumo 1
Prioridad	STRING, con valores restringidos a un conjunto predeterminado de acuerdo a lo especificado para el atributo en el estándar IEEE (IEEE, 1993)	Obligatorio y a lo sumo 1
Valor a Cliente	STRING, con valores restringidos a un conjunto predeterminado de acuerdo a lo especificado para el atributo en el estándar IEEE (IEEE, 1993)	Obligatorio y a lo sumo 1
Impacto	STRING, con valores restringidos a un conjunto predeterminado de acuerdo a lo especificado para el atributo en el estándar IEEE (IEEE, 1993)	Obligatorio y a lo sumo 1
Síntoma	STRING, con valores restringidos a un conjunto predeterminado de acuerdo a lo especificado para el atributo en el estándar IEEE (IEEE, 1993)	Obligatorio y a lo sumo 1

Paso 7. Crear Instancias.

En cuanto a las instancias, para esta segunda versión de la ontología se propone el mismo objetivo de validación que fuera propuesto con la ontología resultante de la primera iteración. Representar el ejemplo de análisis causal para el prototipo desarrollado aplicando la metodología PSP.

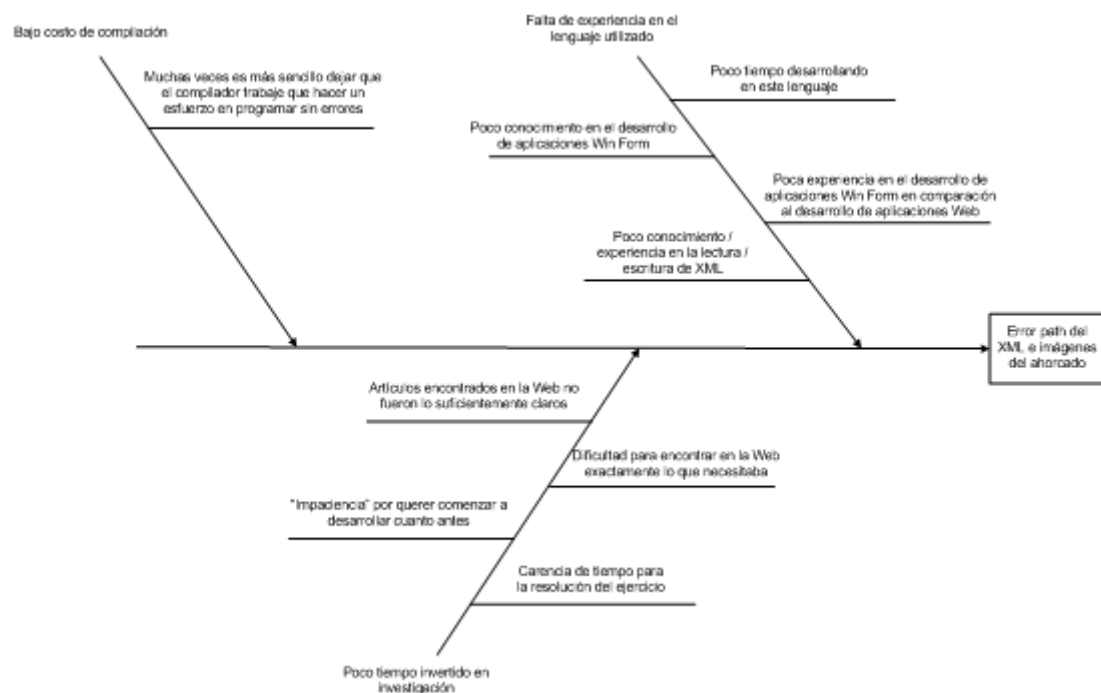


Figura 14 - Problema representado en un diagrama de Ishikawa.

La Tabla 17, presenta el listado de instancias generado para el problema de validación estipulado:

Tabla 17 - Listado de instancias para el problema de validación

Nombre	Tipo
Defectos PSP, Laboratorio 4, .NET	Canvas
Error path del XML e imágenes del ahorcado	Defecto
Artículos encontrados en la Web no fueron lo suficientemente claros	Causa
Bajo costo de compilación	Causa
Carencia de tiempo para la resolución del ejercicio	Causa
Dificultad para buscar en la Web exactamente lo que necesitaba	Causa
Falta de experiencia en el lenguaje utilizado	Causa
Muchas veces es más sencillo dejar que el compilador trabaje que hacer un esfuerzo en programar sin errores	Causa
Poca experiencia en el desarrollo de aplicaciones WinForm en comparación al desarrollo de aplicaciones Web	Causa
Poco conocimiento / experiencia en la lectura / escritura de XML	Causa
Poco conocimiento en el desarrollo de aplicaciones WinForm	Causa
Poco tiempo desarrollando en este lenguaje	Causa
Poco tiempo invertido en investigación	Causa
“Impaciencia” por querer comenzar a desarrollar cuanto antes	Causa
ontologiaDominio_Canvas_Error path del XML e imágenes del ahorcado	Canvas_Defecto
ontologiaDominio_Bajo costo de compilación_Error path del XML e imágenes del ahorcado	Causa_Defecto
ontologiaDominio_Falta de experiencia en el lenguaje utilizado_Error path del XML e imágenes del ahorcado	Causa_Defecto
ontologiaDominio_Poco tiempo invertido en investigación_Error path del XML e imágenes del ahorcado	Causa_Defecto
ontologiaDominio_Artículos encontrados en la Web no fueron lo suficientemente claros_Poco tiempo invertido en investigación	Causa_Causa
tiempo para la resolución del ejercicio_Poco tiempo invertido en investigación	Causa_Causa
ontologiaDominio_Dificultad para buscar en la Web exactamente lo que necesitaba_Poco tiempo invertido en investigación	Causa_Causa
ontologiaDominio_Impaciencia por querer comenzar a desarrollar cuanto antes_Poco tiempo invertido en investigación	Causa_Causa
ontologiaDominio_Muchas veces es más sencillo dejar que el compilador trabaje que hacer un esfuerzo en programar sin errores_Bajo costo de compilación	Causa_Causa
ontologiaDominio_Poca experiencia en el desarrollo de aplicaciones WinForm en comparación al desarrollo de aplicaciones Web_Falta de experiencia en el lenguaje utilizado	Causa_Causa
ontologiaDominio_Poco conocimiento / experiencia en la lectura / escritura de XML_Falta de experiencia en el lenguaje utilizado	Causa_Causa
ontologiaDominio_Poco conocimiento en el desarrollo de aplicaciones WinForm_Falta de experiencia en el lenguaje utilizado	Causa_Causa
ontologiaDominio_Poco tiempo desarrollando en este lenguaje_Falta de experiencia en el lenguaje utilizado	Causa_Causa

Tras el ensayo de validación para la ontología realizada con el Paso 7 del proceso, se esbozan algunas diferencias respecto de la primera versión:

- La cantidad de instancias en general será menor a pesar de que la cantidad de instancias para conceptos clave como causas, defectos y relaciones permanece igual. Generar una cantidad menor de instancias facilita la recolección de resultados a partir de las consultas a la base de conocimiento por la reutilización de entidades

- A diferencia de la primera versión de la ontología (ver Tabla 9), la ontología actual trata con conceptualizaciones cercanas al dominio. Esto facilita la interpretación de las instancias al momento del diseño de las consultas a la base de conocimiento.
- Se consideran aspectos del contexto del defecto, lo cual deberá facilitar la interpretación de causas a partir de información de diferentes contextos similares dentro de la organización software.

Es a partir de esta versión de la ontología que se ejecuta el proceso de validación al completo, cuyo detalle se presenta en la [sección 5](#).

4.2.3 Formalización de la ontología

En las ciencias de la computación existen diferentes formalismos que pueden ser utilizados para representar las entidades del dominio en una ontología. En particular para formalizar la ontología presentada durante esta sección, hemos escogido utilizar el lenguaje OWL ([sección 2.5.4](#)) ya que se trata de un lenguaje formal, basado en lógica descriptiva, estructurado (RDF y XML) y que es utilizado para representar ontologías. OWL es además el lenguaje nativo para Protégé.

A partir de formalizar la ontología utilizando OWL, también es posible formalizarla aplicando lógica descriptiva. Uno de los procedimientos para lograr la transformación a partir de la ontología en lenguaje OWL es deducir el modelo entidad relación (MER) correspondiente. Existen algoritmos, como el de (Calvanese, Lenzerini *et al.*, 1994), que traducen el MER hacia sentencias lógicas descriptivas.

Sin embargo la formalización OWL prevista no incluye a las consultas (*query*) que representan las preguntas de competencia para validar la ontología ya que no forman parte del dominio ni la base de conocimiento, son solo un instrumento para validar la ontología. También existen limitaciones en los formalismos lógicos (de primer y segundo orden) que dificultan la representación formal de consultas.

La formalización completa del dominio de la ontología (sin incluir sus instancias) en lenguaje OWL es presentada en el [Anexo I](#).

5. Validación de la ontología

Durante el desarrollo de esta sección se describen aquellos aspectos relacionados con el proceso de validación realizado sobre la ontología ([sección 4.2](#)). La ontología ha sido desarrollada con el objetivo de modelar el conocimiento relacionado con DCA en la industria del software, por lo tanto el objetivo de validación es conocer si la ontología es capaz de modelar el conocimiento de las actividades de DCA realizadas por una empresa que opera en la industria del software uruguaya.

El proceso de validación, consiste en modelar en la ontología la información de defectos obtenida y posteriormente someter dicha información a las preguntas de competencia. De esta forma se logra evaluar las capacidades de modelado e inferencia sobre las entidades de dominio que presenta la ontología diseñada.

En primer término se presenta una descripción del juego de datos utilizado como entrada para la generación de instancias y posterior validación de la ontología, ver [sección 5.1](#).

Posteriormente se presenta una introducción al modelado de los datos en la ontología, describiendo el proceso de carga de datos en la base de conocimiento para lo cual se desarrolla un ejemplo. Ver [sección 5.2](#).

Finalmente se presenta una descripción acerca de cómo la ontología resultante responde a las preguntas de competencia planteadas durante la fase de diseño ([sección 5.3](#)). Para ello se utiliza las facilidades de consulta (*query*) que brinda Protégé, las *query* no forman parte de la base de conocimiento pero son uno de los instrumentos implicados en el proceso de validación de una ontología.

5.1 Descripción del juego de datos

El juego de datos utilizado como entrada durante el ejercicio de validación de la ontología ha sido brindado por la empresa *Innicia* (www.innicia.com).

Innicia es una empresa de servicios especializada en el desarrollo de soluciones en el área de las tecnologías de la información. Desde el año 2007, la empresa ofrece un servicio integral de *outsourcing* en tecnología de la información, desarrolla a medida productos para sus clientes y realiza consultoría en IT. Todos estos proyectos los desarrolla sobre plataformas Microsoft. Es miembro de la Cámara Uruguaya de Tecnologías de la Información CUTI (<http://www.cuti.org.uy/>) y *Microsoft Certified Partner*.

Durante el año 2010 ha sido galardonada con el premio *Country Partner of the Year*. El premio destaca el uso innovador de tecnología Microsoft para dar soluciones creativas a los negocios y minimizar los problemas empresariales vinculados a nuevas tecnologías.

Durante 2010, *Innicia* ha comenzado el desarrollo e implantación de su sistema de gestión de calidad para potenciar aún más la calidad de sus productos. Como parte del sistema de gestión de calidad se incluye el registro y análisis causal de los defectos detectados durante las entregas realizadas a sus clientes.

El juego de datos utilizado como parte de validación de la ontología es resultado de los procesos de validación y verificación realizados en 7 diferentes proyectos desarrollados durante el año 2010 para clientes del extranjero.

Los proyectos tomados como fuente para la obtención de los defectos, han sido seleccionados en base a su tamaño medido en cantidad de componentes. Dichos proyectos son listados en la Tabla 18.

La categorización de los proyectos según su tamaño es la siguiente:

- Grande – Mayor a 10 componentes
- Medio – De 5 a 10 componentes
- Pequeño – Menos de 5 componentes

Tabla 18 - Proyectos utilizados como fuente de datos de defectos

Código de proyecto	Descripción del proyecto	Tamaño estimado
IBF2	Gestión de información de ventas de productos offline	Grande
PNR	Control del calidad de PNR de Amadeus	Medio
CAJA	Gestión del módulo de CAJA	Medio
GRP	Gestión de cotizaciones de grupos/individuales	Grande
EUF	Procesos para facturas pendientes de envío	Pequeño
DSS	Modulo de DataCenter (ERP)	Grande
Nominas	Información de nóminas	Medio

Los defectos extraídos de cada componente son registrados durante los procesos de validación y verificación del sistema de calidad de Innicia, los atributos registrados son los establecidos por el estándar IEEE 1044-1993 para clasificación de anomalías en el software (IEEE, 1993).

5.2 Modelado de los datos en la ontología

Con el desarrollo de esta sección se presenta cómo se modela un defecto del conjunto de datos de entrada en la ontología, desarrollando un ejemplo.

La Tabla 19 presenta la información relevada para proyectos y defectos en el juego de datos cedidos por Innicia.

Tabla 19 - Información relevada para proyectos y defectos en el juego de datos

Entidad	Nombre de la propiedad	Valores para la propiedad
Proyecto	Código de proyecto	Alfanumérico de 3 a 10 caracteres
Proyecto	Nombre del proyecto	Alfanumérico (texto libre)
Proyecto	Tamaño	Grande Medio Pequeño
Defecto	Proyecto	El código de proyecto donde se origina
Defecto	Descripción	Alfanumérico (texto libre)
Defecto	Actividad	<i>Project activity</i> en (IEEE, 1993)
Defecto	Fase	<i>Project Phase</i> en (IEEE, 1993)

Defecto	Repetibilidad	<i>Repeatability</i> en (IEEE, 1993)
Defecto	Síntoma	<i>Actual cause</i> en (IEEE, 1993)
Defecto	Esfuerzo	Valor numérico
Defecto	Causa Actual	<i>Actual cause</i> en (IEEE, 1993)
Defecto	Fuente	<i>Source</i> en (IEEE, 1993)
Defecto	Causa detallada	<i>Type</i> en (IEEE, 1993)
Defecto	Resolución	<i>Resolution</i> en (IEEE, 1993)
Defecto	Correctivo	<i>Corrective action</i> en (IEEE, 1993)
Defecto	Severidad	<i>Severity</i> en (IEEE, 1993)
Defecto	Prioridad	<i>Priority</i> en (IEEE, 1993)
Defecto	Cliente	<i>Customer value</i> en (IEEE, 1993)
Defecto	Costo	<i>Project cost</i> en (IEEE, 1993)
Defecto	Calendario	<i>Project schedule</i> en (IEEE, 1993)
Defecto	Peso	La importancia del defecto medida en los mismos valores que para los campos Cliente, Calendario, Peso.

Para el desarrollo de esta etapa se utiliza la herramienta *Instance Browser*, que es una herramienta (plug-in) facilitada por Protégé para permitir a los usuarios la creación y edición de instancias de una ontología. Para trabajar en la definición de los atributos de una instancia se utiliza el editor de instancias disponible en la herramienta.

Utilizando Instance Browser para la creación de instancias en la ontología, el proceso a seguir es ingresar como una instancia en la ontología a cada uno de los registros del juego de datos, defectos, causas y sus correspondientes relaciones.

Con tal objetivo se define la siguiente equivalencia entre las entidades en la ontología y los datos:

❖ *Instancia del tipo Canvas*

Se genera en la ontología una instancia del tipo *Canvas* por cada Proyecto. En cuanto a la nomenclatura utilizada, una instancia del tipo *Canvas* es denominada según el nombre de proyecto que representa, ej. IBERINFO2.

Para ejemplificar el modelado de datos, la Figura 15 presenta la instancia del tipo *Canvas* del proyecto Iberinfo2 en la ontología.

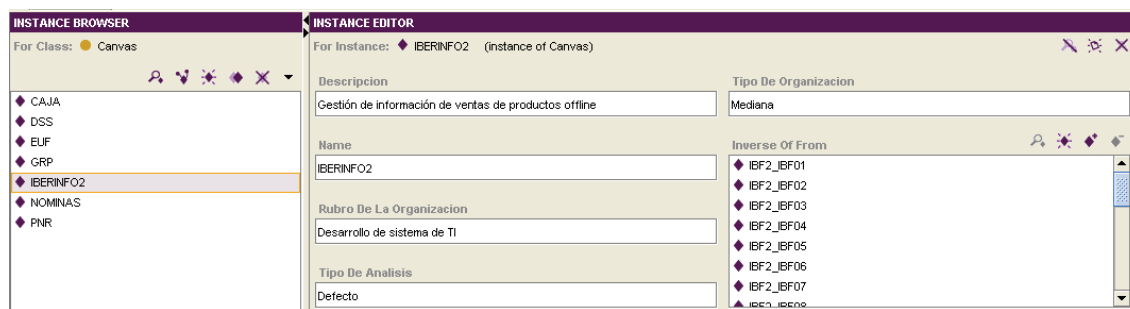


Figura 15 - Instancia del tipo Canvas para el proyecto Iberinfo2

❖ Instancia del tipo Causa

Se genera en la ontología una instancia del tipo *Causa* por cada causa definida en estándar IEEE 1044-1993. En cuanto a la nomenclatura utilizada, una instancia del tipo *Causa* es denominada de acuerdo con la descripción definida en el estándar IEEE 1044-1993 para la causa que representa. Ej. *Forgotten cases or steps*.

El ejemplo en la Figura 16, presenta la instancia del tipo Causa que representa a la causa *Forgotten cases or steps* del estándar IEEE 1044-1993

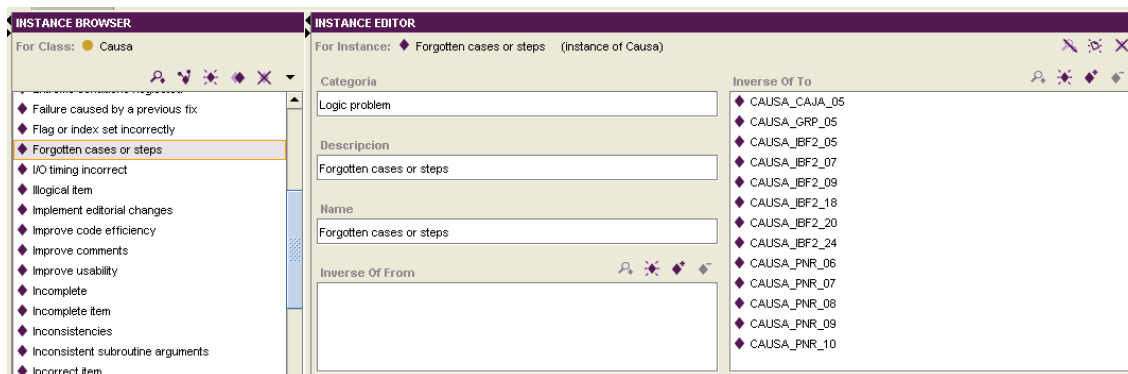


Figura 16 - Instancia del tipo Causa para la causa *Forgotten cases or steps*

❖ Instancia del tipo Defecto

Se genera en la ontología una instancia del tipo *Defecto* por cada uno de los defectos de la fuente de datos. En cuanto a la nomenclatura utilizada, una instancia del tipo *Defecto* es denominada con el código del proyecto en el cual ha sido detectado concatenado con un índice que representa el orden en que ha sido registrado el defecto. Ej. CAJA_01, IBF2_01.

La Figura 17, muestra la instancia del tipo Defecto que representa el primero de los defectos registrados para el proyecto Iberinfo2.

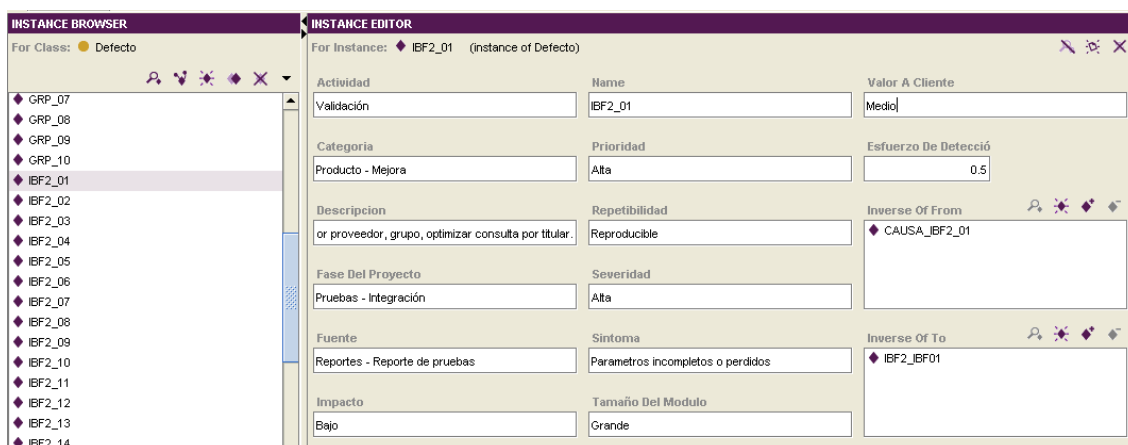


Figura 17 - Instancia del tipo Defecto para un defecto en el proyecto Iberinfo2

❖ *Instancia del tipo Canvas_Defecto*

Se genera en la ontología una instancia del tipo *Canvas_Defecto* por cada defecto detectado en un proyecto en particular. En cuanto a la nomenclatura utilizada, una instancia del tipo *Canvas_Defecto* es denominada según el código de proyecto en el cual se origina el defecto concatenado con el nombre de la instancia del tipo defecto asociada. Ej. CAJA_CAJA01

❖ *Instancia del tipo Causa_Causa*

Se genera en la ontología una instancia del tipo *Causa_Causa* por cada relación del tipo Causa y Sub-causa definida en el estándar IEEE 1044-1993. En cuanto a la nomenclatura utilizada, una instancia del tipo *Causa_Causa* se nombra según la concatenación de los nombres de las instancias del tipo Causa que relaciona. Ej. *Enhancement - Change in program requirements - Add new capability*.

❖ *Instancia del tipo Causa_Defecto*

Se genera en la ontología una instancia del tipo *Causa_Defecto* por cada asociación de un defecto a una causa en particular de acuerdo al análisis causal realizado para un proyecto en particular. En cuanto a la nomenclatura utilizada, una instancia del tipo *Causa_Defecto* es denominada a partir de la concatenación del prefijo “CAUSA” y el nombre de la instancia del tipo Defecto que se relaciona con el análisis causal modelado. Ej. CAUSA_CAJA_01

La Figura 18 ejemplifica una instancia del tipo *Relación*, que representa el análisis causal para una instancia del tipo Defecto, particularmente se trata de una instancia del tipo *Causa_Defecto* vinculada con la instancia del tipo Causa para *Forgotten cases or steps*.

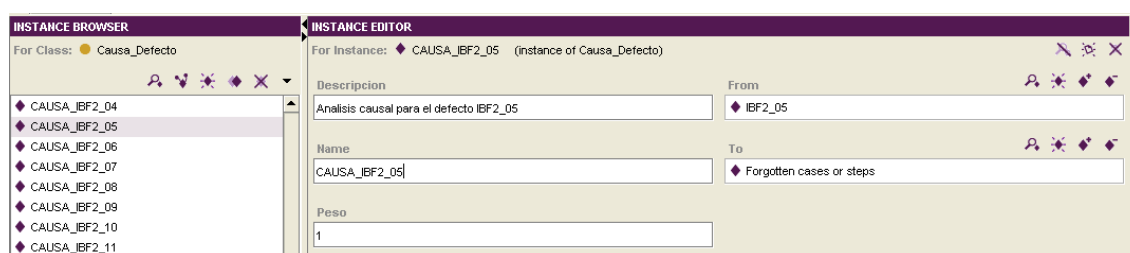


Figura 18 - Instancia del tipo Causa_Defecto para un caso de análisis causal en la ontología

❖ *Instancia del tipo Relación*

No se generan en la ontología instancias del tipo *Relación* ya que es considerada una entidad abstracta.

El proceso de modelado de datos en la ontología se completa repitiendo el criterio de equivalencia respecto de las entidades en la ontología, con el resto de los registros en el juego de datos brindado por la empresa.

La Tabla 20 presenta un resumen respecto del número total de defectos modelados en la ontología, según el proyecto fuente del cual han sido extraídos.

Tabla 20 - Cantidad de defectos según proyecto fuente

Código de proyecto	Cantidad de defectos en la ontología	Cantidad de Defectos en la muestra
IBF2	25	25
PNR	10	10
CAJA	10	10
GRP	10	10
EUUF	2	2
DSS	2	2
Nominas	7	7

Como resultado del modelado de datos en la ontología, se generan un total de:

- 7 instancias del tipo *Canvas*
- 72 instancias del tipo *Causa* (causas y sub-causas según IEEE 1044-1993)
- 66 instancias del tipo *Defecto*
- 66 instancias del tipo *Canvas_Defecto*
- 20 instancias del tipo *Causa_Causa*
- 66 instancias del tipo *Causa_Defecto*

La Figura 19 resume la cantidad de instancias según la ontología en Protégé.



Figura 19 - Cantidad de instancias según tipo en la ontología

5.3 Preguntas de competencia

Las preguntas de competencia ([sección 2.5.3](#)) son aquellas preguntas de validación diseñadas para que la ontología sea capaz de responder. En esta sección se presenta el criterio a partir del cual la ontología es capaz de dar respuesta a las preguntas de competencia, que fueran seleccionadas durante el proceso de diseño de la ontología.

Para dar respuesta a las preguntas habrá que definir qué se entiende por *tipo de defecto* en este contexto. La ontología ha sido diseñada de forma tal que los defectos modelados en ella puedan ser agrupados (o tipificados) desde diferentes perspectivas, a partir de la combinación de diferentes atributos de los defectos.

La [sección 5.3.1](#) presenta el método de resolución para cada una de las preguntas de competencia definidas para la ontología, mientras la [sección 5.3.2](#) presenta un ejercicio de resolución desarrollado en Protégé.

5.3.1 Método de resolución

En esta sub-sección se presenta el método de resolución previsto para obtener las respuestas a cada una de las preguntas de competencia. Las preguntas de competencia aplicadas para la ontología resultante son las siguientes:

1. *Dado los pasados proyectos en el contexto de una organización, ¿cuáles son las causas que lideran un tipo particular de defecto?*

Resolución: Los defectos pueden ser tipificados a partir de la combinación de sus atributos lo cual permite agruparlos desde diferentes perspectivas para ser analizados. Al seleccionar un tipo de defecto en particular es posible consultar a la ontología cuáles son las causas (en la ontología son relaciones del tipo *Causa_Defecto*) asociadas a cada defecto en esta tipificación. Con este conjunto de *Relaciones* la ontología es capaz de indicar cuál es la causa asociada a cada defecto, obteniendo en respuesta a la pregunta un conjunto de instancias del tipo *Causa* correspondiente a la tipificación seleccionada.

2. *Dado los proyectos pasados en el contexto de una organización, ¿cuáles son las causas que lideran un conjunto de tipos de defectos?*

Resolución: Cuando se repite el proceso descrito para responder a la primera de las preguntas para un conjunto particular de tipificaciones de defectos, se obtiene un conjunto de instancias del tipo *Causa* correspondiente a los defectos dentro de cada tipo de defecto elegido, respondiendo a la segunda pregunta.

3. *Dado los pasados proyectos en el contexto de una organización, ¿es posible encontrar que una causa en particular lidere un tipo de defecto en particular?*

Resolución: Cuando se cuenta con un grupo de defectos bajo una tipificación conocida e interesa conocer si una causa en particular tiene efecto sobre dicho grupo, es posible lograrlo combinando el proceso para conocer las causas de un tipo de defecto, con la causa en cuestión. La respuesta será un conjunto vacío si la causa no influye en el tipo de defecto particular, mientras que para el caso en que sí exista influencia la respuesta será la instancia correspondiente a la propia causa consultada. Esta respuesta puede ser interpretada como respuesta booleana.

5.3.2 Ejemplo de resolución

Esta sub-sección presenta un ejemplo de resolución para las preguntas de competencia a partir de los métodos descritos en la sección anterior.

Resolución: Pregunta 1

Para la información de proyectos instanciada en la ontología, supongamos que se busca analizar cuáles son las causas para los defectos con severidad *Alta* o *Urgente* de forma de responder a la pregunta 1.

El método de resolución para la pregunta 1 se puede resolver en 3 pasos que se listan a continuación:

- 1) Cuales son todos los defectos con severidad *Alta* o *Urgente*.
- 2) Cuales son todas las causas asociadas a los defectos obtenidos en 1).
- 3) De la combinación de 1) y 2) se obtiene la respuesta a la pregunta original.

La figura 20 presenta como luce la consulta para (1) en el editor de Protégé.

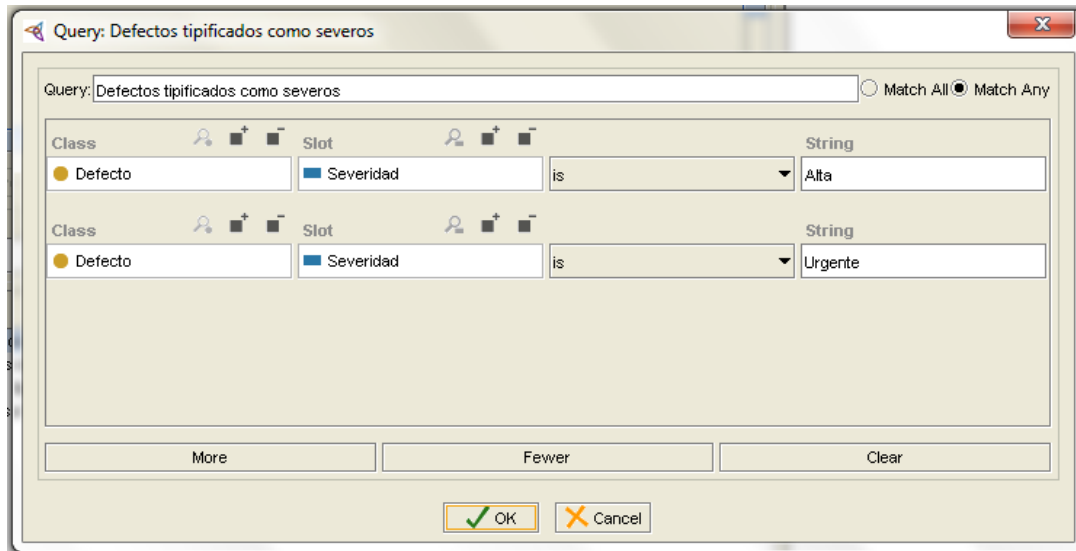


Figura 20 - Consulta: Defectos con severidad *Alta* o *Urgente*

El resultado es presentado en la Tabla 21.

Tabla 21 - instancias para Defectos con severidad *Alta* o *Urgente*

Código	Proyecto
CAJA_01	CAJA
CAJA_02	CAJA
CAJA_05	CAJA
CAJA_06	CAJA
CAJA_07	CAJA
DSS_01	DSS
DSS_02	DSS
GRP_05	GRP
GRP_06	GRP
GRP_07	GRP
GRP_09	GRP
GRP_10	GRP
IBF2_01	IBERINFO
IBF2_02	IBERINFO
IBF2_04	IBERINFO
IBF2_05	IBERINFO
IBF2_06	IBERINFO
IBF2_07	IBERINFO
IBF2_08	IBERINFO
IBF2_09	IBERINFO

IBF2_10	IBERINFO
IBF2_11	IBERINFO
IBF2_12	IBERINFO
IBF2_13	IBERINFO
IBF2_14	IBERINFO
IBF2_15	IBERINFO
IBF2_16	IBERINFO
IBF2_17	IBERINFO
IBF2_18	IBERINFO
IBF2_19	IBERINFO
IBF2_20	IBERINFO
IBF2_21	IBERINFO
IBF2_23	IBERINFO
IBF2_24	IBERINFO
IBF2_25	IBERINFO
PNR_04	PNR
PNR_07	PNR
PNR_09	PNR
CAJA_03	PNR
CAJA_04	PNR
EUF_01	EUF
EUF_02	EUF
Nominas_01	NOMINAS
Nominas_02	NOMINAS
Nominas_03	NOMINAS
Nominas_04	NOMINAS
Nominas_05	NOMINAS
Nominas_06	NOMINAS
Nominas_07	NOMINAS
PNR_01	PNR
PNR_06	PNR
PNR_08	PNR

El siguiente paso (2) es conocer las vinculaciones de los defectos obtenidos en (1) con las instancias del tipo Causa. Para ellos se diseña una consulta que retorne las instancias del tipo Relación para los defectos con severidad *Alta* o *Urgente*.

La consulta se presenta en la Figura 21.

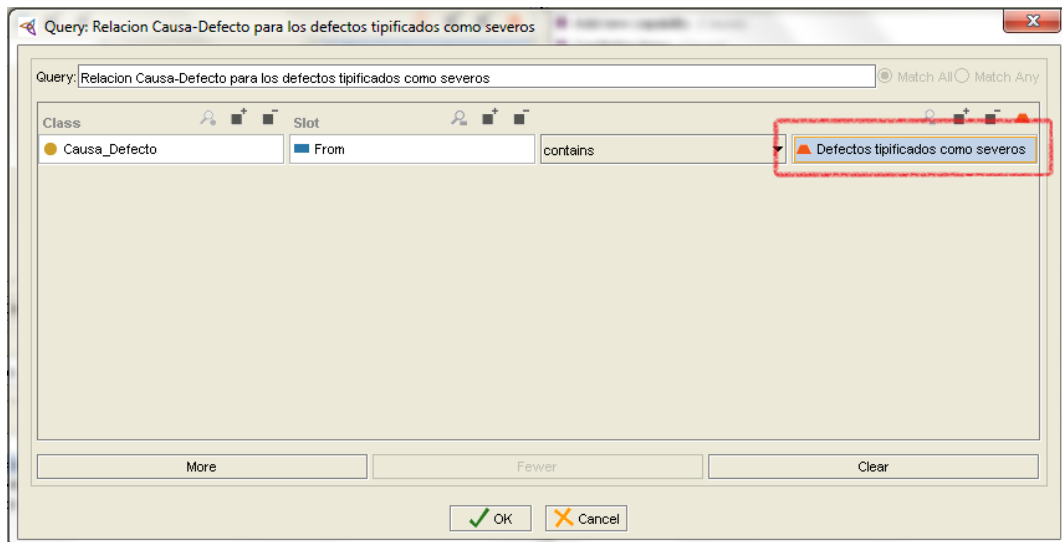


Figura 21 - Relación Causa-Defecto para los defectos con severidad *Alta* o *Urgente*

El último paso (3) para conocer cuáles son las causas que lideran los defectos, es tomar las causas asociadas a las relaciones del tipo *Causa_Defecto* para los defectos con severidad *Alta* o *Urgente*. La Figura 22 enseña la consulta en Protege.

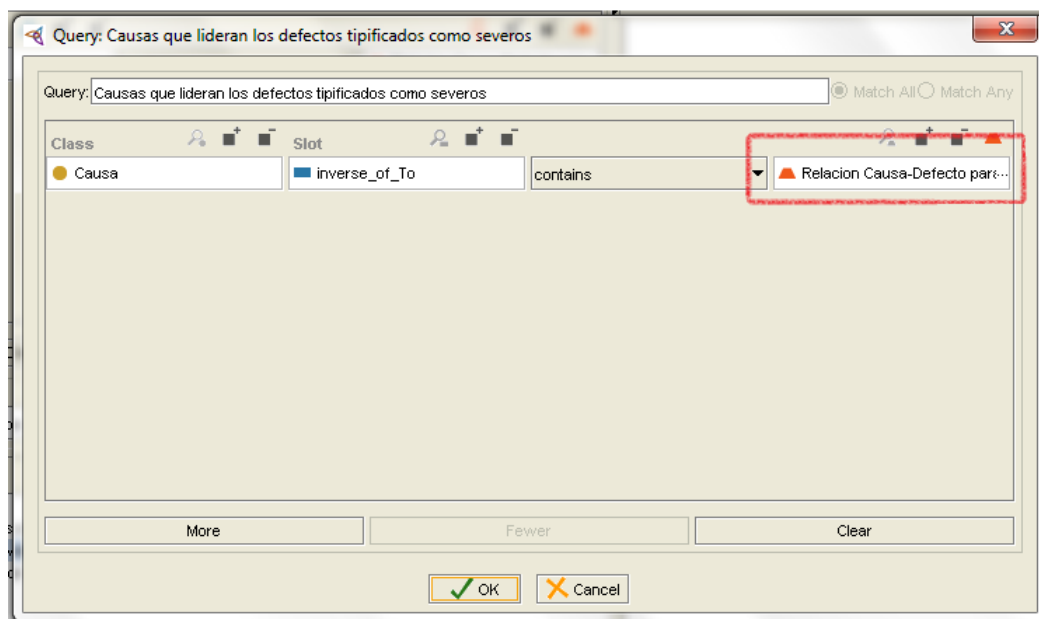


Figura 22 - Causas que lideren los defectos con severidad *Alta* o *Urgente*

El resultado de la consulta anterior, es un conjunto de instancias del tipo *Causa* las cuales responden a las causas asociadas a las relaciones identificadas con las consultas anteriores.

La lista de instancias asociadas al resultado de esta consulta se presenta en la Figura 23.

- ◆ Add new capability (Causa)
- ◆ Conflicting items (Causa)
- ◆ Confusing item (Causa)
- ◆ Equation insufficient or incorrect (Causa)
- ◆ Extreme conditions neglected (Causa)
- ◆ Failure caused by a previous fix (Causa)
- ◆ Forgotten cases or steps (Causa)
- ◆ Initialized data incorrectly (Causa)
- ◆ Input data incorrect or missing (Causa)
- ◆ Interrupts handled incorrectly (Causa)
- ◆ Missing condition test (Causa)
- ◆ Operator in equation incorrect (Causa)
- ◆ Output data incorrect or missing (Causa)
- ◆ Performance problem (Causa)
- ◆ Rounding or truncation fault (Causa)
- ◆ Scaling or units of data incorrect (Causa)
- ◆ Scope of data incorrect (Causa)
- ◆ Sensor data incorrect or missing (Causa)
- ◆ Standards conformance problem (Causa)
- ◆ Wrong subroutine called (Causa)

Figura 23 - instancias para las causas que lideren los defectos con severidad *Alta* o *Urgente*

Con lo presentado hasta aquí, se ejemplifica un caso que da respuesta a la primera de las preguntas de competencia planificadas: *Dado los pasados proyectos en el contexto de una organización, ¿cuáles son las causas que lideran un tipo particular de defecto?* donde el tipo de defecto en particular queda dado por la severidad del defecto.

Resolución: Pregunta 2

Cuando de repite el proceso presentado previamente, para diferentes tipificaciones de los defectos (nuevamente a partir de la combinación de los diferentes atributos del defecto), se da respuesta a la segunda de las preguntas: *Dado los proyectos pasados en el contexto de una organización, ¿cuáles son las causas que lideran un conjunto de tipos de defectos?*

Por ejemplo al combinar los resultados obtenidos al aplicar el procedimiento de resolución seguido para responder a la pregunta 1 a los defectos con severidad *Alta* o *Urgente* y a los defectos detectados durante la fase de pruebas de aceptación, se habrán obtenido las causas que lideran ambas tipificaciones.

La Figura 24 presenta el ejemplo modelado en Protégé.

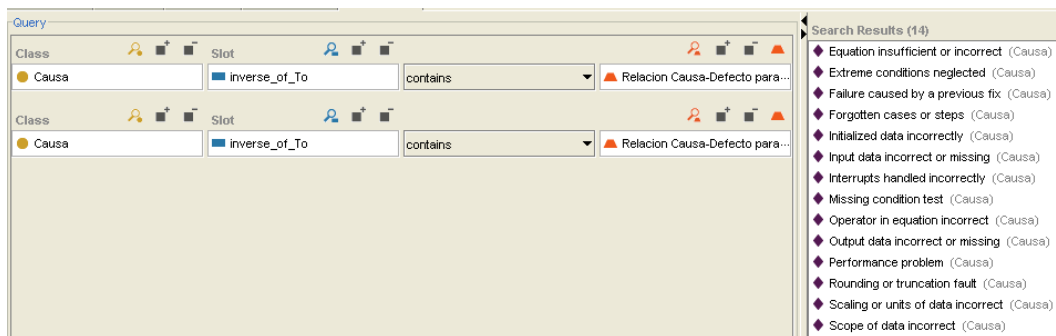


Figura 24 – Causas para defectos con severidad *Alta* o *Urgente* y defectos detectados durante pruebas de aceptación

Resolución: Pregunta 3

Dado un grupo de defectos bajo una tipificación particular y una causa en particular, al combinar el proceso anterior con la causa en particular se está dando respuesta a la tercera de las preguntas: *¿Dado los pasados proyectos en el contexto de una organización, es posible encontrar que una causa en particular lidere un tipo de defecto en particular?*

Ejemplificando esta última situación, ya se conoce (del ejemplo anterior) cuales son las causas que lideran los defectos tipificados como severos, pero se busca saber si una de ellas es específicamente la causa *Forgotten cases or steps*. La Figura 25 muestra la consulta en el diseñador de Protégé.

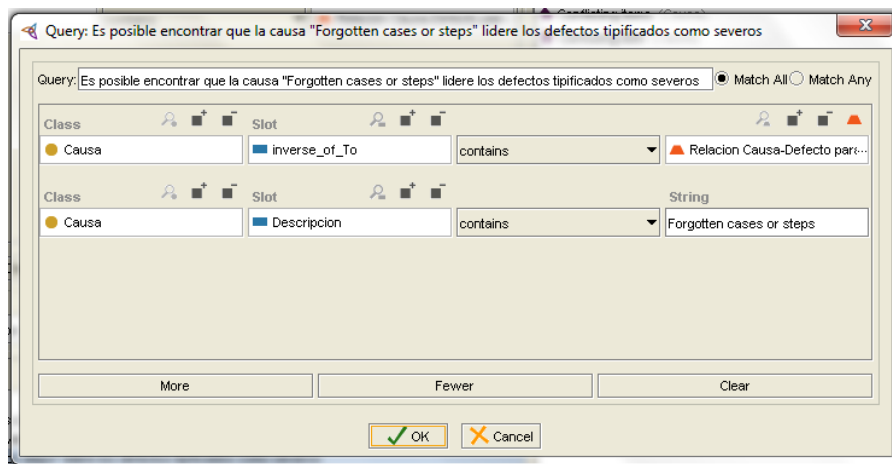


Figura 25 - Causa particular que lidere un tipo de defecto puntual

Si la causa es una de las que lidere el tipo de defecto esperado, la ontología devolverá la instancia consultada, en otro caso no devolverá ninguna instancia con lo cual puede ser utilizado como respuesta del tipo Booleana, y así responder a la pregunta.

La figura 26 muestra el resultado obtenido para la consulta.

◆ Forgotten cases or steps (Causa)

Figura 26 – La causa Forgotten cases or steps lidere el tipo de defecto particular

5.3.3 Resumen

Del ejercicio de resolución desarrollado para las preguntas de competencia se observa cómo a partir de consultas a la ontología es posible obtener información procesable que puede ser utilizada para mejorar las capacidades de análisis de una organización.

Con la ontología para DCA desarrollada ([sección 4](#)) y validada ([sección 5](#)), el siguiente paso es evolucionar el conocimiento que se obtiene de la ontología. Mejorar el conocimiento depende de la evolución de la información en la ontología, esto se realiza mediante un proceso de alimentación de la ontología con nuevos datos y realizando nuevas preguntas de competencia.

Al cargar los datos en la ontología, se modela el defecto, la causa para el defecto y la relación causal correspondiente realizando así el proceso completo de análisis causal del defecto en cuestión directamente con la ontología como si se utilizara alguna de las herramientas por ejemplo diagramas de Ishikawa.

La Figura 27 resume el método para la evolución de la información en la ontología.

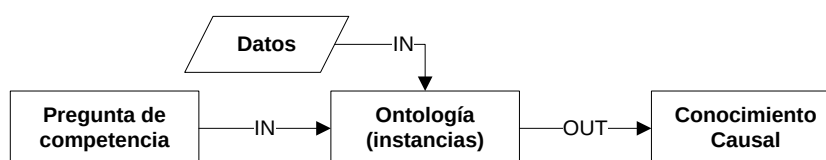


Figura 27 - Método para generación de información ontológica

Como se ve en la Figura 27, el ejercicio de validación mostró cómo es posible obtener conocimiento del problema causal a partir de la ontología. Partiendo de una base de defectos (la ontología) y mediante la aplicación de preguntas de competencia a las instancias en la ontología se puede determinar nuevo conocimiento sobre los defectos de una organización como por ejemplo:

- a) ¿Cuáles son las causas que lideran cuales tipos de defectos?
- b) ¿Cuáles son las causas que lideran un tipo de defecto en particular?
- c) ¿Existe una causa que lidera un tipo de defecto en particular?

El método para obtener conocimiento de la ontología podría resumirse en los siguientes pasos:

1. Modelar los datos de defectos (y su contexto) en la ontología, siguiendo el esquema presentado en la [sección 5.2](#)
2. Con los datos modelados, introducir una consulta en la ontología y ejecutarla, siguiendo el procedimiento especificado en la [sección 5.3.1](#)
3. Al ejecutar la consulta introducida en 2) se obtiene un listado de instancias de la ontología que puede ser utilizado como información de entrada a los procesos de análisis de la organización.

6. Conclusiones y futuras líneas de investigación

Gestionar defectos es aun una tarea compleja para muchas organizaciones. El análisis de los defectos, cuando se realiza, usualmente no presenta los mecanismos adecuados para aprender de los problemas principales y las causas raíces provocando que las organizaciones de software tengan problemas en la adopción de modelos de mejora que guíen la implementación de actividades de DCA en sus proyectos de software.

En este trabajo de tesis se presentó un enfoque para DCA conducido por ontologías, este enfoque facilita la captura y reutilización del conocimiento causal adquirido durante la ejecución del proceso de DCA. Entendemos que este mecanismo facilitará la adopción de procesos de DCA en organizaciones de software.

Se presentó una implementación para el enfoque propuesto, a partir del desarrollo de una ontología que permite capturar el conocimiento causal, brindando una representación formal y explícita de las entidades identificadas en el dominio de DCA.

Con el objetivo de evaluar la capacidad de representación de la ontología diseñada y su aplicabilidad al dominio del DCA, se realizaron dos etapas de validación.

La primera etapa se realizó durante la ejecución de la metodología de desarrollo, en donde se somete a la ontología a un ejercicio de DCA a partir de datos obtenidos de una experiencia de laboratorio. Esta experiencia permitió identificar debilidades de la ontología obtenida, evolucionando a una ontología más cercana al dominio de DCA. Esta nueva versión de la ontología es sometida al mismo ejercicio de validación.

En la segunda etapa, la ontología fue sometida a un ejercicio de validación a partir de datos de defectos cedidos por una empresa. Esta experiencia resume un total de 66 defectos cargados en la ontología, lo cual permitió modelar el conocimiento causal de la empresa. Validando la ontología, y a su vez permitió identificar las debilidades en los procesos de producción de la empresa.

La alternativa de aplicar ontologías al DCA surge con la realización de una SR, cuyo objetivo fue el identificar cómo las organizaciones de software están realizando las actividades relacionadas con DCA en sus procesos.

Se ha mostrado que ninguna de las herramientas de soporte al DCA identificadas durante la revisión propone un enfoque de aprendizaje que facilite la captura y reutilización del conocimiento adquirido durante las actividades de DCA que realice una organización de software.

Se ha visto que las relaciones causales responden a un enfoque ontológico, describiendo objetivamente las restricciones físicas al dominio de interés, mientras las relaciones probabilísticas responden a un enfoque epistemológico, reflejando lo que sabemos o creemos sobre nuestro mundo.

Es entonces que surgió la aplicación de un enfoque ontológico al DCA, ya que las ontologías permiten (modelando *Slots* o propiedades y *Facets* o restricciones) describir de forma objetiva restricciones al dominio de DCA. Además las ontologías son una herramienta para gestión de

conocimiento, con lo cual su aplicación permitirá la captura y reutilización del conocimiento adquirido durante las actividades de DCA.

En resumen, las principales aportaciones de este trabajo de tesis son:

❖ *Una revisión sistemática del estado del arte de las herramientas de DCA*

La SR identificó un total de 14 técnicas aplicadas a DCA en diferentes contextos. Del total de las técnicas identificadas: 3 han sido utilizadas para realizar análisis causal en otras áreas de conocimiento pero aun no han sido aplicadas en el contexto de ingeniería de software, 5 han sido solo propuestas de aplicación y finalmente las 6 restantes cuentan con antecedentes de aplicación en IS. A partir de los resultados de la revisión, se ha visto que:

- Las herramientas identificadas representan conocimiento relacionado con el mismo dominio. Su objetivo es determinar si existe una influencia causal entre los conceptos involucrados en un proceso. Además ninguna brinda la posibilidad de representar restricciones al dominio de interés.
- Existe una diferencia conceptual respecto del tipo de relación que involucran las herramientas. Es posible diferenciar entre aquellas que son capaces de inducir relaciones causales (*causalidad*) de aquellas que son capaces de inducir relaciones probabilísticas (*asociación*).

Para aportar una solución a este problema se presentó un enfoque ontológico para el DCA.

❖ *Un enfoque ontológico para DCA*

Este trabajo mostró la viabilidad de la aplicación de un enfoque ontológico al DCA en organizaciones de software. Para cada una de las características de una ontología se ha indicado la forma en que estas pueden ofrecer soporte a las actividades de DCA de una organización.

❖ *Una oportunidad para mejorar la gestión del conocimiento en DCA*

El enfoque ontológico esta soportado por la aplicación de ontologías. Estas permitirán la mejora en la gestión y administración del conocimiento que las organizaciones de software generen durante el proceso.

❖ *Una ontología para la gestión del conocimiento causal*

La resolución de esta tesis se complementa con el diseño de una ontología que modela el conocimiento relacionado en el dominio del DCA. El diseño de la ontología ha permitido complementar el enfoque ontológico propuesto.

❖ *Un escenario de validación para la ontología*

Se realizaron validaciones con defectos de laboratorio y mediante la creación de 66 instancias con defectos tomados de una organización de software.

❖ *Una ontología para compartir con la comunidad*

Se comparte con la comunidad una ontología capaz de modelar el conocimiento causal. De esta forma permitir la evolución de la ontología con la colaboración de la comunidad según avanza el conocimiento respecto de DCA. Como medio de publicación se utilizó el sitio institucional dispuesto por Stanford para proyectos que utilizan Protégé para el desarrollo y validación de ontologías.

Disponible en (última fecha de acceso 24/05/2011)

<http://protege.cim3.net/cgi-bin/wiki.pl?ProjectsThatUseProtegeLtoQ>

6.1 Futuras líneas de investigación

Esa sección presenta las oportunidades a considerar para futuras líneas de investigación que se han identificado con este trabajo de tesis:

❖ *Evolución de la ontología: una limitante la temporalidad*

La ontología presentada ha sido desarrollada sobre una base de conceptos que fueron abstraídos desde un conjunto de herramientas utilizadas por la comunidad de ingeniería de software para DCA. Considerar para la ontología únicamente los conceptos involucrados en estas herramientas puede no ser suficiente a fin de modelar los conceptos detrás del análisis de relaciones causales en DCA.

Por lo cual la ontología presentada en esta tesis podría evolucionar conforme evoluciona el conocimiento sobre DCA en el ámbito de la comunidad de ingeniería de software.

De hecho la ontología presentada tiene una limitante al respecto del modelado de relaciones causales y la representación de temporalidad en las relaciones entre una causa y un efecto, concepto que es necesario para demostrar causalidad entre dos eventos.

❖ *Estudio del isomorfismo de las representaciones de DCA.*

Se ha realizado un ejercicio que permitió conjeturar que las representaciones gráficas de las herramientas para DCA tienen una capacidad de representación de conceptualizaciones similar. Un camino a desarrollar es: analizar desde una perspectiva formal si las representaciones son isomorfas.

❖ *Estudio empírico de la eficacia y eficiencia*

Por otra parte se ha conjeturado que al tratarse de ontologías, estas deberían facilitar la gestión del conocimiento causal adquirido por una organización de forma de realizar las actividades de DCA de forma más eficaz y eficiente. Es necesario validar empíricamente esta conjetura mediante un estudio de campo.

❖ *Soporte a la automatización del proceso*

La implantación exitosa de la ontología como herramienta para DCA dependerá de la existencia de sistemas de software que brinden este soporte. Protégé es una herramienta para tratar con ontologías, pero presenta algunas limitaciones:

- El mecanismo de consulta (*query*) sobre la base de conocimiento en la ontología es limitado. Las herramientas de edición no son amigables.
- Se brindan formularios para la carga de datos, pero al realizar algunas modificaciones estructurales en la ontología la información cargada previamente queda inconsistente.
- La definición de tipo de defecto es un concepto relacionado con la ontología particular con lo cual Protégé no está diseñado para tratar específicamente con ello.
- Al no estar diseñado Protégé para tratar específicamente con la ontología diseñada las capacidades de análisis que brinda Protégé están limitadas a las consultas (*query*) que se realicen sobre la ontología.

Debido al avance que se adquirió sobre esta línea, se presentan algunos de los requisitos que este sistema de software debería de contemplar en su alcance.

Los requisitos son los siguientes:

- *Almacenamiento*: El aplicativo debe brindar capacidades de almacenamiento para la información en la ontología de modo que además pueda ser consumida por otros sistemas de información. Por lo tanto la opción más adecuada sería un modelo relacional de bases de datos, dada las características de estos modelos y su facilidad para manipular grandes volúmenes de información. La propuesta consiste en que cada elemento de una ontología tenga su representación homónima en el esquema relacional, de modo que sea posible realizar una correspondencia punto a punto entre cada entidad modelada en la ontología y el esquema relacional resultante.
- *Mantenimiento de la ontología*: Una ontología define un lenguaje común para una comunidad, por lo cual la ontología podría evolucionar conforme evoluciona el concepto de defecto en el ámbito de ingeniería de software. Por lo tanto el sistema de software debe disponer de un módulo que facilite el mantenimiento de la ontología (entidades, propiedades, restricciones) facilitando la evolución del conocimiento causal capturado en ella.
- *Modelado de datos en la ontología*: El sistema de software debe ser capaz de brindar esta facilidad; de modo que los usuarios del aplicativo sean capaces de cargar información que aun no ha sido evaluada. De esta manera evolucionar la base de conocimiento a partir de información obtenida de los procesos, sin que sea necesario que haya sido analizada en actividades de DCA previamente a realizar el modelado.

- *Definición de tipos de defecto:* Para responder a las preguntas para las cuales la ontología ha sido diseñada, es importante definir qué se entiende por tipo de defecto dado un contexto en particular. Por lo tanto, el sistema de software debe permitir a los usuarios modelar los tipos de defecto que interese a su organización analizar. El modelado se brinda a partir de las diferentes combinaciones de los atributos del defecto, de modo que se puedan realizar diferentes sesgos en la información para aportar diferentes visiones durante el análisis.
- *Iniciar un nuevo análisis:* El inicio de un nuevo análisis, implica que el sistema de software debe permitir iniciar una nueva instancia de análisis para un defecto en particular. Además debe permitir analizar la evolución de los resultados en respuesta a las preguntas de competencia para las cuales la ontología ha sido diseñada.

7. Referencias

- Ackermann, Fran, Colin Eden, *et al.* (1992). Getting Started With Cognitive Mapping. 7th Young OR Conference. University of Warwick, Decision Explorer Workshop Edition 2004: 65-82.
- Bento, Al and Regina Bento (2004). "The use of causal analysis techniques in information system research: a methodological note." Journal of Information Technology Management, Published by Association of Management, Journal of Information Technology Management XV, Numbers 3-4.
- Bezerra, Carla, Adriano Albuquerque, *et al.* (2008). MiniDMAIC: An Approach for Causal Analysis and Resolution in Software Development Projects. Advances in Computer and Information Sciences and Engineering, Springer Science+Business Media B.V.
- Buglione, Luigi (2006). Introducing Root-Cause Analysis and Orthogonal Defect Classification at Lower CMMI Maturity Levels. MENSURA 2006. M. 2006. Cadiz, España: 29-40.
- Buglione, Luigi (2008). Strengthening CMMI Maturity Levels with a Quantitative Approach to Root-Cause Analysis. 5th Software Measurement European Forums. Milan, Italia: 67-82.
- Buzan, Tony, Ed. (2002). How to Mind Map. Londres, Harpers Collins Published Ltd.
- Buzan, Tony and Bo Buzan, Eds. (1994). The Mind Map Book, How To Use Radiant Thinking to Maximize Your Brain's Untapped Potential. The Mind Map Book. Londres, BBC Books, 1993.
- Calvanese, Diego, Maurizio Lenzerini, *et al.* (1994). A Unified Framework for Class-Based Representation Formalisms. Appeared in Proc. of KR'9.
- Card, David (1993). "Defect Causal Analysis Drives Down Error Rates." IEEE Software 1/1993 **10**(4): 98-99.
- Card, David (1998). Learning from our mistakes with defect causal analysis. SOFTWARE PRODUCTIVITY CONSORTIUM. IEEE, IEEE Xplore. **15**: 56-63.
- Card, David (2004). Understanding Causal Systems. Crosstalk.
- Card, David (2006). Myths and Strategies of Defect Causal Analysis. Pacific Northwest Software Quality Conference: 6.
- Connolly, Dan, Frank van Harmelen, *et al.* (2001) "DAML+OIL (March 2001) Reference Description."
- Chaib-draa, Bo and Jo Desharnais (1998) "A Relational Model of Cognitive Maps."
- Chillarege, Ram, Inderpal Bhandri, *et al.* (1992). "Orthogonal Defect Classification - A Concept for In-Process Measurement." IEEE Transaction on Software Engineering **18**(11): 943-956.
- Dyba, Tore, Vigdis By Kampenes, *et al.* (2006). "A systematic review of statistical power in software engineering experiments" Information and Software Technology. Information and Software Technology **48**(1): 745-755.
- Eckes, George., Ed. (2000). The Six Sigma Revolutions: How General Electric and Others Turned Process Into Profits. The Six Sigma Revolutions. USA, John Wiley and Sons.
- Farlex. (2010). "The Free Dictionary." from <http://www.thefreedictionary.com/>.
- Fernández-López, M., A. Gómez-Pérez, *et al.* (1999). Building a Chemical Ontology Using Methontology and the Ontology Design Environment. IEEE Intelligent Systems & their applications. **January/February 1999**: 37-46.
- Gómez, Asunción, Natalia Juristo, *et al.* (1997). Ingeniería del Conocimiento. Tomás Bretón, 21, 28045, Madrid, España, Editorial Centro de Estudios Ramón Areces SA.

- Grady, R. B. (1992). Practical Software Metrics for Project Management and Process Improvement, Prentice-Hal.
- Grimm, Stephan, Pascal Hitzler, *et al.* (2007). "Knowledge Representation and Ontologies: Logic, Ontologies and Semantic Web Languages." Springer.
- Heijst, G. van, A. Th. Schreiber, *et al.* (1997). "Using Explicit Ontologies in KBS Development." International Journal of Human-Computer Studies. **46**(2/3): 183-292.
- Henderson, Craig (2008). "Managing Software Defects: Defect Analysis and traceability." ACM SIGSOFT Software Engineering Notes **33**, Number 4(4): 1-3.
- Holland, Paul W. (1986). "Statistics and Causal Inference." Journal of the American Statistical Association **81**(396): 945-960.
- Humphrey, Watts S. (2000). The Personal Software Process (PSP), Software Engineering Institute 40.
- IEEE (1990). IEEE Std 610.12-1990, IEEE Standard Glossary of Software Engineering Terminology. IEEE Std 610.12-1990. I. Press. New York, USA, IEEE Standards Board. **Standard 610**.
- IEEE, Ed. (1993). IEEE Standard Classification for Software Anomalies, IEEE Std 1044. 345 East 47th Street, New York, NY 10017-2394, USA, IEEE Standards Board.
- Ishikawa, Karou, Ed. (1976). Guide to Quality Control. Tokyo, Asian Productivity Organization.
- ISO (2004). ISO/IEC 12207: Information technology – Software life-cycle processes. ISO/IEC 12207. ISO/IEC. Geneve, ISO, Int. Org. for Standardization and the Int. Electrotechnical Commission. **Amendment 2**.
- Itabashi, Yoshinori and Koichiro Ochimizu (2008). Prediction of organization-specific process weaknesses.
- Jalote, Pankaj and Naresh Agrawal (2005). Using Defect Analysis Feedback for Improving Quality and Productivity in Iterative Software Development. 3rd Int. Conf. on Information and Communication Technology. ICICT. Cairo, ICICT: 701-713.
- Jørgensen, Magne and Martin Shepperd (2007). "A Systematic Review of Software Development Cost Estimation Studies." IEEE Transaction on Software Engineering **33**(1): 33-53.
- Juristo, Natalia and Ana M. Moreno (2001). Basic of Software Engineering Experimentation, Kluwer Academic Publisher, Boston.
- KACTUS. (1996). "The KACTUS Booklet version 1.0 Esprit Project 8145." 2010, from <http://www.swi.psy.uva.nl/projects/NewKACTUS/Reports.html>.
- Kalinowski, Marcos, Emilia Mendes, *et al.* (2010). Applying DPPI: A Defect Causal Analysis Approach Using Bayesian Networks. PROFES 2010, LNCS 6156, Springer.
- Kalinowski, Marcos and Guilherme Travassos (2008). A systematic review regarding software defect causal analysis, COPPE/UFRJ: 158.
- Kalinowski, Marcos, Guilherme Travassos, *et al.* (2008a). Guidance for Efficiently Implementing Defect Causal Analysis. VII Simpósio Brasileiro de Qualidade de Software. Florianópolis-SC, Anais do VII Simpósio Brasileiro de Qualidade de Software.
- Kalinowski, Marcos, Guilherme Travassos, *et al.* (2008b). Towards a Defect Prevention Based Process Improvement Approach. IEEE, 34th Euromicro Conference Software Engineering And Advanced Applications. IEEE. Europe, IEEE Xplore. **10.1109/SEAA.2008.47: 8**.
- Kalyanpur, Aditya, Bijan Parsia, *et al.* (2005) "Swoop: A 'Web' Ontology Editing Browser." 20.
- Khondoker, M. Rahamatullah and Paul Mueller (2010). Comparing Ontology Development Tools Based on an Online Survey. the World Congress on Engineering 2010, London, U.K., WCE 2010.

- Kitchenham, Barbara A. (2007) "Guidelines for performing Systematic Literature Reviews in Software Engineering, Version 2.3." 65.
- Klyne, G. and J. Carroll (2004) "RDF Concepts and Abstract Syntax."
- Lauritzen, Steffen (1996). Graphical models, Oxford University Press.
- Lopez, Mario Fernandez (1999). "Overview Of Methodologies For Building Ontologies." Proceedings of the IJCAI-99 workshop on Ontologies and Problem-Solving Methods (KRR5).
- Matalonga, Santiago and Tomas San Feliu (2008). Linking Return on Training Investment with Defects Causal Analysis. Twentieth International Conference on Software Engineering & Knowledge Engineering (SEKE'2008), S. I. G. S. 2008. San Francisco, CA, USA, Systems Institute Graduate School 2008: 42-47.
- Mian, Paula, Tayana Conte, *et al.* (2005) "A Systematic Review Process for Software Engineering."
- Mizukami, D (2007). Analyzing Defects Can Tell a LOT About a Company. SEPG Conference 2007.
- Murphy, Kevin P. (2001). An Introduction to Graphical Models.
- Noy, Natalya F. and Deborah L. Guinness. (2000). "Ontology Development 101: A Guide to Creating Your First Ontology." Retrieved 10/12/2009, 2009, from http://protege.stanford.edu/publications/ontology_development/ontology101.html.
- Oslapas, Arunas (1993). Beyond Brainstorming: Creative Problem-Solving Techniques. Frontiers in Education Conference. IEEE, Western Washington University, Poster 29: 837.
- Patel-Schneider, P.F., P. Hayes, *et al.* (2009) "OWL Web Ontology Language; Semantics and Abstract Syntax."
- Pearl, Judea (2000). Causality: Models, Reasoning, and Inference. California LA, USA, Cambridge Univeristy Press.
- Pearl, Judea (2009). "Causal Inference in Statistics: An Overview." Statistics Surveys 3 96-146.
- Pérez, Asunción Gómez (2002) "OntoWeb Ontology-based information exchange for knowledge management and electronic commerce IST-2000-29243 Deliverable 1.3: A survey on ontology tools." 96.
- PMI (2008). A Guide to the Project Management Body of Knowledge. Newtown Square, Pennsylvania, USA, Project Manager Institute, Inc.
- Pyzdek, Thomas (2003). The Six Sigma Handbook, a Complete Guide for Green Belts, Black Belts and Managers at All Levels. The Six Sigma Handbook. McGraw-Hill. New York, USA, McGraw-Hill. 1: 850.
- Reformat, Marek and Efe Igide (2005) "Isolation of Defects: Extracting Knowledge with Confidence." 114-119 DOI: 0-7803-9093-8/05/\$20.00 ©2005 IEEE.
- Sander, Greenland and Judea Pearl (2006). "Causal Diagrams." Encyclopedia of Epidemiology(Technical Report R-332): 12.
- SEI (2006). CMMI for Development (CMMI-DEV). CMMI-DEV. C. M. University. Pittsburgh, PA, Software Engineering Institute. **Version 1.2**: 253.
- Senge, Peter M., Ed. (2004). The fifth discipline, The art & practice of the Learning Organizations. USA, Doubleday Business.
- Shah, Rachna and Susan Goldstein (2005). "Use of Structual Equation Modeling in operations management research:Looking back and forward." Journal of Operations Management 2005: 22.
- Siau, Keng and Xin Tan (2005) "Use of Cognitive Mapping Techniques in Information System." 4.
- Sowa, John F. (2001). "Glossary." Retrieved Noviembre, 2010, from <http://users.bestweb.net/~sowa/ontology/gloss.htm>.

- Swartout, B., Ramesh P., *et al.* (1997). Toward Distributed Use of Large-Scale Ontologies. Symposium on Ontological Engineering of AAAI, Stanford (California).
- Uchold, M. and M. King (1995). Towards a Methodology for Building Ontologies. Workshop on Basic Ontological Issues in Knowledge Sharing.
- van Oorschot, K.E., Sengupta K., *et al.* (2009). "Dynamics of Agile Software Development." 27th International Conference of the System Dynamics Society, Albuquerque, New Mexico.
- VanMoll, J.H., J. C. Jacobs, *et al.* (2002). The Importance of Life Cycle Modeling to Defect Detection and Prevention. 10th International Workshop on Software Technology and Engineering Practice, 2002. STEP 2002. S. 2002: 144-155.
- Vegas, Sira, Natalia Juristo, *et al.* (2009). "Maturing Software Engineering Knowledge through Classification: A Case Study on Unit Testing Techniques." IEEE Transaction on Software Engineering 35(4).
- Vicente, André Abe, Marcio Eduardo Delamaro, *et al.* (2008). Uma Revisao Sistemática sobre a atividade de Teste de Software no contexto de Métodos Ágeis.
- Wagner, Stefan (2008). Defects Classification and Defect Types Revisited. Defects'08, ACM. M. A. Universidad Técnica de Munich. Seattle, Washington, USA, ACM: 2.
- Wellman, Po Mo (1994). "Inference in Cognitive Maps." Mathematics and Computers Simulation: 18.

Anexo I: Formalización de la ontología (OWL)

```
<rdf:RDF
  xmlns:xsp="http://www.owl-ontologies.com/2005/08/07/xsp.owl#"
  xmlns:swrlb="http://www.w3.org/2003/11/swrlb#"
  xmlns="http://www.owl-ontologies.com/Ontology1305852097.owl#"
  xmlns:swrl="http://www.w3.org/2003/11/swrl#"
  xmlns:protege="http://protege.stanford.edu/plugins/owl/protege#"
  xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema#"
  xmlns:rdfs="http://www.w3.org/2000/01/rdf-schema#"
  xmlns:owl="http://www.w3.org/2002/07/owl#"
xml:base="http://www.owl-ontologies.com/Ontology1305852097.owl">
<owl:Ontology rdf:about=""/>
<owl:Class>
  <owl:unionOf rdf:parseType="Collection">
    <rdf:Description rdf:about="http://www.w3.org/2002/07/owl#Thing"/>
    <owl:Class rdf:ID="Defecto"/>
  </owl:unionOf>
</owl:Class>
<owl:Class rdf:ID="Canvas_Defecto">
  <rdfs:label rdf:datatype="http://www.w3.org/2001/XMLSchema#string">Canvas_Defecto</rdfs:label>
  <rdfs:subClassOf>
    <owl:Class rdf:ID="Relacion"/>
  </rdfs:subClassOf>
</owl:Class>
<owl:Class rdf:ID="Causa">
  <rdfs:label rdf:datatype="http://www.w3.org/2001/XMLSchema#string">Causa</rdfs:label>
</owl:Class>
<owl:Class rdf:about="#Relacion">
  <rdfs:label rdf:datatype="http://www.w3.org/2001/XMLSchema#string">Relacion</rdfs:label>
</owl:Class>
<owl:Class rdf:ID="Causa_Defecto">
  <rdfs:label rdf:datatype="http://www.w3.org/2001/XMLSchema#string">Causa_Defecto</rdfs:label>
  <rdfs:subClassOf rdf:resource="#Relacion"/>
</owl:Class>
<owl:Class rdf:ID="Canvas">
  <rdfs:label rdf:datatype="http://www.w3.org/2001/XMLSchema#string">Canvas</rdfs:label>
</owl:Class>
<owl:Class>
  <owl:unionOf rdf:parseType="Collection">
    <rdf:Description rdf:about="http://www.w3.org/2002/07/owl#Thing"/>
    <owl:Class rdf:about="#Defecto"/>
  </owl:unionOf>
</owl:Class>
<owl:Class>
  <owl:unionOf rdf:parseType="Collection">
    <rdf:Description rdf:about="http://www.w3.org/2002/07/owl#Thing"/>
    <owl:Class rdf:ID="Causa_Causa"/>
    <owl:Class rdf:about="#Causa_Defecto"/>
  </owl:unionOf>
</owl:Class>
<owl:Class>
  <owl:unionOf rdf:parseType="Collection">
    <rdf:Description rdf:about="http://www.w3.org/2002/07/owl#Thing"/>
    <owl:Class rdf:about="#Relacion"/>
  </owl:unionOf>
</owl:Class>
<owl:Class>
```

```

<owl:unionOf rdf:parseType="Collection">
  <rdf:Description rdf:about="http://www.w3.org/2002/07/owl#Thing"/>
  <owl:Class rdf:about="#Defecto"/>
  <owl:Class rdf:about="#Causa"/>
</owl:unionOf>
</owl:Class>
<owl:Class rdf:about="#Defecto">
  <rdfs:label rdf:datatype="http://www.w3.org/2001/XMLSchema#string">Defecto</rdfs:label>
</owl:Class>
<owl:Class rdf:about="#Causa_Causa">
  <rdfs:subClassOf rdf:resource="#Relacion"/>
  <rdfs:label rdf:datatype="http://www.w3.org/2001/XMLSchema#string">Causa_Causa</rdfs:label>
</owl:Class>
<owl:Class>
  <owl:unionOf rdf:parseType="Collection">
    <rdf:Description rdf:about="http://www.w3.org/2002/07/owl#Thing"/>
    <owl:Class rdf:about="#Causa_Causa"/>
    <owl:Class rdf:about="#Causa_Defecto"/>
  </owl:unionOf>
</owl:Class>
<owl:Class>
  <owl:unionOf rdf:parseType="Collection">
    <rdf:Description rdf:about="http://www.w3.org/2002/07/owl#Thing"/>
    <owl:Class rdf:about="#Causa_Causa"/>
  </owl:unionOf>
</owl:Class>
<owl:Class>
  <owl:unionOf rdf:parseType="Collection">
    <rdf:Description rdf:about="http://www.w3.org/2002/07/owl#Thing"/>
    <owl:Class rdf:about="#Defecto"/>
  </owl:unionOf>
</owl:Class>
<owl:Class>
  <owl:unionOf rdf:parseType="Collection">
    <rdf:Description rdf:about="http://www.w3.org/2002/07/owl#Thing"/>
    <owl:Class rdf:about="#Relacion"/>
    <owl:Class rdf:about="#Defecto"/>
  </owl:unionOf>
</owl:Class>
<owl:Class>
  <owl:unionOf rdf:parseType="Collection">
    <rdf:Description rdf:about="http://www.w3.org/2002/07/owl#Thing"/>
    <owl:Class rdf:about="#Relacion"/>
    <owl:Class rdf:about="#Defecto"/>
    <owl:Class rdf:about="#Causa"/>
  </owl:unionOf>
</owl:Class>
<owl:Class>
  <owl:unionOf rdf:parseType="Collection">
    <rdf:Description rdf:about="http://www.w3.org/2002/07/owl#Thing"/>
    <owl:Class rdf:about="#Causa_Causa"/>
  </owl:unionOf>
</owl:Class>
<owl:ObjectProperty rdf:ID="inverse_of_To">
  <rdfs:range rdf:resource="#Canvas_Defecto"/>
  <rdfs:range rdf:resource="#Causa_Causa"/>
  <rdfs:range rdf:resource="#Causa_Defecto"/>
  <owl:inverseOf>
    <owl:FunctionalProperty rdf:ID="To"/>
  </owl:inverseOf>
  <rdfs:label rdf:datatype="http://www.w3.org/2001/XMLSchema#string">inverse_of_To</rdfs:label>

```

```

<rdfs:domain>
  <owl:Class>
    <owl:unionOf rdf:parseType="Collection">
      <rdf:Description rdf:about="http://www.w3.org/2002/07/owl#Thing"/>
      <owl:Class rdf:about="#Defecto"/>
      <owl:Class rdf:about="#Causa"/>
    </owl:unionOf>
  </owl:Class>
</rdfs:domain>
</owl:ObjectProperty>
<owl:ObjectProperty rdf:ID="inverse_of_From">
  <rdfs:range rdf:resource="#Canvas_Defecto"/>
  <rdfs:range rdf:resource="#Causa_Causa"/>
  <rdfs:range rdf:resource="#Causa_Defecto"/>
  <rdfs:label rdf:datatype="http://www.w3.org/2001/XMLSchema#string"
>inverse_of_From</rdfs:label>
<rdfs:domain>
  <owl:Class>
    <owl:unionOf rdf:parseType="Collection">
      <rdf:Description rdf:about="http://www.w3.org/2002/07/owl#Thing"/>
      <owl:Class rdf:about="#Defecto"/>
      <owl:Class rdf:about="#Causa"/>
      <owl:Class rdf:about="#Canvas"/>
    </owl:unionOf>
  </owl:Class>
</rdfs:domain>
<owl:inverseOf>
  <owl:FunctionalProperty rdf:ID="From"/>
</owl:inverseOf>
</owl:ObjectProperty>
<owl:FunctionalProperty rdf:ID="Categoria">
  <rdf:type rdf:resource="http://www.w3.org/2002/07/owl#DatatypeProperty"/>
  <rdfs:range rdf:resource="http://www.w3.org/2001/XMLSchema#string"/>
  <rdfs:domain>
    <owl:Class>
      <owl:unionOf rdf:parseType="Collection">
        <rdf:Description rdf:about="http://www.w3.org/2002/07/owl#Thing"/>
        <owl:Class rdf:about="#Defecto"/>
        <owl:Class rdf:about="#Causa"/>
      </owl:unionOf>
    </owl:Class>
  </rdfs:domain>
  <rdfs:label rdf:datatype="http://www.w3.org/2001/XMLSchema#string">Categoria</rdfs:label>
</owl:FunctionalProperty>
<owl:FunctionalProperty rdf:ID="Descripcion">
  <rdfs:range rdf:resource="http://www.w3.org/2001/XMLSchema#string"/>
  <rdfs:label rdf:datatype="http://www.w3.org/2001/XMLSchema#string">Descripcion</rdfs:label>
  <rdf:type rdf:resource="http://www.w3.org/2002/07/owl#DatatypeProperty"/>
  <rdfs:domain>
    <owl:Class>
      <owl:unionOf rdf:parseType="Collection">
        <rdf:Description rdf:about="http://www.w3.org/2002/07/owl#Thing"/>
        <owl:Class rdf:about="#Relacion"/>
        <owl:Class rdf:about="#Defecto"/>
        <owl:Class rdf:about="#Causa"/>
        <owl:Class rdf:about="#Canvas"/>
      </owl:unionOf>
    </owl:Class>
  </rdfs:domain>
</owl:FunctionalProperty>
<owl:FunctionalProperty rdf:ID="Fuente">

```

```

<rdfs:range rdf:resource="http://www.w3.org/2001/XMLSchema#string"/>
<rdf:type rdf:resource="http://www.w3.org/2002/07/owl#DatatypeProperty"/>
<rdfs:label rdf:datatype="http://www.w3.org/2001/XMLSchema#string">Fuente</rdfs:label>
<rdfs:domain>
  <owl:Class>
    <owl:unionOf rdf:parseType="Collection">
      <rdf:Description rdf:about="http://www.w3.org/2002/07/owl#Thing"/>
      <owl:Class rdf:about="#Defecto"/>
    </owl:unionOf>
  </owl:Class>
</rdfs:domain>
</owl:FunctionalProperty>
<owl:FunctionalProperty rdf:ID="Prioridad">
  <rdfs:label rdf:datatype="http://www.w3.org/2001/XMLSchema#string">Prioridad</rdfs:label>
  <rdfs:domain>
    <owl:Class>
      <owl:unionOf rdf:parseType="Collection">
        <rdf:Description rdf:about="http://www.w3.org/2002/07/owl#Thing"/>
        <owl:Class rdf:about="#Defecto"/>
      </owl:unionOf>
    </owl:Class>
  </rdfs:domain>
  <rdf:type rdf:resource="http://www.w3.org/2002/07/owl#DatatypeProperty"/>
  <rdfs:range rdf:resource="http://www.w3.org/2001/XMLSchema#string"/>
</owl:FunctionalProperty>
<owl:FunctionalProperty rdf:ID="Sintoma">
  <rdf:type rdf:resource="http://www.w3.org/2002/07/owl#DatatypeProperty"/>
  <rdfs:label rdf:datatype="http://www.w3.org/2001/XMLSchema#string">Sintoma</rdfs:label>
  <rdfs:domain>
    <owl:Class>
      <owl:unionOf rdf:parseType="Collection">
        <rdf:Description rdf:about="http://www.w3.org/2002/07/owl#Thing"/>
        <owl:Class rdf:about="#Defecto"/>
      </owl:unionOf>
    </owl:Class>
  </rdfs:domain>
  <rdfs:range rdf:resource="http://www.w3.org/2001/XMLSchema#string"/>
</owl:FunctionalProperty>
<owl:FunctionalProperty rdf:ID="Resolucion">
  <rdfs:range rdf:resource="http://www.w3.org/2001/XMLSchema#string"/>
  <rdf:type rdf:resource="http://www.w3.org/2002/07/owl#DatatypeProperty"/>
  <rdfs:label rdf:datatype="http://www.w3.org/2001/XMLSchema#string">Resolucion</rdfs:label>
</owl:FunctionalProperty>
<owl:FunctionalProperty rdf:ID="Correctivo">
  <rdf:type rdf:resource="http://www.w3.org/2002/07/owl#DatatypeProperty"/>
  <rdfs:range rdf:resource="http://www.w3.org/2001/XMLSchema#string"/>
  <rdfs:label rdf:datatype="http://www.w3.org/2001/XMLSchema#string">Correctivo</rdfs:label>
</owl:FunctionalProperty>
<owl:FunctionalProperty rdf:ID="Rubro_de_la_organizacion">
  <rdfs:label rdf:datatype="http://www.w3.org/2001/XMLSchema#string">Rubro de la organizacion</rdfs:label>
  <rdfs:domain>
    <owl:Class>
      <owl:unionOf rdf:parseType="Collection">
        <rdf:Description rdf:about="http://www.w3.org/2002/07/owl#Thing"/>
        <owl:Class rdf:about="#Canvas"/>
      </owl:unionOf>
    </owl:Class>
  </rdfs:domain>
  <rdf:type rdf:resource="http://www.w3.org/2002/07/owl#DatatypeProperty"/>

```

```

    <rdfs:range rdf:resource="http://www.w3.org/2001/XMLSchema#string"/>
</owl:FunctionalProperty>
<owl:FunctionalProperty rdf:ID="Peso">
<rdfs:range rdf:resource="http://www.w3.org/2001/XMLSchema#string"/>
<rdfs:domain>
  <owl:Class>
    <owl:unionOf rdf:parseType="Collection">
      <rdf:Description rdf:about="http://www.w3.org/2002/07/owl#Thing"/>
      <owl:Class rdf:about="#Relacion"/>
    </owl:unionOf>
  </owl:Class>
</rdfs:domain>
<rdf:type rdf:resource="http://www.w3.org/2002/07/owl#DatatypeProperty"/>
<rdfs:label rdf:datatype="http://www.w3.org/2001/XMLSchema#string">Peso</rdfs:label>
</owl:FunctionalProperty>
<owl:FunctionalProperty rdf:ID="Severidad">
<rdfs:range rdf:resource="http://www.w3.org/2001/XMLSchema#string"/>
<rdf:type rdf:resource="http://www.w3.org/2002/07/owl#DatatypeProperty"/>
<rdfs:label rdf:datatype="http://www.w3.org/2001/XMLSchema#string">Severidad</rdfs:label>
<rdfs:domain>
  <owl:Class>
    <owl:unionOf rdf:parseType="Collection">
      <rdf:Description rdf:about="http://www.w3.org/2002/07/owl#Thing"/>
      <owl:Class rdf:about="#Defecto"/>
    </owl:unionOf>
  </owl:Class>
</rdfs:domain>
</owl:FunctionalProperty>
<owl:FunctionalProperty rdf:ID="Tipo_de_organizacion">
  <rdfs:label rdf:datatype="http://www.w3.org/2001/XMLSchema#string">Tipo de organización
</rdfs:label>
<rdfs:domain>
  <owl:Class>
    <owl:unionOf rdf:parseType="Collection">
      <rdf:Description rdf:about="http://www.w3.org/2002/07/owl#Thing"/>
      <owl:Class rdf:about="#Canvas"/>
    </owl:unionOf>
  </owl:Class>
</rdfs:domain>
<rdfs:range rdf:resource="http://www.w3.org/2001/XMLSchema#string"/>
<rdf:type rdf:resource="http://www.w3.org/2002/07/owl#DatatypeProperty"/>
</owl:FunctionalProperty>
<owl:FunctionalProperty rdf:ID="Valor_a_Cliente">
<rdfs:domain>
  <owl:Class>
    <owl:unionOf rdf:parseType="Collection">
      <rdf:Description rdf:about="http://www.w3.org/2002/07/owl#Thing"/>
      <owl:Class rdf:about="#Defecto"/>
    </owl:unionOf>
  </owl:Class>
</rdfs:domain>
<rdfs:range rdf:resource="http://www.w3.org/2001/XMLSchema#string"/>
<rdfs:label rdf:datatype="http://www.w3.org/2001/XMLSchema#string">Valor a Cliente</rdfs:label>
<rdf:type rdf:resource="http://www.w3.org/2002/07/owl#DatatypeProperty"/>
</owl:FunctionalProperty>
<owl:FunctionalProperty rdf:ID="Fase_del_proyecto">
<rdfs:range rdf:resource="http://www.w3.org/2001/XMLSchema#string"/>
<rdf:type rdf:resource="http://www.w3.org/2002/07/owl#DatatypeProperty"/>
<rdfs:domain>
  <owl:Class>
    <owl:unionOf rdf:parseType="Collection">

```

```

    <rdf:Description rdf:about="http://www.w3.org/2002/07/owl#Thing"/>
    <owl:Class rdf:about="#Defecto"/>
  </owl:unionOf>
</owl:Class>
</rdfs:domain>
<rdfs:label rdf:datatype="http://www.w3.org/2001/XMLSchema#string"
>Fase del proyecto</rdfs:label>
</owl:FunctionalProperty>
<owl:FunctionalProperty rdf:ID="Repetibilidad">
  <rdfs:label rdf:datatype="http://www.w3.org/2001/XMLSchema#string">Repetibilidad</rdfs:label>
  <rdf:type rdf:resource="http://www.w3.org/2002/07/owl#DatatypeProperty"/>
  <rdfs:domain>
    <owl:Class>
      <owl:unionOf rdf:parseType="Collection">
        <rdf:Description rdf:about="http://www.w3.org/2002/07/owl#Thing"/>
        <owl:Class rdf:about="#Defecto"/>
      </owl:unionOf>
    </owl:Class>
  </rdfs:domain>
  <rdfs:range rdf:resource="http://www.w3.org/2001/XMLSchema#string"/>
</owl:FunctionalProperty>
<owl:FunctionalProperty rdf:about="#From">
  <rdfs:range rdf:resource="#Causa"/>
  <rdfs:range rdf:resource="#Defecto"/>
  <rdfs:range rdf:resource="#Canvas"/>
  <owl:inverseOf rdf:resource="#inverse_of_From"/>
  <rdfs:domain>
    <owl:Class>
      <owl:unionOf rdf:parseType="Collection">
        <rdf:Description rdf:about="http://www.w3.org/2002/07/owl#Thing"/>
        <owl:Class rdf:about="#Causa_Causa"/>
        <owl:Class rdf:about="#Causa_Defecto"/>
        <owl:Class rdf:about="#Canvas_Defecto"/>
      </owl:unionOf>
    </owl:Class>
  </rdfs:domain>
  <rdfs:label rdf:datatype="http://www.w3.org/2001/XMLSchema#string">From</rdfs:label>
  <rdf:type rdf:resource="http://www.w3.org/2002/07/owl#ObjectProperty"/>
</owl:FunctionalProperty>
<owl:FunctionalProperty rdf:ID="Actividad">
  <rdfs:domain>
    <owl:Class>
      <owl:unionOf rdf:parseType="Collection">
        <rdf:Description rdf:about="http://www.w3.org/2002/07/owl#Thing"/>
        <owl:Class rdf:about="#Defecto"/>
      </owl:unionOf>
    </owl:Class>
  </rdfs:domain>
  <rdfs:range rdf:resource="http://www.w3.org/2001/XMLSchema#string"/>
  <rdfs:label rdf:datatype="http://www.w3.org/2001/XMLSchema#string">Actividad</rdfs:label>
  <rdf:type rdf:resource="http://www.w3.org/2002/07/owl#DatatypeProperty"/>
</owl:FunctionalProperty>
<owl:FunctionalProperty rdf:about="#To">
  <owl:inverseOf rdf:resource="#inverse_of_To"/>
  <rdfs:label rdf:datatype="http://www.w3.org/2001/XMLSchema#string">To</rdfs:label>
  <rdf:type rdf:resource="http://www.w3.org/2002/07/owl#ObjectProperty"/>
  <rdfs:range rdf:resource="#Causa"/>
  <rdfs:range rdf:resource="#Defecto"/>
  <rdfs:domain>
    <owl:Class>
      <owl:unionOf rdf:parseType="Collection">

```



```

    <rdf:Description rdf:about="http://www.w3.org/2002/07/owl#Thing"/>
    <owl:Class rdf:about="#Causa_Causa"/>
    <owl:Class rdf:about="#Causa_Defecto"/>
    <owl:Class rdf:about="#Canvas_Defecto"/>
  </owl:unionOf>
</owl:Class>
</rdfs:domain>
</owl:FunctionalProperty>
<owl:FunctionalProperty rdf:ID="Tamaño_del_modulo">
  <rdfs:range rdf:resource="http://www.w3.org/2001/XMLSchema#string"/>
  <rdfs:domain>
    <owl:Class>
      <owl:unionOf rdf:parseType="Collection">
        <rdf:Description rdf:about="http://www.w3.org/2002/07/owl#Thing"/>
        <owl:Class rdf:about="#Defecto"/>
      </owl:unionOf>
    </owl:Class>
  </rdfs:domain>
  <rdfs:label rdf:datatype="http://www.w3.org/2001/XMLSchema#string">Tamaño del modulo
</rdfs:label>
  <rdf:type rdf:resource="http://www.w3.org/2002/07/owl#DatatypeProperty"/>
</owl:FunctionalProperty>
<owl:FunctionalProperty rdf:ID="Impacto">
  <rdf:type rdf:resource="http://www.w3.org/2002/07/owl#DatatypeProperty"/>
  <rdfs:range rdf:resource="http://www.w3.org/2001/XMLSchema#string"/>
  <rdfs:domain>
    <owl:Class>
      <owl:unionOf rdf:parseType="Collection">
        <rdf:Description rdf:about="http://www.w3.org/2002/07/owl#Thing"/>
        <owl:Class rdf:about="#Defecto"/>
      </owl:unionOf>
    </owl:Class>
  </rdfs:domain>
  <rdfs:label rdf:datatype="http://www.w3.org/2001/XMLSchema#string">Impacto</rdfs:label>
</owl:FunctionalProperty>
<owl:FunctionalProperty rdf:ID="Tipo_de_Analisis">
  <rdfs:label rdf:datatype="http://www.w3.org/2001/XMLSchema#string">Tipo de Analisis</rdfs:label>
  <rdfs:domain>
    <owl:Class>
      <owl:unionOf rdf:parseType="Collection">
        <rdf:Description rdf:about="http://www.w3.org/2002/07/owl#Thing"/>
        <owl:Class rdf:about="#Canvas"/>
      </owl:unionOf>
    </owl:Class>
  </rdfs:domain>
  <rdf:type rdf:resource="http://www.w3.org/2002/07/owl#DatatypeProperty"/>
  <rdfs:range rdf:resource="http://www.w3.org/2001/XMLSchema#string"/>
</owl:FunctionalProperty>
<owl:FunctionalProperty rdf:ID="Esfuerzo_de_detección">
  <rdfs:label rdf:datatype="http://www.w3.org/2001/XMLSchema#string">Esfuerzo de detección
</rdfs:label>
  <rdf:type rdf:resource="http://www.w3.org/2002/07/owl#DatatypeProperty"/>
  <rdfs:range rdf:resource="http://www.w3.org/2001/XMLSchema#float"/>
  <rdfs:domain>
    <owl:Class>
      <owl:unionOf rdf:parseType="Collection">
        <rdf:Description rdf:about="http://www.w3.org/2002/07/owl#Thing"/>
        <owl:Class rdf:about="#Defecto"/>
      </owl:unionOf>
    </owl:Class>
  </rdfs:domain>

```

```
</owl:FunctionalProperty>  
</rdf:RDF>
```